UNIVERSIDAD CATÓLICA SANTO TORIBIO DE MOGROVEJO

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



APLICACIÓN WEB BASADA EN DEEP LEARNING DE APOYO A LA CLASIFICACIÓN DE GRAVEDAD DEL ACNÉ VULGAR EN EL CONSULTORIO DERMEDIC

DOCUMENTO DE AVANCE DE TESIS
PARA EVIDENCIAR LA EJECUCIÓN DE TESIS
EN EL CURSO DE SEMINARIO DE TESIS I

AUTOR

Andree Requejo Díaz

ASESOR

Dr. Juan Elias Villegas Cubas

https://orcid.org/0000-0001-7026-9767

Chiclayo, 2025

INFORMACIÓN GENERAL

1. **Facultad y Escuela**

   Facultad de Ingeniería – Escuela de Ingeniería de Sistemas y Computación

2. **Título del informe de tesis**

   Aplicación web basada en deep learning de apoyo a la clasificación de gravedad del acné vulgar en el consultorio Dermedic.

3. **Autor(es) y firma(s)**

   Andree Requejo Díaz

   Autor

4. **Asesor y firma**

   Dr. Juan Elias Villegas Cubas

   Asesor

5. **Línea y área de la investigación**

   Desarrollo e innovación tecnológica – Ingeniería y Tecnología

6. **Fecha de presentación**

   26 de septiembre de 2025

ÍNDICE

I. RESULTADOS 4

1.1. EN BASE A LA METODOLOGÍA UTILIZADA 4

1.1.1. SPRINT INCEPTION 4

1.1.2. SPRINT #1 15

1.1.3. SPRINT #2 27

1.1.4. SPRINT #3 31

1.1.5. SPRINT #4 40

1.1.6. SPRINT #5 45

1.1.7. SPRINT #6 49

1.1.8. SPRINT #7 54

1.2. EN BASE A LOS OBJETIVOS DEL PROYECTO 65

1.2.1. DEFINIR EL ALGORITMO DE DEEP LEARNING MÁS ADECUADO PARA LA CLASIFICACIÓN DE LA GRAVEDAD DEL ACNÉ VULGAR A PARTIR DE IMÁGENES FACIALES MEDIANTE ANÁLISIS COMPARATIVO. 65

1.2.2. DESARROLLAR UN MODELO BASADO EN DEEP LEARNING PARA LA CLASIFICACIÓN AUTOMÁTICA DE LA GRAVEDAD DEL ACNÉ VULGAR EN IMÁGENES FACIALES. 66

1.2.3. DESARROLLAR UNA APLICACIÓN WEB RESPONSIVE QUE INTEGRE EL MODELO DE CLASIFICACIÓN DE GRAVEDAD DEL ACNÉ VULGAR, PARA FACILITAR SU USO POR PARTE DEL ESPECIALISTA. 68

1.2.4. EVALUAR EL RENDIMIENTO DEL SISTEMA DESARROLLADO MEDIANTE MÉTRICAS DE DESEMPEÑO COMO EXACTITUD, PRECISIÓN, SENSIBILIDAD Y F1-SCORE. 69

1.3. ANEXOS 71

1.3.1. ANEXO 01: 71

1.3.2. ANEXO 02: 75

1.3.3. ANEXO 03: 76

1.3.4. ANEXO 04: 77

I. RESULTADOS

1.1. En base a la metodología utilizada

Se empleó la metodología CRISP-DM, compuesta por seis fases, junto con el marco de trabajo SCRUM, con el propósito de desarrollar un software ágil capaz de ajustarse de manera flexible a los cambios en los requerimientos.

1.1.1. Sprint Inception

Inception	Inicio	Término	Épica	Historia	
Adquirir una comprensión clara del marco de trabajo SCRUM y del alcance del producto	Lun 18/08/2025	Vie 22/08/2025	Planificación inicial	1	Metodología del trabajo
				2	Desarrollo del product backlog
				3	Arquitectura del sistema
				4	Definir interfaces iniciales
				5	Base de datos

Desarrollo del Sprint Inception

- **Metodología de Trabajo**
  - ***Funcionamiento del marco de trabajo SCRUM***

Es fundamental comprender el marco de trabajo **SCRUM**, ya que permite gestionar el desarrollo de software de forma ágil y adaptable frente a cambios en los requerimientos o imprevistos durante la construcción de historias de usuario. Su aplicación se organiza en etapas principales:

- **Product Backlog:** Lista priorizada de requisitos y funciones del proyecto, organizada en épicas que se dividen en historias de usuario, cada una con un estado y una prioridad asignada.
- **Sprint Planning:** Sesión en la que se seleccionan y planifican las historias que se trabajarán en un periodo concreto, ajustándose con flexibilidad si el resultado difiere de lo previsto.

        - **Sprint Review:** Revisión de los avances del sprint, contrastando lo alcanzado con los objetivos iniciales y presentando los entregables.
        - **Sprint Retrospective:** Reflexión final orientada a reconocer aciertos y detectar mejoras para los próximos ciclos.

    - ***Integración de la metodología CRISP-DM***

    SCRUM, al ser un marco adaptable, puede integrarse con CRISP-DM para estructurar el desarrollo del modelo de deep learning orientado a la clasificación de la severidad del acné en imágenes. En este contexto, las seis fases de CRISP-DM guiarán el trabajo: desde la comprensión del problema y los datos, pasando por su preparación y el diseño del modelo, hasta la evaluación de resultados y el despliegue final de la solución.

- **Desarrollo del backlog**

    - ***Alcance del proyecto***

    El alcance del proyecto se centra en dar respuesta a la problemática que representa la evaluación clínica del acné vulgar, la cual suele ser un proceso subjetivo y lento, ya que depende tanto de la apreciación del especialista como del conteo manual de lesiones. Con el fin de optimizar este procedimiento, se plantea el desarrollo de una aplicación móvil que integre un modelo de deep learning capaz de analizar imágenes faciales y clasificar la severidad del acné de manera automatizada. El proyecto se encuentra alineado con los recursos disponibles, el tiempo definido y la viabilidad tecnológica, garantizando la posibilidad de alcanzar los objetivos propuestos. La solución busca complementar la labor del profesional médico, ofreciendo un soporte confiable que facilite el diagnóstico y seguimiento de los pacientes.

    - ***Elaborar el product backlog***

    Este apartado presenta el Product Backlog correspondiente al desarrollo de la aplicación web para la clasificación de la severidad del acné vulgar a partir de imágenes faciales. Dado que el proyecto se está desarrollando de manera individual y siguiendo el marco de trabajo Scrum, este backlog constituye la herramienta principal para organizar y priorizar las funcionalidades que se implementarán en cada sprint.

La construcción del backlog se ha realizado a partir de las necesidades planteadas por el problema de investigación, integrando tanto las fases de la metodología CRISP-DM como los módulos esenciales del sistema, entre ellos la gestión de pacientes, el procesamiento de imágenes y la visualización de resultados. Asimismo, se han definido prioridades y estimaciones de esfuerzo para cada elemento, lo que garantiza una planificación más eficiente y un desarrollo iterativo que pueda adaptarse a los ajustes y retroalimentación obtenida durante el proceso.

Sprint	Épica	Historias de Usuario
1	Seguridad y gestión de pacientes	- Iniciar sesión con credenciales (email y contraseña). - Registrar pacientes mediante dni, nombres, apellidos, edad y género. - Editar o eliminar registros de pacientes. - Visualizar las evaluaciones realizadas a un paciente.
2	Comprensión del negocio	- Determinar objetivos del negocio. - Valorar la situación. - Plan de proyecto.
3	Comprensión y preparación de los datos	- Recolectar datos iniciales. - Describir los datos. - Explorar los datos. - Verificar la calidad de los datos. - Seleccionar y limpiar los datos. - Formatear los datos.
4	Modelado	- Seleccionar técnica y evaluar el modelo. - Construir el modelo.
5	Evaluación	- Evaluar los resultados. - Determinar próximos pasos.
6	Despliegue	- Definir plan de despliegue.

7	Evaluación de gravedad e historial	- Tomar una foto con la cámara o subir una imagen desde el dispositivo. - Clasificar automáticamente la gravedad del acné. - Guardar un historial de evaluaciones por paciente. - Consultar el historial de cada paciente. - Generar un informe PDF con los resultados de las evaluaciones.

- ***Acordar revisiones***

  En este proyecto, las revisiones se organizarán conforme al cronograma de asesorías definido por el ingeniero responsable. La duración de los sprints podrá variar entre una y dos semanas, y al término de cada uno se realizará una evaluación por parte del asesor, quien validará el cumplimiento de las tareas y ofrecerá retroalimentación para la mejora del producto.

Item	Descripción / Fase	Duración (Días)	Comienzo	Fin	% Avance
01	Sprint Inception	5	18/08/2025	22/08/2025	50%
02	Sprint 1	15	22/08/2025	5/09/2025	
03	Sprint 2	8	5/09/2025	12/09/2025	
AVANCE AL 50%			12/09/2025	12/09/2025	
04	Sprint 3	15	15/09/2025	29/09/2025	100%
05	Sprint 4	22	29/09/2025	20/10/2025	
06	Sprint 5	8	20/10/2025	27/10/2025	
07	Sprint 6	8	27/10/2025	3/11/2025	
08	Sprint 7	15	3/11/2025	17/11/2025	
AVANCE AL 100%			21/11/2025	21/11/2025	
SUSTENTACION FINAL			5/12/2025	5/12/2025	100%

Figura 1 - Cronograma de actividades

- **Arquitectura del sistema**

En esta sección se describe la infraestructura tecnológica requerida para el proyecto, especificando los distintos componentes que la conforman y su papel en el desarrollo de la aplicación web.

- **Usuario:** El dermatólogo representa al usuario principal del sistema y es quien interactúa directamente con la plataforma para apoyar su labor clínica, este accede a la aplicación web mediante un navegador desde dispositivos como computadoras o celulares. Su rol es subir imágenes del rostro de los pacientes para obtener una clasificación automatizada del grado de acné. Además, puede consultar el

historial de evaluaciones realizadas, lo que le permite llevar un seguimiento clínico preciso. El sistema está diseñado para facilitar el diagnóstico, reduciendo el tiempo de análisis visual manual.

- **Dispositivos:** Se trata del equipo que el dermatólogo utiliza para acceder a la aplicación web, ya sea una computadora o teléfono, siempre desde un navegador web. A través de este dispositivo, se puede interactuar con la interfaz, cargar imágenes faciales de pacientes, visualizar los resultados proporcionados por el modelo y revisar los registros almacenados en la base de datos. La interfaz está optimizada para ofrecer una experiencia intuitiva y adaptable a distintas resoluciones de pantalla.
- **Cloud Services:** Es el entorno en la nube donde se aloja y ejecuta la aplicación web. Proporciona los recursos necesarios para que el sistema funcione de forma eficiente y escalable, incluyendo capacidad de procesamiento, almacenamiento y red. Gracias a esta infraestructura, el usuario puede acceder al sistema desde cualquier lugar con conexión a internet, garantizando disponibilidad, seguridad en el manejo de datos sensibles y facilidad para futuras mejoras en el sistema.
- **Aplicación Web:** Es la interfaz gráfica que permite al dermatólogo interactuar con el sistema. Está alojada en la nube y permite la interacción entre el usuario y los componentes del backend. A través de esta aplicación, el dermatólogo puede subir imágenes, evaluar una imagen, visualizar el resultado de la clasificación de gravedad del acné y acceder al historial de las evaluaciones de los pacientes. La aplicación maneja la comunicación con la lógica de negocio, el modelo de deep learning y la base de datos de forma interna y sin interrumpir la interacción del usuario. Esta aplicación se desarrollará utilizando *Laravel 12 junto Inertia React*.
- **Lógica de la Aplicación:** Es el componente central del backend y se encarga de gestionar y procesar todas las funcionalidades del sistema. Gestiona las solicitudes del dermatólogo, valida los datos ingresados, coordina el envío de imágenes al modelo de deep learning, interpreta los resultados y los guarda en la base de datos.

Además, establece las reglas de negocio necesarias para el funcionamiento correcto de la aplicación, como la gestión de sesiones, el control de acceso y la integridad de la información manejada.

- **Modelo Deep Learning:** Es un modelo de deep learning previamente entrenado con imágenes faciales clasificadas según el grado de gravedad del acné vulgar. Su función es recibir una imagen enviada por el dermatólogo, analizarla automáticamente y devolver una predicción sobre la gravedad (por ejemplo: leve, moderado o severo). Este modelo forma parte del sistema como componente de inteligencia artificial encargado de realizar la clasificación de manera automatizada.
- **Base de Datos:** Sistema de almacenamiento estructurado que guarda toda la información relacionada con el uso del sistema. Aquí se registran los datos del dermatólogo, las imágenes subidas, los resultados de las clasificaciones, y el historial de análisis por paciente. Esta base de datos permite acceder rápidamente a información previa, realizar consultas históricas y mantener un control de los diagnósticos realizados, siendo fundamental para el seguimiento de los pacientes.

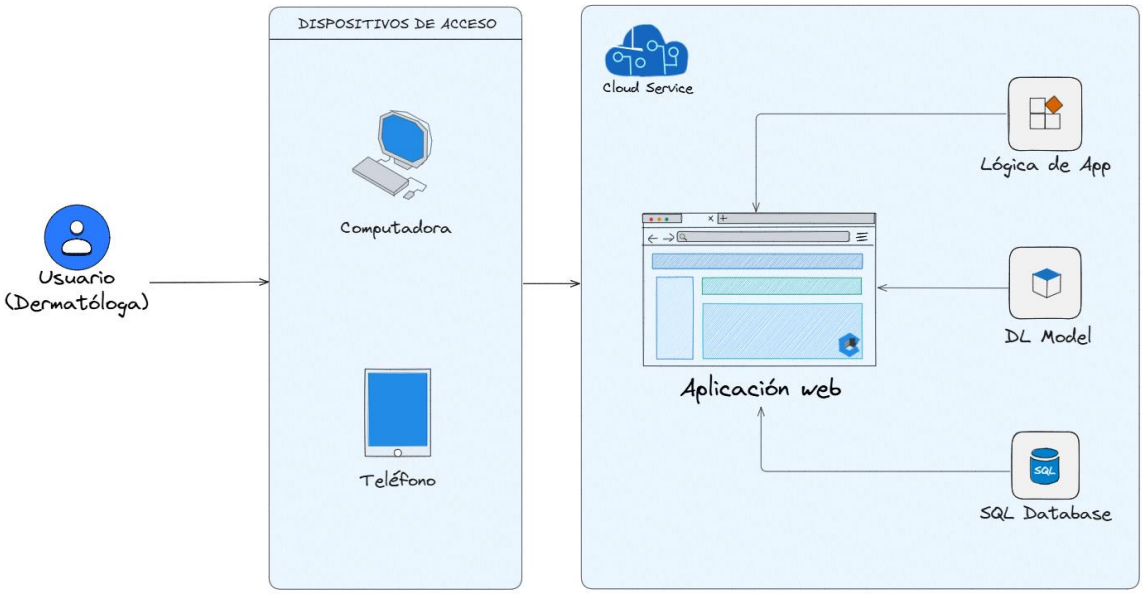


Figura 2 - Arquitectura del sistema

- **Definir interfaces iniciales**

- ***Identificar interfaces:***

  En esta actividad se identificarán las interfaces principales que requiere la aplicación para garantizar su correcto funcionamiento, atendiendo a los requerimientos definidos y a la problemática planteada. En esta etapa únicamente se busca establecer cuáles serán las interfaces necesarias, mientras que su descripción detallada se desarrollará en la actividad posterior.

  - Interfaz de inicio de sesión
  - Panel principal
  - Interfaz de gestión de pacientes
  - Interfaz de detalle de Paciente
  - Interfaz de historial de evaluaciones
  - Interfaz de detalle de evaluación
  - Interfaz para carga de imágenes
  - Interfaz de evaluación de gravedad de acné

- ***Descripción de interfaces y diseño de aplicación:***

  - **Interfaz de inicio de sesión:** Esta pantalla permitirá el acceso seguro al sistema mediante credenciales. El objetivo es garantizar que solo los usuarios autorizados puedan utilizar la aplicación, protegiendo así la información sensible de los pacientes.

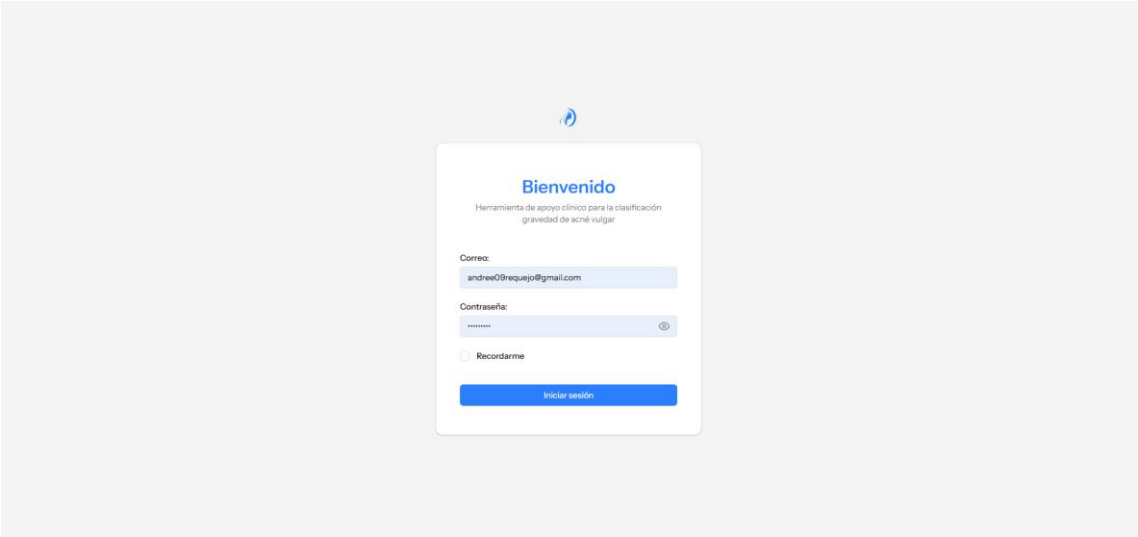


Figura 3 - Interfaz de inicio de sesión

- **Panel principal:** Corresponde a la vista inicial tras iniciar sesión. Presentará un resumen general del sistema y servirá como punto de acceso hacia las funcionalidades principales: gestión de pacientes, historial de evaluaciones y realización de una nueva evaluación.

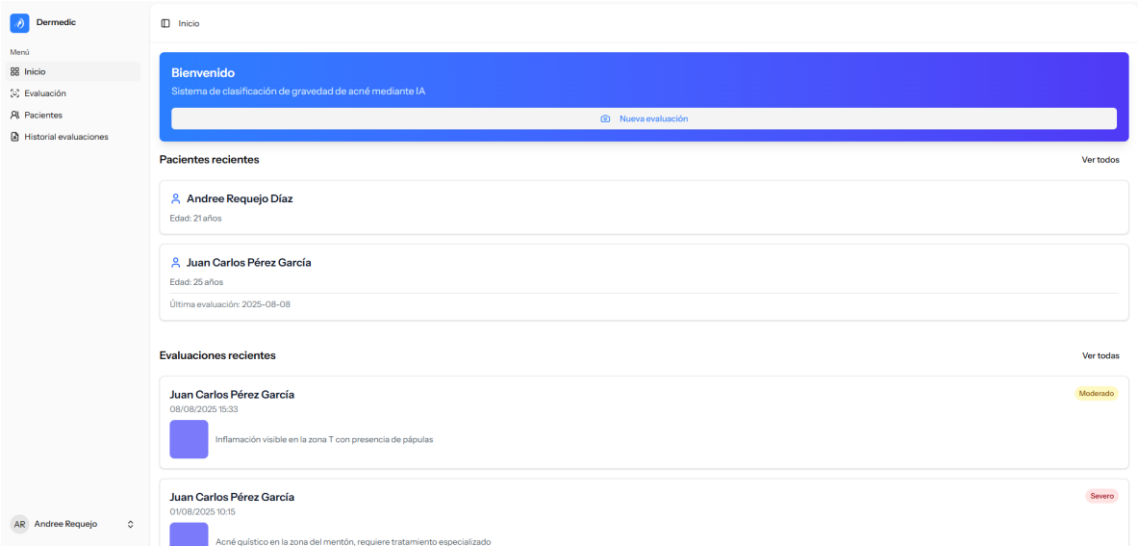


Figura 4- Panel principal

- **Interfaz de gestión de pacientes:** Permitirá registrar nuevos pacientes ingresando datos básicos como nombre, edad y género. Además, ofrecerá la opción de listar, editar o eliminar registros existentes, brindando una administración completa de la información de los pacientes.

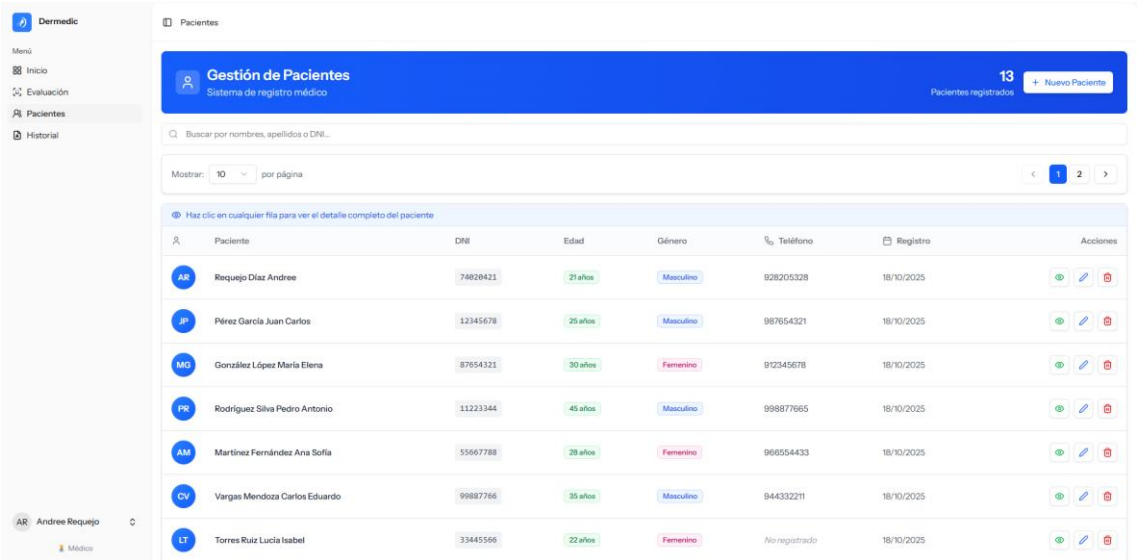


Figura 5 - Interfaz de gestión de pacientes

- **Interfaz de detalle de paciente:** Mostrará la información específica de un paciente en particular, incluyendo sus datos personales y el acceso a su historial de evaluaciones, facilitando la consulta individualizada.

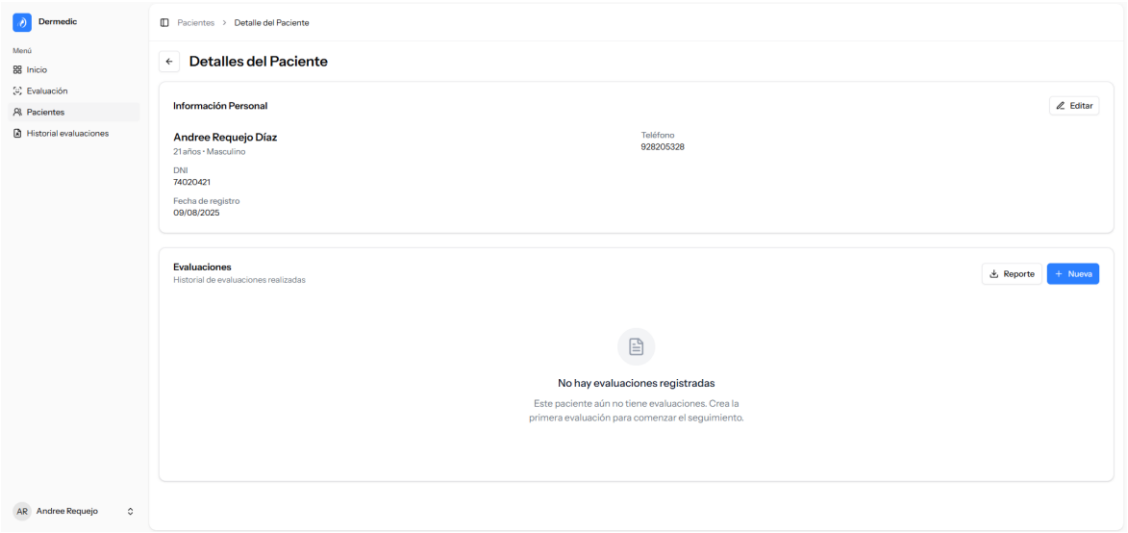


Figura 6 - Interfaz de detalle de Paciente

- **Interfaz de historial de evaluaciones:** Permitirá visualizar el conjunto de evaluaciones realizadas a un paciente. En ella se listarán las fechas, resultados y enlaces hacia el detalle de cada evaluación.

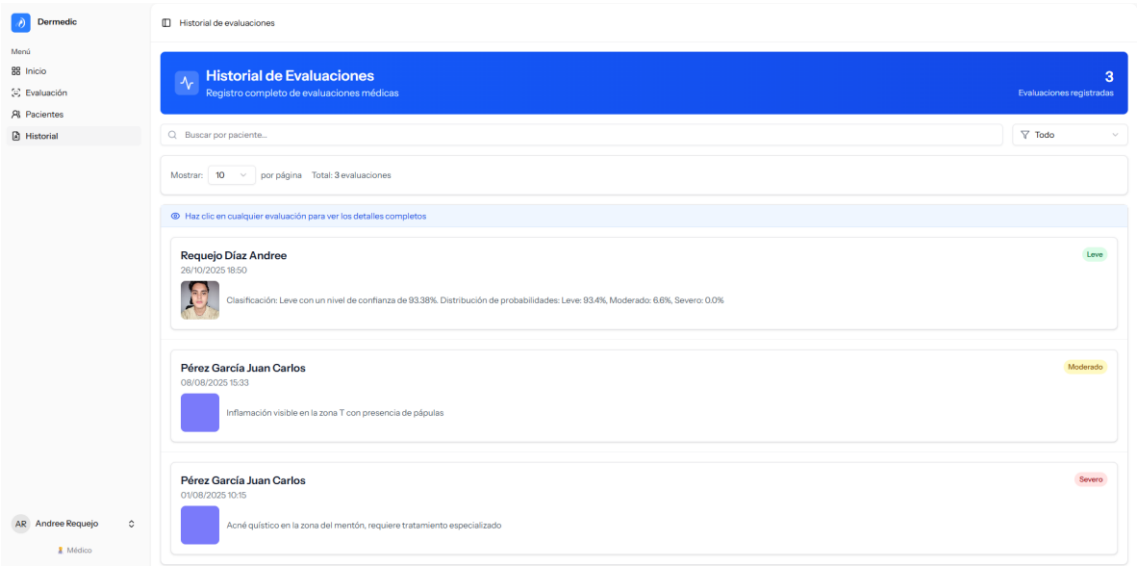


Figura 7 - Interfaz de historial de evaluaciones

- **Interfaz de detalle de evaluación:** Mostrará la información correspondiente a una evaluación específica, incluyendo la

imagen procesada, el resultado de la clasificación de severidad del acné y cualquier anotación asociada.

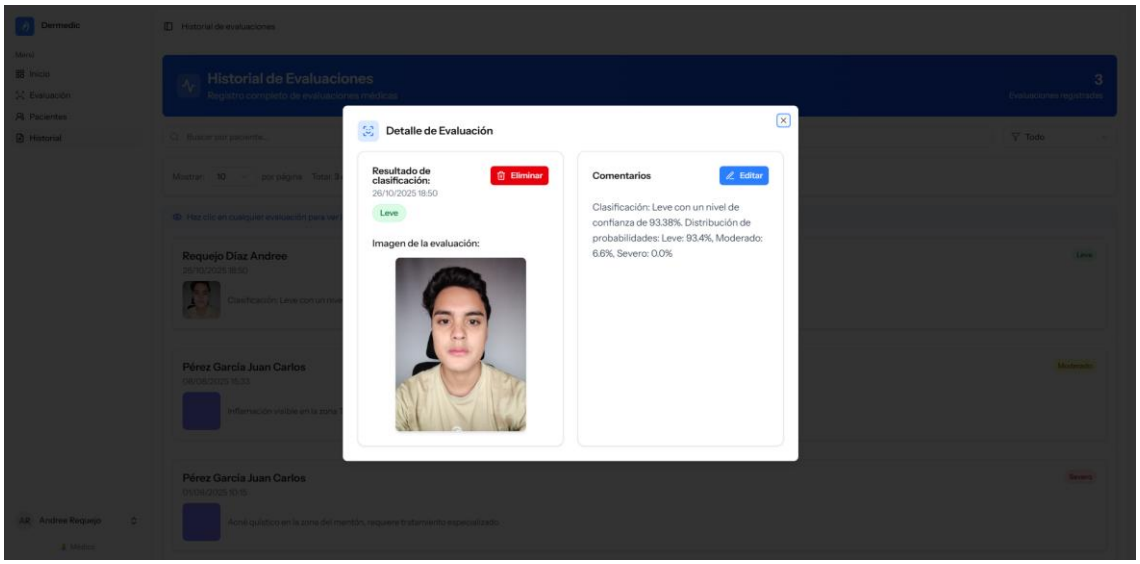


Figura 8 - Interfaz de detalle de evaluación

- **Interfaz para carga de imágenes:** Proporcionará la opción de subir imágenes faciales desde un dispositivo o capturarlas directamente mediante la cámara. Esta interfaz será la entrada principal al proceso de análisis con el modelo de deep learning.

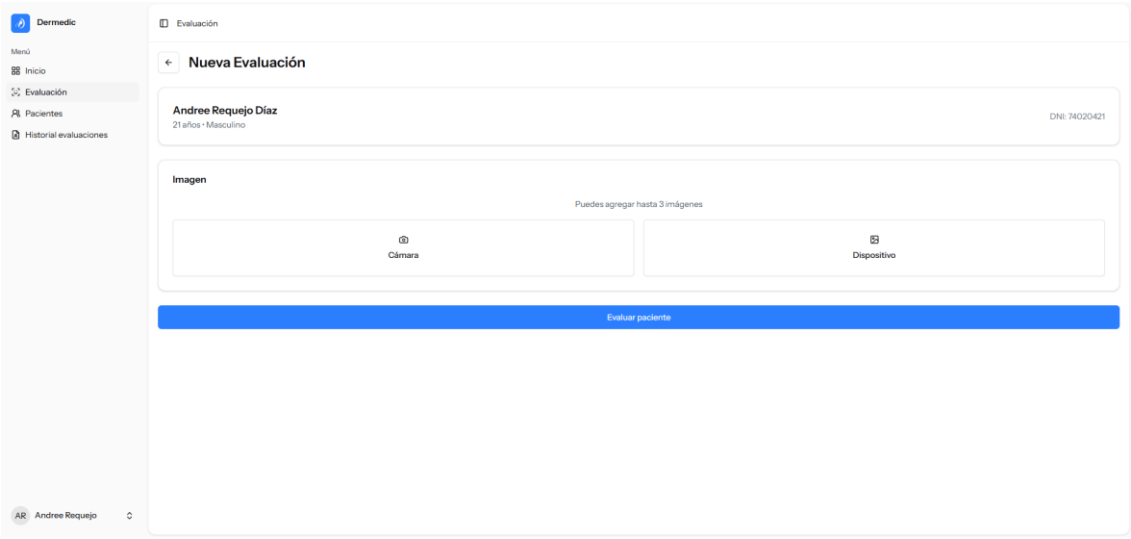


Figura 9 - Interfaz para carga de imágenes y evaluación

- **Interfaz de evaluación de gravedad de acné:** Procesará la imagen cargada y mostrará el resultado de la clasificación automática (leve, moderado o severo). Además, permitirá guardar la evaluación en el historial del paciente y asociarla a su registro clínico dentro de la aplicación.

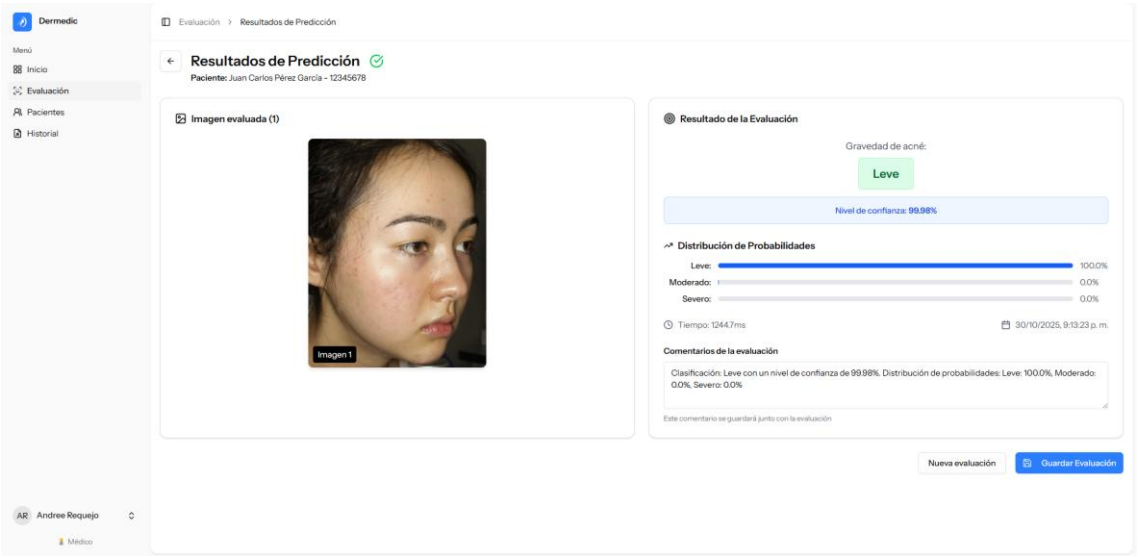


Figura 10 - Interfaz de resultados de predicción

- **Modelo físico de la base de datos**

Esta base de datos está diseñada para un consultorio dermatológico especializado en el diagnóstico y seguimiento de pacientes con acné. El sistema permite gestionar usuarios médicos, registrar pacientes, almacenar evaluaciones dermatológicas y procesar imágenes para predecir automáticamente la gravedad del acné mediante un sistema de clasificación. Así mismo, se permiten gestionar los roles dentro del consultorio para la admisión y la atención clínica.

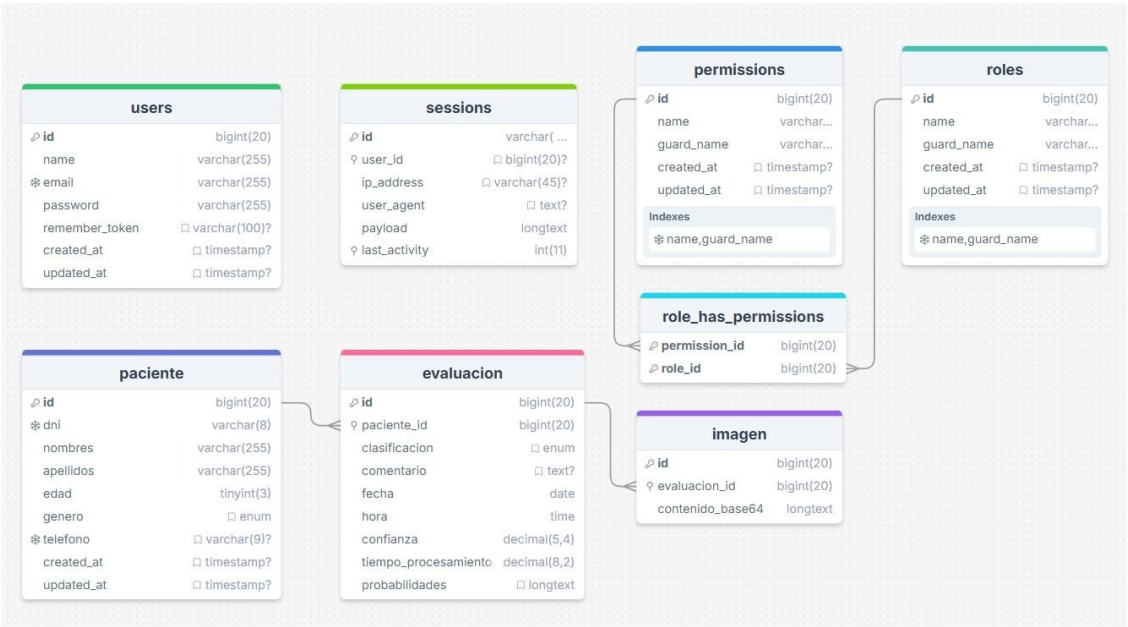


Figura 11 - Modelo físico de la BD

1.1.2. Sprint #1

Sprint	Inicio	Término	Épica	Historia	
SPRINT 1	Lun 22/08/2025	Vie 05/09/2025	Seguridad y gestión de pacientes	6	Inicio de sesión
				7	Listado de pacientes
				8	Registro de pacientes mediante dni, nombres, apellidos, edad y género.
				9	Edición y eliminación registros de pacientes.
				10	Visualización de las evaluaciones realizadas a un paciente.

Desarrollo del Sprint 1

En este sprint se implementarán las funcionalidades básicas de seguridad y gestión de pacientes. Se desarrollará el módulo de inicio de sesión con credenciales seguras y el registro de pacientes con sus datos principales. Asimismo, se habilitarán las opciones para editar o eliminar registros existentes y se incorporará la visualización del detalle del paciente dónde se observarán sus evaluaciones asociadas.

- **Inicio de sesión:**

Objetivo: Implementar el módulo de inicio de sesión para permitir que los usuarios accedan de forma segura al sistema mediante sus credenciales registradas.

  - *Funciones implementadas:*

La función de reglas define las reglas de validación que se tendrán en cuenta al momento de un inicio de sesión, los cuales son: el correo electrónico del usuario correspondiente y la contraseña asignada a este mismo, si estos se encuentran vacíos al momento de intentar iniciar sesión el logueo fallará.



```php
/**
     * Reglas de validación que se aplican a la petición.
     *
     * Este método define las reglas de validación para la solicitud de inicio de sesión:
     * - El campo "email" es obligatorio, debe ser una cadena de texto y tener formato de correo electrónico válido.
     * - El campo "password" es obligatorio y debe ser una cadena de texto.
     *
     * Estas reglas se ejecutan automáticamente antes del proceso de autenticación.
     * Si los datos no cumplen con estas condiciones, la validación fallará
     * y no se intentará autenticar al usuario.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }
```

Figura 12 - Función reglas de autenticación

La siguiente función es muy importante ya que limita la cantidad de intentos de inicio de sesión que un usuario puede intentar de manera consecutiva. Esto busca como objetivo evitar ataques de fuerza bruta. Si supera el límite de 5 intentos, el email con el que se intenta acceder se bloqueará durante 60 segundos.



```php
/**
     * Verificar que la solicitud de inicio de sesión no esté limitada por intentos.
     *
     * Este método protege contra ataques de fuerza bruta:
     * - Comprueba si se superó el número máximo de intentos permitidos (en este caso 5).
     * - Si no se superó el límite, la función simplemente termina y permite continuar.
     * - Si se superó el límite:
     *   - Dispara el evento Lockout, que puede usarse para registrar logs o disparar alertas.
     *   - Obtiene el tiempo restante (en segundos) que el usuario debe esperar antes
     *     de volver a intentar iniciar sesión.
     *   - Lanza una excepción de validación con un mensaje de error indicando
     *     cuánto tiempo debe esperar.
     *
     * Con esto, Laravel bloquea temporalmente el login cuando hay demasiados intentos fallidos.
     *
     */
    public function ensureIsNotRateLimited(): void
    {
        if (! RateLimiter::tooManyAttempts($this->throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));

        $seconds = RateLimiter::availableIn($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => __('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds / 60),
            ]),
        ]);
    }
```

Figura 13 - Función que verifica intentos de fuerza bruta

La siguiente función es la que permite la autenticación de un usuario, a través de esta se validará que tanto el correo electrónico y la contraseña ingresada sean las correctas. Si

falla el logueo se mostrará un mensaje al usuario indicando que las credenciales ingresadas son incorrectas y de igual forma, se irá almacenando la cantidad de intentos erróneos con el fin de lanzar la función de bloqueo anterior, para evitar ataques de fuerza bruta.



```php
/**
     * Autenticar las credenciales de la petición.
     *
     * Este método maneja el proceso principal del inicio de sesión:
     * 1. Primero, verifica que el usuario no haya superado el número máximo
     *    de intentos permitidos de inicio de sesión (esto previene ataques de fuerza bruta).
     *    Si se excedió el límite, se lanza una excepción de bloqueo.
     *
     * 2. Luego intenta autenticar al usuario usando los datos "email" y "password"
     *    a través de Auth::attempt().
     *    - Si es exitoso: el usuario queda autenticado en la sesión y se limpia el contador
     *      de intentos fallidos.
     *    - Si falla: se incrementa el contador de intentos fallidos en el RateLimiter
     *      y se lanza una excepción de validación con un mensaje de error.
     *
     * De esta forma, se garantiza que solo los usuarios con credenciales correctas
     * puedan acceder al sistema.
     *
     */
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => 'Estas credenciales no coinciden con nuestros registros.',
            ]);
        }

        RateLimiter::clear($this->throttleKey());
    }
```

Figura 14 - Función de inicio de sesión

- *Integración con el backend:*

Esta función maneja el envío de los valores que el usuario ingresa dentro del formulario de login. Cuando el usuario hace clic en "Iniciar sesión", esta función envía los datos (email, password y remember) mediante una petición POST a la ruta /login. Si las credenciales son correctas da pase a la ruta de /dashboard, caso contrario muestra un mensaje de error indicando que las credenciales ingresadas no son correctas.



```
const { data, setData, post, processing, errors, reset } = useForm<Required<LoginForm>>({
        email: '',
        password: '',
        remember: false,
    });
        const [showPassword, setShowPassword] = useState(false);

    const submit: FormEventHandler = (e) => {
        e.preventDefault();
        post(route('login'), {
            onFinish: () => reset('password'),
        });
    };
```

Figura 15 - Uso de funciones de autenticación en frontend

- *Funcionalidad en uso:*

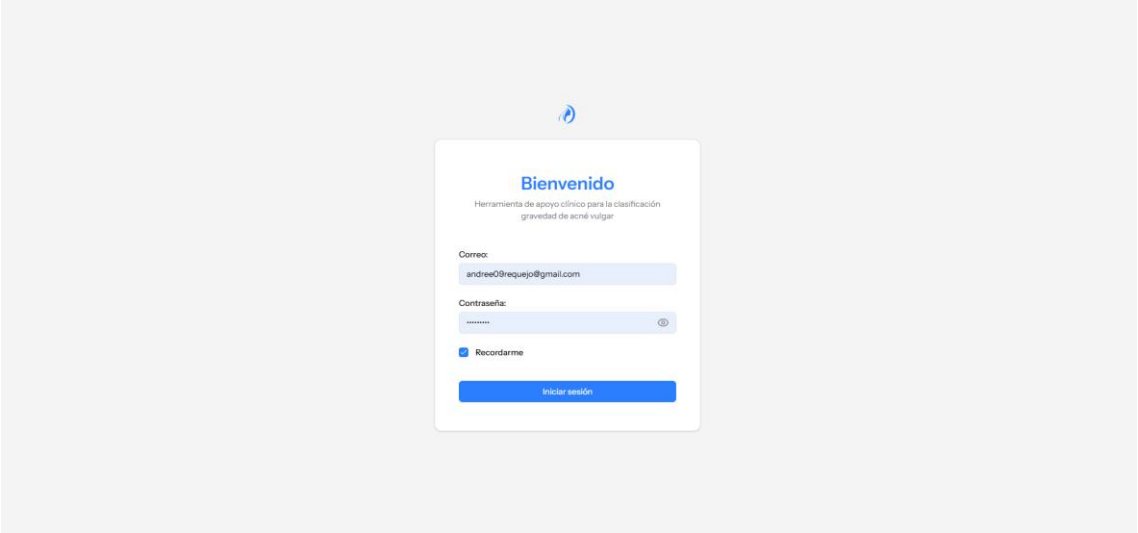


Figura 16 - Interfaz de inicio de sesión con credenciales

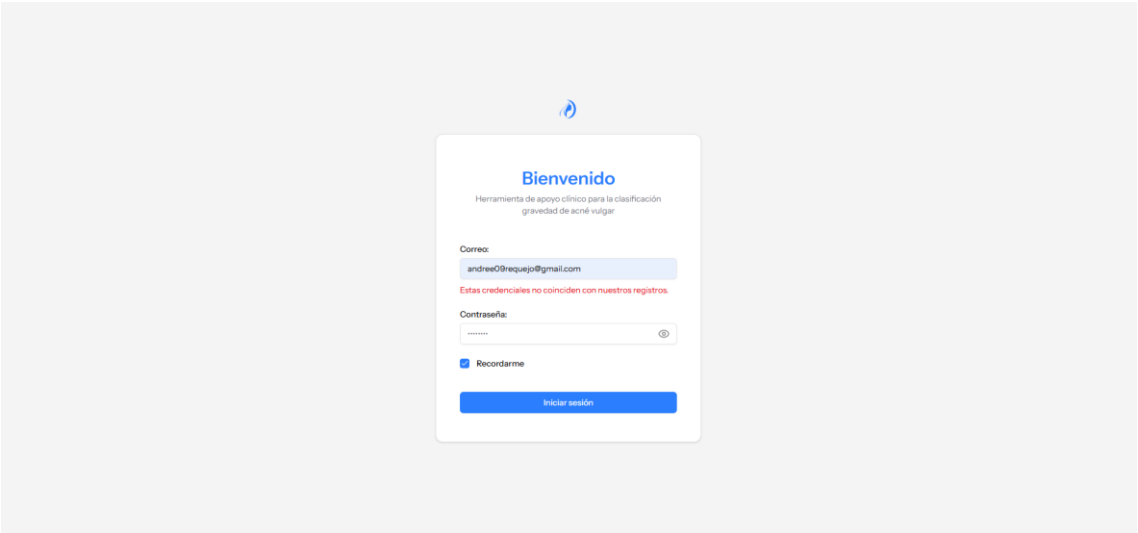


Figura 17 - Interfaz de inicio de sesión con credenciales incorrectas

- **Listado de pacientes:**

Objetivo: Implementar una vista que muestre el listado de pacientes registrados, permitiendo su consulta de forma rápida y organizada.

- *Funciones implementadas:*

Esta función maneja la visualización de la lista de pacientes en el sistema. Cuando se accede a esta página, la función consulta todos los pacientes registrados en la base de datos. Si el usuario utiliza el campo de búsqueda, la función filtra los resultados buscando coincidencias en los nombres, apellidos y DNI de los pacientes. Luego, ordena los resultados por fecha de creación de forma descendente (los más recientes primero) y los divide en páginas mostrando 6 pacientes por página. Finalmente, renderiza la vista 'patients/index' enviando la lista de pacientes paginados y los filtros de búsqueda aplicados.

```php
public function index(Request $request)
    {
        $query = Paciente::query();

        // Búsqueda
        if ($request->has('search') && $request->search) {
            $search = $request->search;
            $query->where(function ($q) use ($search) {
                $q->where('nombres', 'like', "%{$search}%")
                  ->orWhere('apellidos', 'like', "%{$search}%")
                  ->orWhere('dni', 'like', "%{$search}%");
            });
        }

        // Paginación
        $pacientes = $query->orderBy('created_at', 'desc')->paginate(6);

        return Inertia::render('patients/index', [
            'pacientes' => $pacientes,
            'filters' => $request->only(['search'])
        ]);
    }
```

Figura 18 - Función listado de pacientes

- *Funcionalidad en uso:*

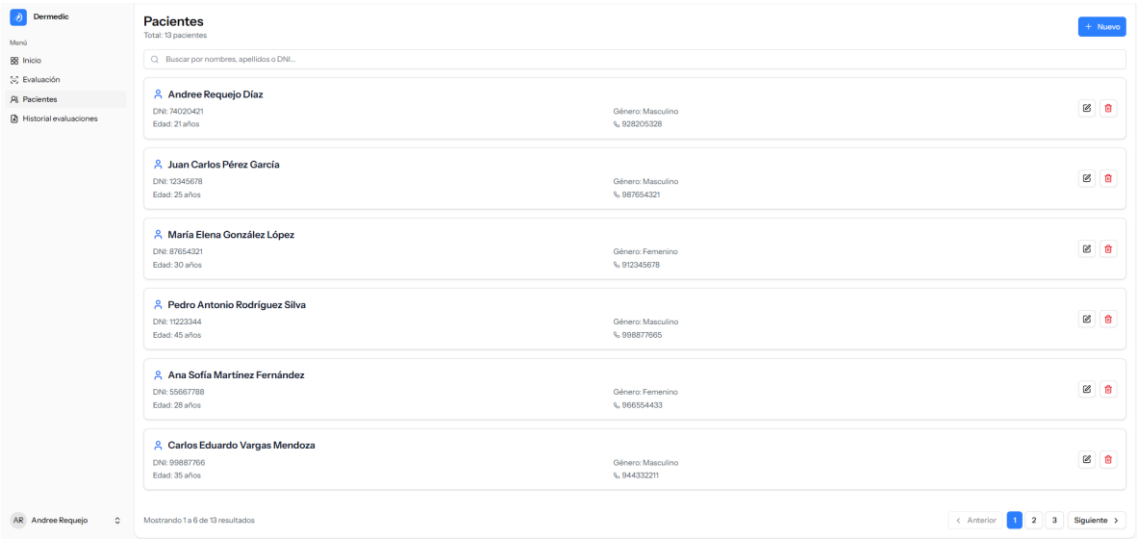


Figura 19 - Interfaz de pacientes con paginación, filtros y búsqueda

- **Registro de pacientes mediante dni, nombres, apellidos, edad y género:**

Objetivo: Desarrollar la funcionalidad de registro de pacientes para almacenar datos personales como DNI, nombres, apellidos, edad y género.

- *Funciones implementadas:*

Esta función maneja el registro de un nuevo paciente en el sistema. Cuando el usuario completa el formulario de creación y hace clic en "Guardar", la función valida que todos los campos requeridos cumplan con las reglas establecidas. Si todos los datos son válidos, crea el nuevo paciente en la base de datos y redirige a la lista de pacientes mostrando un mensaje de éxito "Paciente creado exitosamente". Si hay errores en la validación, muestra los mensajes correspondientes en el formulario.

```php
public function store(Request $request)
    {
        $validated = $request->validate([
            'dni' => 'required|string|max:8|unique:paciente,dni',
            'nombres' => 'required|string|max:255',
            'apellidos' => 'required|string|max:255',
            'edad' => 'required|integer|min:0|max:150',
            'genero' => 'required|in:Masculino,Femenino',
            'telefono' => 'nullable|string|max:15',
        ]);

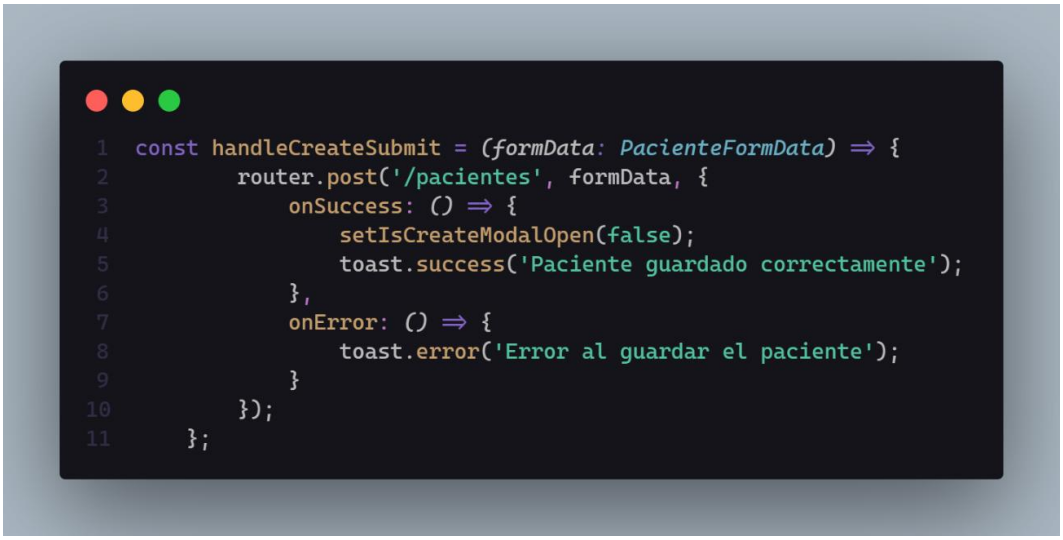
        $paciente = Paciente::create($validated);

        return redirect()->route('pacientes.index')->with('success', 'Paciente creado exitosamente.');
    }
```

Figura 20 - Función de creación de pacientes

- *Integración con el backend:*

Esta función maneja el envío de los datos del formulario de creación de paciente. Cuando el usuario completa el formulario y hace clic en "Guardar", la función envía los datos ingresados mediante una petición POST a la ruta '/pacientes'. Si el paciente se guarda correctamente en la base de datos, cierra el modal de creación y muestra un mensaje de éxito. Si ocurre algún error durante el proceso de guardado, muestra un mensaje de error indicando que no se pudo completar la operación.



```
const handleCreateSubmit = (formData: PacienteFormData) => {
        router.post('/pacientes', formData, {
            onSuccess: () => {
                setIsCreateModalOpen(false);
                toast.success('Paciente guardado correctamente');
            },
            onError: () => {
                toast.error('Error al guardar el paciente');
            }
        });
    };
```

Figura 21 - Integración en frontend para creación de pacientes

- *Funcionalidad en uso:*

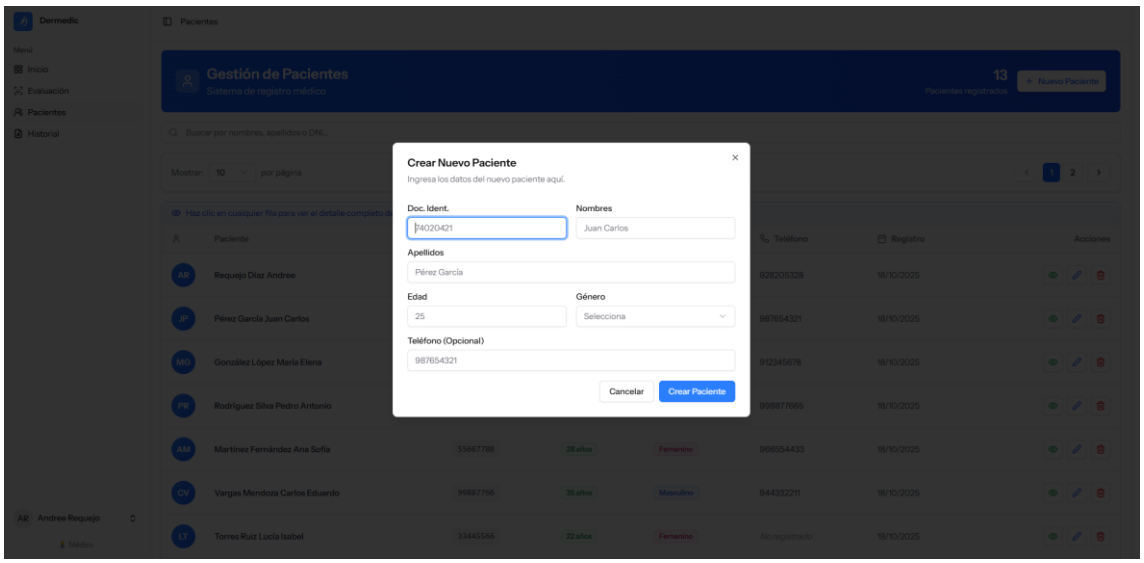


Figura 22 - Modal de creación de pacientes

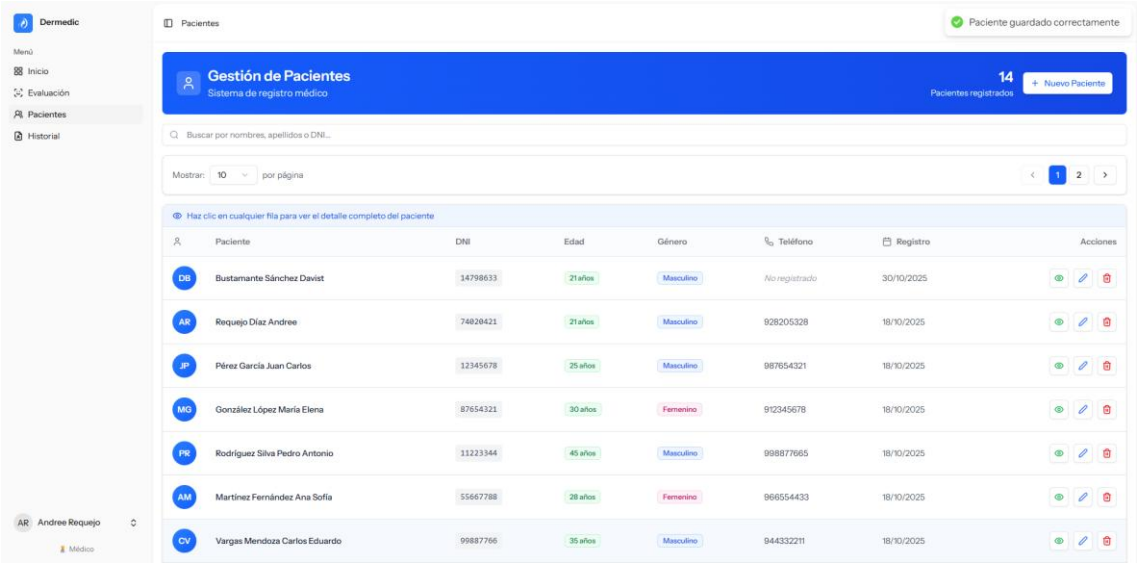


Figura 23 - Notificación de éxito en creación de paciente

- **Edición y eliminación de registros de pacientes:**

Objetivo: Incorporar opciones de edición y eliminación que permitan mantener actualizada la información de los pacientes registrados.

- *Funciones implementadas:*

Esta función maneja la actualización de los datos de un paciente existente. Cuando el usuario modifica la información en el formulario de edición y hace clic en "Actualizar", la función valida que todos los campos cumplan con las reglas establecidas. Si los datos son válidos, actualiza la información del paciente en la base de datos. Si la petición viene desde la página de detalle del paciente, simplemente retorna a esa página mostrando un mensaje de éxito. Si viene desde la lista de pacientes, redirige al índice con el mismo mensaje de éxito.

```php
public function update(Request $request, Paciente $paciente)
    {
        $validated = $request->validate([
            'dni' => 'required|string|max:8|unique:paciente,dni,' . $paciente->id,
            'nombres' => 'required|string|max:255',
            'apellidos' => 'required|string|max:255',
            'edad' => 'required|integer|min:0|max:150',
            'genero' => 'required|in:Masculino,Femenino',
            'telefono' => 'nullable|string|max:15',
        ]);

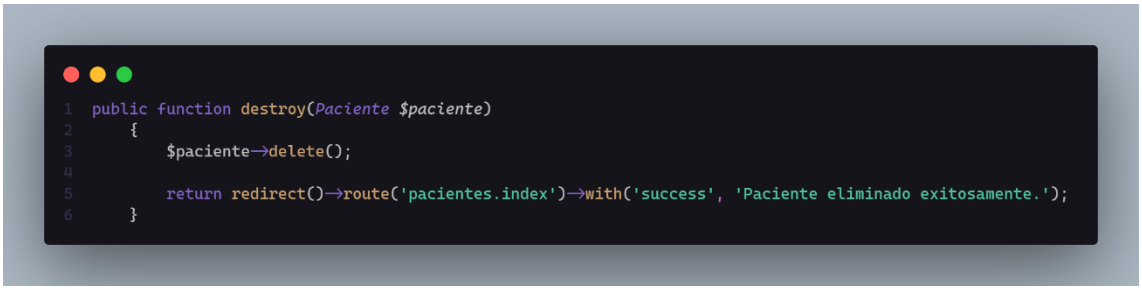
        $paciente->update($validated);

        // Si la petición viene desde la página de detalle, devolver los datos actualizados
        if ($request->header('X-Inertia')) {
            return back()->with('success', 'Paciente actualizado exitosamente.');
        }

        return redirect()->route('pacientes.index')->with('success', 'Paciente actualizado exitosamente.');
    }
```

Figura 24 - Función de edición de pacientes

Esta función maneja la eliminación de un paciente del sistema. Cuando el usuario confirma que desea eliminar un paciente, la función procede a eliminarlo permanentemente de la base de datos. Una vez completada la eliminación, redirige al usuario a la lista de pacientes mostrando un mensaje de éxito para confirmar que la operación se realizó correctamente.



```php
public function destroy(Paciente $paciente)
    {
        $paciente->delete();

        return redirect()->route('pacientes.index')->with('success', 'Paciente eliminado exitosamente.');
    }
```

Figura 25 - Función de eliminación de pacientes

- *Integración con el backend:*

Esta función maneja el envío de los datos del formulario de edición de paciente. Cuando el usuario modifica la información de un paciente y hace clic en "Actualizar", la función verifica que haya un paciente seleccionado y envía los datos actualizados mediante una petición PUT a la ruta '/pacientes/{id}'. Si la edición se completa exitosamente, cierra el modal de edición, limpia la selección del paciente y muestra un mensaje de éxito. Si ocurre algún error durante el proceso, muestra un mensaje de error indicando que no se pudo completar la operación.



```tsx
const handleEditSubmit = (formData: PacienteFormData) => {
        if (selectedPatient) {
            router.put(`/pacientes/${selectedPatient.id}`, formData, {
                preserveState: true,
                onSuccess: () => {
                    setIsEditModalOpen(false);
                    setSelectedPatient(null);
                    toast.success('Paciente editado correctamente');
                },
                onError: () => {
                    toast.error('Error al editar el paciente');
                }
            });
        }
    };
```

Figura 26 - Integración en frontend para edición de pacientes

Esta función maneja la confirmación de eliminación de un paciente. Cuando el usuario confirma que desea eliminar un paciente del sistema, la función verifica que haya un paciente seleccionado y envía una petición DELETE a la ruta '/pacientes/{id}'. Si la eliminación se realiza correctamente, cierra el modal de confirmación, limpia la selección del paciente y muestra un mensaje de éxito. Si ocurre algún error durante el proceso de eliminación, muestra un mensaje de error indicando que no se pudo completar la operación.



```
const handleDeleteConfirm = () => {
        if (selectedPatient) {
            router.delete(`/pacientes/${selectedPatient.id}`, {
                onSuccess: () => {
                    setIsDeleteModalOpen(false);
                    setSelectedPatient(null);
                    toast.success('Paciente eliminado correctamente');
                },
                onError: () => {
                    toast.error('Error al eliminar el paciente');
                }
            });
        }
    };
```

Figura 27 - Integración en frontend para eliminación de pacientes

- *Funcionalidad en uso:*

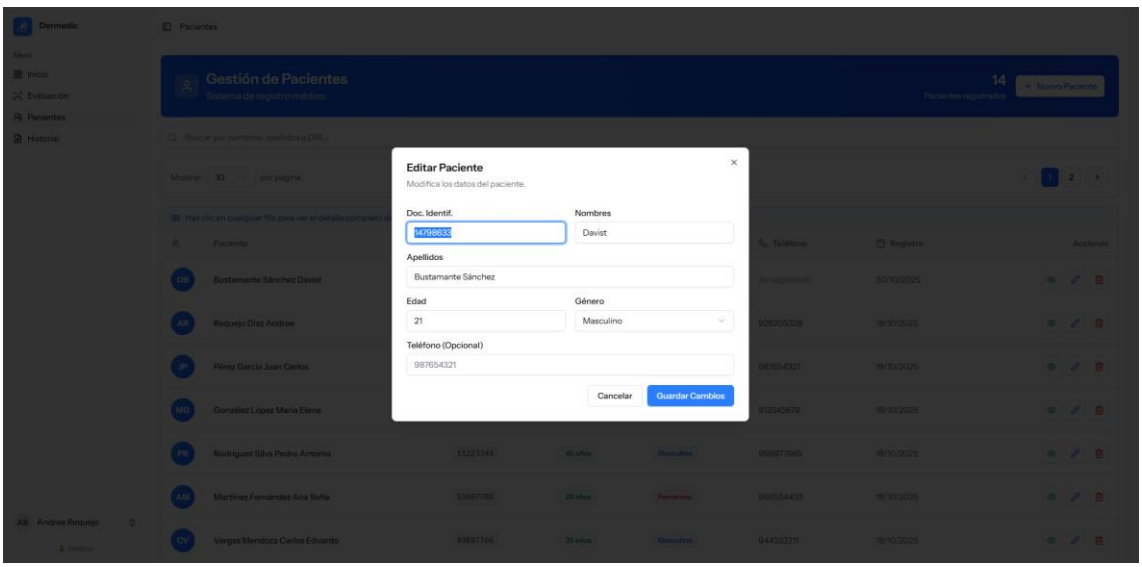


Figura 28 - Modal de edición de pacientes

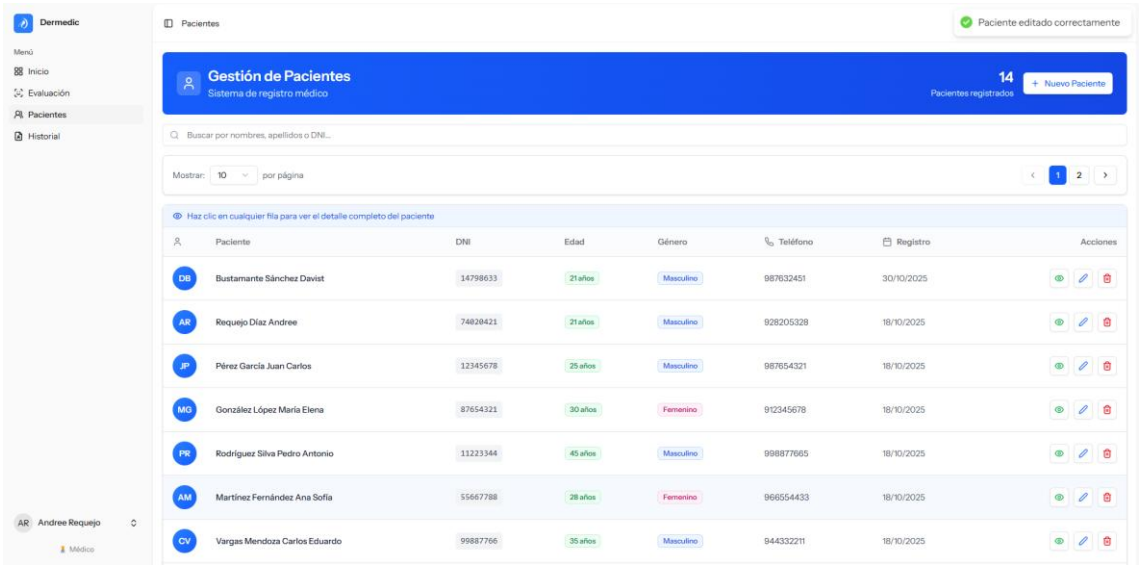


Figura 29 - Notificación de éxito de edición de paciente

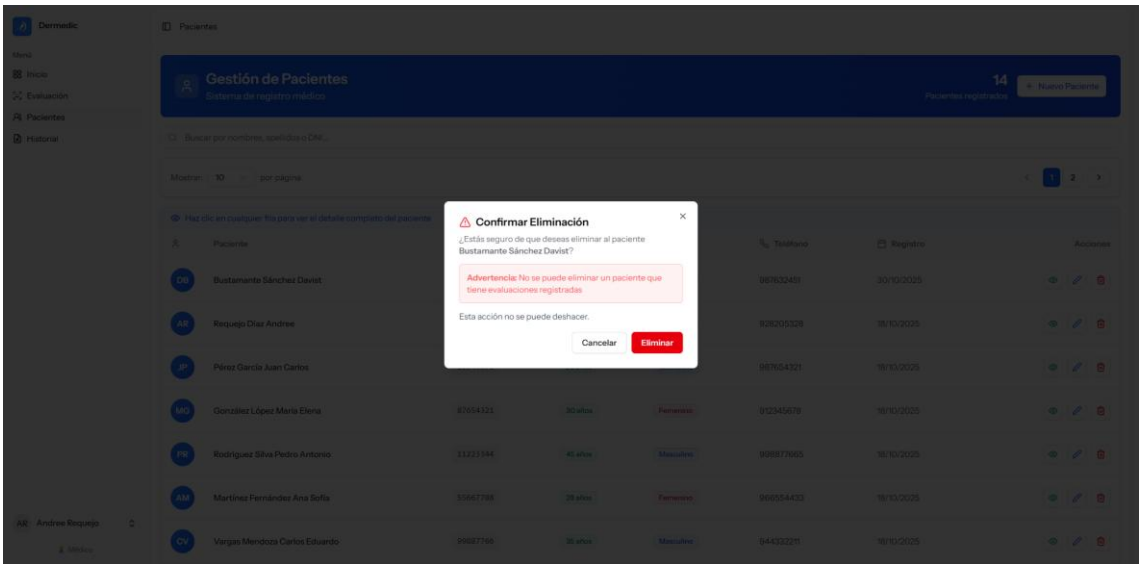


Figura 30 - Modal de eliminación de paciente

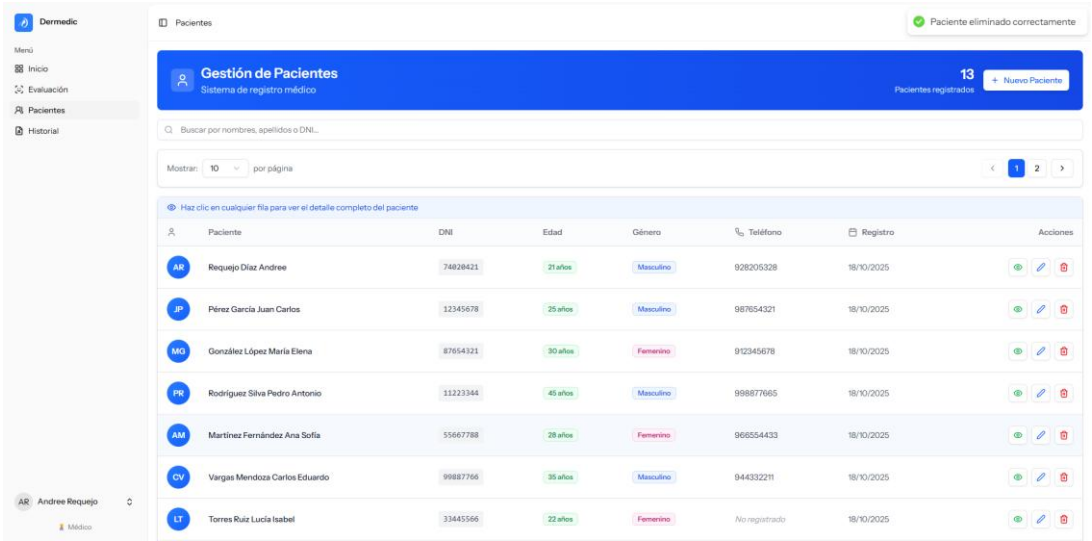


Figura 31 - Notificación de éxito de eliminación de paciente

- **Visualización de las evaluaciones realizadas a un paciente:**

Objetivo: Implementar la visualización de las evaluaciones asociadas a cada paciente para facilitar su consulta.

- *Funciones implementadas:*

Esta función maneja la visualización de los detalles de un paciente específico. Cuando el usuario accede a la información de un paciente, la función carga los datos completos del paciente incluyendo sus evaluaciones con sus imágenes principales asociadas. Luego renderiza la vista 'patients/detalle' mostrando toda la información detallada del paciente seleccionado junto con el historial de sus evaluaciones.



```
public function show(Paciente $paciente)
    {
        return Inertia::render('patients/detalle', [
            'paciente' => $paciente->load(['evaluaciones.imagenPrincipal'])
        ]);
    }
```

Figura 32 - Función de visualización de evaluaciones de paciente

- *Funcionalidad en uso:*

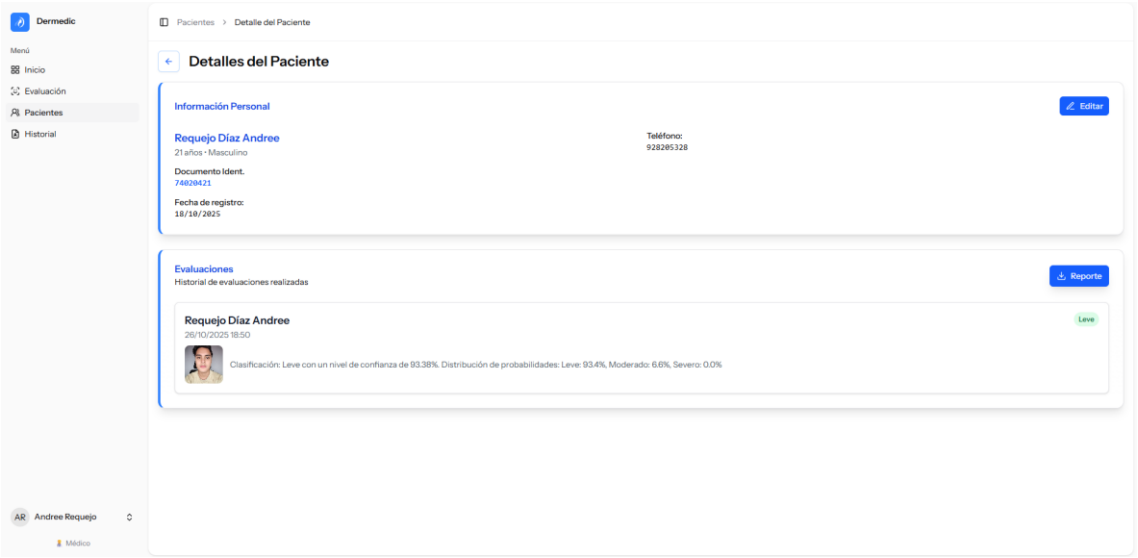


Figura 33 - Interfaz de detalle de paciente y evaluaciones

Sprint Review 1:

Épica	Proceso	Enunciado de la historia	Estado

Seguridad y gestión de pacientes	Autenticación	Como dermatólogo, quiero iniciar sesión con credenciales seguras con la finalidad de acceder al sistema únicamente si estoy autorizado.	Finalizada
	Gestión de pacientes	Como dermatólogo, quiero visualizar el listado de pacientes que se han registrado con la finalidad de gestionar fácilmente la información de cada uno.	Finalizada
		Como dermatólogo, quiero registrar pacientes con su dni, nombres, apellidos, edad y género con la finalidad de contar con un historial clínico organizado y accesible.	Finalizada
		Como dermatólogo, quiero editar o eliminar registros de pacientes con la finalidad de mantener actualizada y precisa la información del consultorio.	Finalizada
		Como dermatólogo, quiero visualizar las evaluaciones realizadas a un paciente con la finalidad de dar seguimiento a la evolución de su condición.	Finalizada

1.1.3. Sprint #2

Sprint	Inicio	Término	Épica	Historia	
SPRINT 2 Cumplir con las actividades de la fase 1 de CRISP-DM	Lun 05/09/2025	Vie 12/09/2025	Comprensión del negocio	11	Determinar objetivos del negocio
				12	Valoración de la situación
				13	Plan de proyecto

Desarrollo del Sprint 2:

Para el desarrollo de esta fase se realizó una visita al consultorio Dermedic, donde se recopilaron detalles sobre el proceso actual de diagnóstico de la

gravedad del acné mediante entrevistas con la dermatóloga responsable. Además, se revisó documentación institucional y una guía práctica clínica utilizada en el servicio de atención dermatológica. Estas actividades permitieron comprender el contexto clínico, identificar las principales dificultades que se enfrenta al clasificar las lesiones y reconocer la necesidad de una herramienta tecnológica que optimice y agilice dicho procedimiento.

- **Determinar objetivos del negocio:**

En esta etapa se identificaron las metas principales del proyecto, orientadas a apoyar el diagnóstico clínico de la severidad del acné mediante una aplicación web. El objetivo central es reducir la subjetividad y el tiempo que demanda el conteo manual de lesiones, ofreciendo una herramienta que clasifique automáticamente el grado de afectación de acné de un paciente mediante imágenes faciales. Con ello, se busca brindar soporte al especialista y mejorar la eficiencia en la atención médica.

  - *Objetivo general:*
    - Desarrollar una solución de apoyo a la clasificación de la severidad del acné, reduciendo el tiempo de evaluación y la subjetividad del diagnóstico clínico.
  - *Objetivos específicos:*
    - Diseñar un sistema de gestión de pacientes que permita registrar, consultar y dar seguimiento a las evaluaciones realizadas.
    - Proporcionar un historial digital de cada paciente con el fin de mejorar la continuidad en el tratamiento.
    - Generar informes de evaluación en formato digital que faciliten la comunicación de resultados.
    - Disminuir la dependencia del conteo manual de lesiones o la experiencia profesional, optimizando la labor del especialista.

- **Valoración de la situación:**

  - *Problemática:*

  El diagnóstico de la severidad del acné se basa principalmente en el recuento de lesiones cutáneas presentes en cada paciente. Si bien este procedimiento constituye el criterio objetivo de clasificación, en la práctica depende en gran medida de la experiencia del especialista. Un dermatólogo con mayor trayectoria puede realizar la evaluación más

rápidamente, pero este proceso sigue siendo subjetivo y varía entre profesionales. Como resultado, se generan diferencias en la clasificación y dificultades para mantener un seguimiento uniforme de la evolución del paciente.

- *Limitaciones en el acceso a datos clínicos:*

Un desafío importante es el acceso limitado a datos clínicos etiquetados, en particular a imágenes dermatológicas que incluyan un diagnóstico validado por especialistas. En el Perú, no existe un sistema organizado que centralice ni gestione este tipo de información, lo que dificulta contar con bases de datos amplias y estandarizadas para el entrenamiento de modelos de deep learning.

- *Riesgos técnicos:*

Entre los riesgos identificados se encuentran la posible baja precisión del modelo en imágenes con variaciones de luz, diferentes tipos de piel o niveles de severidad intermedios.

- **Plan de proyecto:**

Con base en los objetivos y en la situación diagnosticada, se definió un plan de trabajo estructurado en sprints, siguiendo la metodología Scrum en conjunto con CRISP-DM. Este plan establece las fases de comprensión, preparación, modelado, evaluación y despliegue, distribuyendo tareas en periodos cortos para permitir avances incrementales.

Fase	Tiempo estimado (días)	Actividades
Comprensión del negocio	6	▪ Análisis de la empresa ▪ Definición de objetivos del negocio
Comprensión de los datos	8	▪ Obtención de datos ▪ Análisis de los datos
Preparación de los datos	8	▪ Limpieza de datos ▪ Formateo de datos para modelo

Modelado	15	▪ Creación de algoritmo de deep learning ▪ Entrenamiento con datos procesados ▪ Modelado finalizado para evaluación
Evaluación	5	▪ Evaluación del modelo a través de métricas ▪ Modelo validado correctamente
Despliegue	8	▪ Integración del modelo en la aplicación web ▪ Pruebas funcionales y no funcionales ▪ Revisión final de la aplicación

Sprint Review 2:

Épica	Proceso	Enunciado de la historia	Estado
FASE 1: Comprensión del negocio	Objetivos del negocio	Como analista, se necesita definir los objetivos del negocio y establecer criterios de éxito, con el propósito de garantizar que la solución desarrollada responda a las necesidades planteadas.	Finalizada
	Situación actual	Como analista, se necesita evaluar el estado actual de la situación, con la finalidad de reconocer los conocimientos disponibles, la cantidad de datos necesaria y la factibilidad del proyecto.	Finalizada

	Plan de proyecto	Como analista, se necesita elaborar un plan de proyecto, orientado a detallar las etapas a seguir y las actividades específicas que se aplicarán en cada una de ellas.	Finalizada

1.1.4. Sprint #3

Sprint	Inicio	Término	Épica	Historia	
SPRINT 2 Cumplir con las actividades de la fase 2 y 3 de CRISP-DM	Lun 15/09/2025	Vie 29/09/2025	Comprensión de los datos	14	Recolección datos iniciales
				15	Descripción de los datos
				16	Exploración de los datos
				17	Verificación de calidad de los datos
			Preparación de los datos	18	Selección de datos
				19	Limpieza de datos
				20	Estructuración de datos
				21	Formateo de datos

Desarrollo del Sprint 3:

En este sprint se desarrollarán las fases 2 (Comprensión de los datos) y 3 (Preparación de los datos) del modelo CRISP-DM. Durante la Fase 2, se llevará a cabo un análisis exploratorio de las imágenes recolectadas,

evaluando su calidad, formato, distribución y características relevantes para el entrenamiento del modelo de deep learning. Asimismo, se identificarán posibles problemas de los datos, como desbalance de clases, presencia de ruido o inconsistencias. En la Fase 3, se procederá con la preparación del conjunto de datos, se definirán las estrategias de aumento de datos (data augmentation) para mejorar la robustez del modelo y se establecerá la partición de los conjuntos de entrenamiento, validación y prueba.

- **Recolección de datos:**

Se empleará el dataset **ACNE04**, el cual reúne imágenes dermatológicas de casos de acné vulgar clasificadas en distintos niveles de severidad. La elección de este conjunto de datos responde a dos factores principales: por un lado, su amplia utilización en diversos estudios científicos relacionados con la clasificación de gravedad del acné; y por otro, la disponibilidad de imágenes debidamente etiquetadas según el nivel de severidad de la enfermedad. Estas características lo convierten en una base sólida y confiable para el desarrollo de un modelo de clasificación.

- **Descripción de los datos:**
    - *Cantidad y clases de imágenes:*

      El dataset cuenta exactamente 1457 imágenes, distribuidas en cuatro categorías clínicas de severidad: leve, moderado, severo y muy severo. Esta clasificación fue realizada por especialistas en dermatología, lo que otorga validez clínica a las etiquetas asociadas.
    - *Etiquetado:*

      Cada imagen está acompañada de una etiqueta que especifica el grado de severidad del acné (definida a través del nombre de la imagen). Estas etiquetas son fundamentales para el aprendizaje supervisado, ya que constituyen la referencia que guía el entrenamiento y la evaluación del modelo.
    - *Variedad de condiciones en las imágenes:*

      El dataset presenta diversidad en cuanto a iluminación, ángulos de captura, tipos de piel y características individuales de los pacientes. Esta variabilidad enriquece el conjunto de datos, pues permite entrenar un modelo con mayor capacidad de generalización.
    - *Posibles limitaciones:*

A pesar de la riqueza del dataset, se anticipan ciertos desafíos como el posible desbalance en la cantidad de imágenes por clase, lo que podría requerir la aplicación de técnicas de balanceo de datos. Asimismo, se reconoce la posibilidad de encontrar imágenes con leves deficiencias de calidad (por ejemplo, desenfoques, sombras o baja resolución), lo que será considerado en la fase de preparación de datos.

- **Exploración de los datos:**
  - *Distribución de clases:*

Se realizó un conteo de las imágenes pertenecientes a cada categoría de severidad (leve, moderado, severo y muy severo). Este análisis permite determinar si las clases están balanceadas o si existe predominancia de alguna categoría. Para esto se hizo uso de la herramienta Google Colab:

```
# Extraer clases
clases = [img.split("_")[0] for img in imagenes]

# Contar las clases
conteo = Counter(clases)

# Mostrar resultados
print("Conteo por clases:")
for clase, cantidad in conteo.items():
    print(f"{clase}: {cantidad}")
```

```
Conteo por clases:
levle1: 637
levle0: 497
levle3: 137
levle2: 186
```

Figura 34 - Cantidad de imágenes por clase

Se evidencia un marcado desbalance en las clases correspondientes a los niveles más severos de acné, lo cual resulta esperable dado que los casos graves son menos frecuentes en la población. Cabe señalar que, en la práctica clínica peruana, se emplean principalmente tres escalas de clasificación, lo que abre la posibilidad de agrupar las categorías asociadas a la mayor severidad para obtener una distribución más equilibrada.

  - *Tamaño y distribución:*

Aunque las imágenes poseen una resolución estándar adecuada para redes neuronales convolucionales, se observó que algunas presentan dimensiones ligeramente distintas. Este hallazgo implica la necesidad de

aplicar un proceso de redimensionamiento para unificar criterios de entrada en el modelo.

- **Verificación de calidad de los datos:**
  - *Completitud de los datos:*

  Se verificó que todas las imágenes tuvieran un nombre de archivo que indicara claramente su clase (por ejemplo: levle0_xxx, levle1_xxx, etc.). Además, se descartaron posibles casos de duplicidad de nombres o errores de escritura en las etiquetas.
  - *Equilibrio y representatividad:*

  El conteo de imágenes mostró un desbalance en las clases, especialmente en los niveles de mayor severidad del acné, donde se dispone de un menor número de ejemplos. Este desbalance debe ser considerado en la fase de modelado, ya que puede influir en el desempeño del clasificador.
  - *Calidad visual de las imágenes:*

  Se realizó una inspección preliminar para identificar imágenes borrosas, con baja resolución o problemas de iluminación que dificulten su interpretación. Aunque la mayoría presenta una calidad aceptable, es recomendable filtrar o mejorar aquellas con deficiencias notorias.
  - *Integridad de los archivos:*

  No se encontraron archivos corruptos o ilegibles durante la lectura automatizada del dataset.
- **Selección de los datos:**

Luego del análisis inicial de las imágenes, se decidió utilizar la totalidad del dataset **ACNE-04**, dado que cumplía con los requerimientos para el entrenamiento del modelo de clasificación basado en técnicas de Deep Learning. Las imágenes fueron tomadas del directorio que contiene las muestras previamente etiquetadas, lo que permitió una correcta asociación entre cada imagen y su respectiva categoría de gravedad del acné.

Adicionalmente, se generaron archivos de anotaciones en formato .txt, tanto para los subconjuntos de train como para test. En total se elaboraron cinco anotaciones, correspondientes a un esquema de validación cruzada (cross-validation) en 5 folds. De esta manera, las imágenes se fueron alternando entre los subconjuntos de entrenamiento y prueba, garantizando que no se

repitieran las mismas muestras en cada iteración. Estos archivos fueron empleados como referencia durante el proceso de fine-tuning (ajuste fino del modelo a partir de parámetros previamente entrenados), facilitando la organización y el control en la etapa de entrenamiento.

- **Limpieza de datos:**

El proceso de limpieza de datos se centró en la preparación y estandarización de las imágenes para garantizar la calidad y consistencia del conjunto de datos. Se aplicó un proceso de normalización que ajusta los valores de intensidad de color de cada píxel en los tres canales RGB (rojo, verde y azul) utilizando valores específicos de (0.5, 0.5, 0.5). Este ajuste permite que el modelo de inteligencia artificial procese las imágenes de manera más eficiente, reduciendo el tiempo de aprendizaje y mejorando la estabilidad durante el entrenamiento.



Figura 35 - Normalización de imágenes

Para el conjunto de entrenamiento, se implementaron técnicas de aumento de datos que modifican las imágenes originales mediante volteos horizontales y verticales aleatorios, rotaciones de hasta 15 grados y deformaciones controladas. Estas modificaciones permiten generar múltiples versiones de cada imagen original, aumentando efectivamente la cantidad de datos disponibles para el entrenamiento. El propósito de estas transformaciones es enseñar al modelo a reconocer los diferentes grados de severidad del acné independientemente de la posición, orientación o ángulo desde el cual fue tomada la fotografía, mejorando así su capacidad para clasificar correctamente imágenes nuevas.



```python
# DATA TRANSFORMS
TRAIN_TRANSFORM = torchvision.transforms.Compose([
    torchvision.transforms.Resize(IMAGE_SIZE),
    torchvision.transforms.RandomHorizontalFlip(p=0.5),
    torchvision.transforms.RandomVerticalFlip(p=0.5),
    torchvision.transforms.RandomRotation(degrees=15),
    torchvision.transforms.ElasticTransform(),
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize(MEAN, STD)
])

TEST_TRANSFORM = torchvision.transforms.Compose([
    torchvision.transforms.Resize(IMAGE_SIZE),
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize(MEAN, STD)
])
```

Figura 36 - Técnicas de data augmentation en imágenes

La organización de los datos se realizó procesando archivos de texto que contienen la información de cada imagen: su ubicación, la clasificación de severidad y el conteo de lesiones presentes. Se dividieron los datos manteniendo la misma proporción de casos para cada nivel de severidad entre los conjuntos de entrenamiento y validación (80%-20%), asegurando que el modelo tenga acceso equilibrado a ejemplos de todos los grados de acné durante su proceso de aprendizaje.

- **Estructuración de datos:**

El conjunto de imágenes dermatológicas fue estructurado para su procesamiento mediante técnicas de Deep Learning. Cada imagen fue redimensionada a un tamaño estándar de 224x224 píxeles para asegurar la consistencia en la entrada del modelo de clasificación de gravedad de acné. Este redimensionamiento fue necesario para adaptarse a las restricciones de la arquitectura seleccionada y optimizar el rendimiento durante el entrenamiento. La estandarización del tamaño de imagen garantiza que todas las muestras mantengan las mismas dimensiones, facilitando el procesamiento en lotes y la extracción uniforme de características por parte de la red neuronal convolucional.

De igual forma el dataset fue organizado siguiendo una estructura jerárquica que separa claramente las imágenes de entrenamiento y prueba, con archivos de metadatos asociados que contienen la información de clasificación y características adicionales de cada muestra. Esta organización sistemática

permite un acceso eficiente a los datos durante el entrenamiento del modelo, Además, se implementó un sistema de indexación que asocia cada imagen con su correspondiente etiqueta de severidad y datos complementarios sobre el conteo de lesiones, creando una estructura de datos coherente y fácilmente accesible para el algoritmo de aprendizaje automático.

Por último, se optó por unir las clases 2 y 3 correspondientes al acné severo, esto debido al desbalance existente en los datos, lo cual representaba la opción más adecuada para el entrenamiento del modelo. Esta decisión se justifica, además, porque en el Perú la clasificación clínica del acné se establece en tres niveles de severidad.



Figura 37 - Archivos de test y entrenamiento estructurados

- **Formateo de datos:**

Como se mencionó en el paso anterior los datos fueron organizados y estructurados en un formato específico que facilita su procesamiento por el modelo de clasificación de gravedad de acné. La organización se basó en la información contenida en archivos de anotaciones especializados que almacenan metadatos esenciales para cada muestra del dataset.

Los datos fueron estructurados en tres componentes principales:

- **Imágenes dermatológicas:** Almacenadas como tensores de PyTorch con dimensiones estandarizadas de 224x224x3 píxeles, representando los tres canales de color RGB.
- **Etiquetas de severidad:** Un vector de clasificación donde "0" representa acné leve, "1" acné moderado y "2" acné severo, correspondiendo a los cuatro niveles de gravedad establecidos en la escala clínica utilizada.
- **Conteo de lesiones:** Información numérica adicional que cuantifica el número de lesiones presentes en cada imagen, proporcionando datos complementarios para el análisis de severidad.

La información fue extraída de archivos de texto estructurados que contienen las anotaciones médicas, donde cada línea especifica el nombre del archivo de imagen, su etiqueta de severidad correspondiente y el número total de lesiones identificadas por especialistas dermatológicos. Esta organización permite mantener la trazabilidad entre cada imagen y sus características clínicas asociadas.



```python
def load_data(train_file, test_file):
    """Load training and test data from files"""
    train_df = pd.read_csv(train_file, names=['path','label','leisons'], sep='  ', engine='python')
    test_df = pd.read_csv(test_file, names=['path','label','leisons'], sep='  ', engine='python')
    return train_df, test_df
```

Figura 38 - Extracción de características de archivos de test y entrenamiento

Los conjuntos de datos estructurados fueron organizados utilizando DataLoaders de PyTorch, permitiendo la carga eficiente en lotes durante el entrenamiento y evaluación del modelo. Esta estructura facilita la reutilización de los datos preprocesados sin necesidad de realizar el formateo cada vez que se ejecuta el algoritmo, optimizando los tiempos de procesamiento y garantizando la consistencia en los experimentos realizados.



```python
def make_loader(dataset, train_batch_size, validation_split=0.2):
    """Crear train_loader y test_loader a partir del dataset

    Args:
        dataset ([object]):
        train_batch_size ([int]):
        validation_split (float, opcional): Default to 0.2.

    Returns:
        [type]: [description]
    """
    # Numero de muestras en train y test set
    train_len = int(len(dataset) * (1 - validation_split))
    test_len = len(dataset) - train_len
    train_set, test_set = torch.utils.data.random_split(dataset, [train_len, test_len])
    # Crear train_loader
    train_loader = torch.utils.data.DataLoader(
        train_set, batch_size=train_batch_size, shuffle=True,
    )
    # Crear test_loader
    test_loader = torch.utils.data.DataLoader(test_set, batch_size=1, shuffle=False,)
    return train_loader, test_loader
```

Figura 39 - Creación de dataloader para entrenamiento y test

Sprint Review 4

Épica	Proceso	Enunciado de la historia	Estado
FASE 2: Comprensión de los datos	Recolección datos iniciales	Como analista, se necesita recolectar las imágenes iniciales del dataset ACNE-04 para el entrenamiento del modelo y su adecuación al proyecto, con la finalidad de iniciar con el procesamiento y disponer de los datos para el posterior entrenamiento.	Finalizada
	Descripción de los datos	Como analista, se necesita realizar una descripción de las imágenes recolectadas, con la finalidad de establecer el volumen de datos, la categorización de las clases, el formato de los archivos y las dimensiones de las imágenes.	Finalizada
	Exploración de los datos	Como analista, se necesita aplicar funciones básicas que revelen propiedades en los datos, como el número de imágenes por categoría, las dimensiones de cada muestra y el balance de clases, con la finalidad de construir un informe sobre la exploración inicial del dataset.	Finalizada

	Verificación de calidad de los datos	Como analista, se necesita verificar la consistencia y calidad de las imágenes, identificando duplicados, baja resolución, ruido visual o distorsiones, con la finalidad de asegurar que los datos sean completos, coherentes y adecuados para el entrenamiento del modelo.	Finalizada
FASE 3: Preparación de los datos	Selección de datos	Como desarrollador, se necesita seleccionar las imágenes útiles del dataset ACNE-04, apoyándose en criterios de calidad previamente establecidos, con la finalidad de garantizar que los datos seleccionados sean los más adecuados para la clasificación del acné.	Finalizada
	Limpieza de datos	Como desarrollador, se necesita aplicar técnicas de limpieza, como la estandarización del tamaño de las imágenes, con la finalidad de optimizar la calidad del dataset para la fase de modelado.	Finalizada
	Estructuración de datos	Como desarrollador, se necesita organizar las imágenes en subconjuntos de train y test mediante esquemas de validación cruzada en 5 anotaciones, con la finalidad de estructurar los datos de forma que se garantice un entrenamiento robusto y una evaluación confiable del modelo.	Finalizada
	Formateo de datos	Como desarrollador, se necesita preparar las imágenes en un formato uniforme, redimensionándolas a las dimensiones requeridas por la red neuronal, normalizando los valores de los píxeles y convirtiéndolas en tensores, con la finalidad de asegurar su compatibilidad con el modelo de Deep Learning.	Finalizada

1.1.5. Sprint #4

Sprint	Inicio	Término	Épica	Historia

SPRINT 4 Cumplir con las actividades de la fase 4 de CRISP-DM	Lun 29/09/2025	Vie 20/10/2025	Modelado	22	Selección y técnica de modelado
				23	Construcción del modelo

Desarrollo del Sprint 4:

En este sprint se desarrollará la Fase 4 del modelo CRISP-DM, correspondiente al modelado. Durante esta etapa se seleccionará la técnica de modelado más adecuada para el problema de clasificación de la gravedad del acné vulgar, considerando la naturaleza de los datos y los objetivos del proyecto. Se procederá a la construcción de los modelos de deep learning, definiendo sus arquitecturas y configurando los parámetros necesarios para su entrenamiento. Asimismo, se realizarán pruebas preliminares para evaluar el rendimiento de los modelos, comparando los resultados entre diferentes enfoques a realizar.

- **Selección y técnica de modelado**

Para abordar el problema de clasificación de la gravedad del acné vulgar se evaluaron diferentes arquitecturas de redes neuronales convolucionales (CNN), tomando en cuenta tanto la literatura relacionada como los avances recientes en el campo del deep learning. Se evaluaron tres arquitecturas principales: ***EfficientNet_V2_S, ResNet50 y VGG16***, cada una seleccionada por criterios específicos y fundamentados en la literatura científica.

ResNet50 y VGG16 fueron consideradas debido a su amplia documentación y validación en estudios previos relacionados con clasificación de imágenes dermatológicas y diagnóstico de acné. Estas arquitecturas han demostrado efectividad en tareas similares según la revisión de literatura especializada, estableciendo un punto de referencia confiable para la comparación de rendimiento.

EfficientNet_V2_S fue incorporada como la opción más avanzada, representando el estado del arte en redes neuronales convolucionales. Esta

arquitectura se distingue por su eficiencia computacional superior y su capacidad para lograr mayor precisión con menor cantidad de parámetros, optimizando tanto el tiempo de entrenamiento como los recursos de memoria requeridos.

Para llevar a cabo las pruebas de modelado, se diseñó una clase, desde la cual se implementaron y configuraron las diferentes arquitecturas de redes neuronales seleccionadas. Esta clase permitió realizar múltiples experimentos de forma estructurada, comparando el desempeño de cada modelo bajo las mismas condiciones de entrenamiento. Tras las evaluaciones realizadas, se identificó que la arquitectura ***EfficientNet_V2_M*** presentó los mejores resultados en términos de rendimiento y capacidad de generalización, superando a las alternativas consideradas (VGG-16 y ResNet-50). Para un mayor detalle del proceso de experimentación y análisis de resultados, véase el desarrollo correspondiente al Objetivo N°01.

```python
class MyNet(nn.Module):
    def __init__(self):
        super(MyNet, self).__init__()

        self.cnn = torchvision.models.efficientnet_v2_m(weights=EfficientNet_V2_M_Weights.DEFAULT).cuda()
        # self.cnn = torchvision.models.resnet50(weights=ResNet50_Weights.DEFAULT).cuda()
        # self.cnn = torchvision.models.vgg16(weights=VGG16_Weights.DEFAULT).cuda()
        for param in self.cnn.parameters():
            param.requires_grad = True
        self.cnn.classifier = nn.Sequential(
            nn.Linear(self.cnn.classifier[1].in_features, 512),
            nn.Dropout(p=0.2),
            nn.ReLU(),
            nn.Linear(512, 128),
            nn.Dropout(p=0.2),
            nn.Linear(128, 64),
            nn.Linear(64, 3), # Cantidad de clases a clasificar
        )

        # self.cnn.fc = nn.Sequential(
        #     nn.Linear(self.cnn.fc.in_features, 512),
        #     nn.Dropout(p=0.2),
        #     nn.ReLU(),
        #     nn.Linear(512, 128),
        #     nn.Dropout(p=0.2),
        #     nn.Linear(128, 64),
        #     nn.Linear(64, 4)
        # )

        # self.cnn.classifier = nn.Sequential(
        #     nn.Linear(25088, 4096),
        #     nn.ReLU(True),
        #     nn.Dropout(),
        #     nn.Linear(4096, 512),
        #     nn.ReLU(True),
        #     nn.Dropout(p=0.2),
        #     nn.Linear(512, 128),
        #     nn.Dropout(p=0.2),
        #     nn.Linear(128, 64),
        #     nn.Linear(64, 4)
        # )

    def forward(self, img):
        output = self.cnn(img)
        return output
```

Figura 40 - Clase Python con diferentes arquitecturas para pruebas

- **Construcción del modelo**

La construcción del modelo se desarrolló utilizando el framework ***PyTorch*** como base tecnológica principal, aprovechando su flexibilidad y capacidades avanzadas para el desarrollo de redes neuronales profundas. El ecosistema de librerías empleado incluye componentes especializados para cada aspecto del proceso de modelado.

PyTorch (torch) proporciona la infraestructura base para operaciones tensoriales y computación en GPU, mientras que *torch.nn* ofrece los módulos de construcción de redes neuronales como capas lineales, funciones de activación y contenedores secuenciales. *torch.nn.functional* aporta funciones especializadas como operaciones de softmax y transformaciones aplicadas durante el procesamiento. *TorchVision* facilita el acceso a modelos preentrenados y sus pesos correspondientes, incluyendo las implementaciones oficiales de EfficientNet_V2_S, ResNet50 y VGG16. Se integró *ReduceLROnPlateau* como scheduler de tasa de aprendizaje adaptativo, que reduce automáticamente la velocidad de aprendizaje cuando las métricas de validación se estancan, mejorando la convergencia del modelo. La librería *tqdm* proporciona barras de progreso visuales durante el entrenamiento, facilitando el monitoreo del proceso de aprendizaje.

El diseño se estructuró en la clase propia del modelo dentro del módulo model.py, proporcionando la capacidad de alternar entre arquitecturas según los requerimientos experimentales. Se utilizaron pesos preentrenados para el modelo, manteniendo inicialmente todos los parámetros como entrenables para permitir el ajuste fino completo de la red. Esta configuración facilita la adaptación de las características aprendidas en ImageNet hacia las particularidades visuales del acné dermatológico.

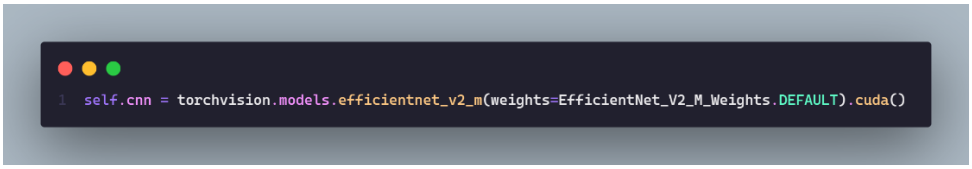


Figura 41 - Pesos preentrenados del modelo seleccionado

- *Personalización de modelo:*

Para la técnica seleccionada se reemplazó la capa clasificadora original con una arquitectura personalizada:



```
self.cnn.classifier = nn.Sequential(
            nn.Linear(self.cnn.classifier[1].in_features, 512),
            nn.Dropout(p=0.2),
            nn.ReLU(),
            nn.Linear(512, 128),
            nn.Dropout(p=0.2),
            nn.Linear(128, 64),
            nn.Linear(64, 3), # Cantidad de clases a clasificar
        )
```

Figura 42 - Arquitectura personalizada del modelo

Esta configuración implementa una reducción progresiva de dimensionalidad desde las características extraídas por la red base hasta las 4 clases de severidad objetivo. Se incorporaron capas de Dropout con probabilidad 0.2 para prevenir sobreajuste, y activaciones ReLU para introducir no linealidad en la clasificación final.

- *Función de pérdida especializada:*

Se implementó ***Label Smoothing Loss*** como función de pérdida, una técnica avanzada que suaviza las etiquetas categóricas para mejorar la generalización. Esta función reduce la confianza excesiva del modelo en sus predicciones, mejorando la robustez ante variaciones en las imágenes dermatológicas y reduciendo el riesgo de sobreajuste a características específicas del conjunto de entrenamiento.



```
class LabelSmoothingLoss(nn.Module):
    def __init__(self, smoothing=0.1, dim=-1):
        super(LabelSmoothingLoss, self).__init__()
        self.smoothing = smoothing
        self.dim = dim

    def forward(self, pred, target):
        target = F.one_hot(target, num_classes=pred.size(-1))
        target = target.float()
        target = (1 - self.smoothing) * target + self.smoothing / pred.size(-1)
        log_pred = F.log_softmax(pred, dim=self.dim)
        loss = nn.KLDivLoss(reduction='batchmean')(log_pred, target)
        return loss
```

Figura 43 - Función de pérdida utilizada

- *Optimizador Adam:*

Se seleccionó el optimizador Adam (Adaptive Moment Estimation) como algoritmo de optimización principal debido a su eficacia

comprobada en tareas de clasificación. Esta configuración es particularmente efectiva para el ajuste fino de redes preentrenadas, proporcionando convergencia estable y adaptación gradual de los pesos hacia el dominio de las clases establecidas.

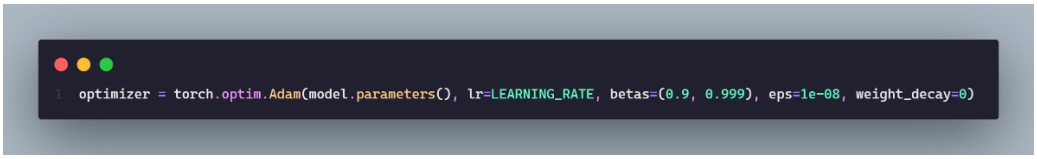


Figura 44 - Optimizador Adam

Sprint Review 4:

Épica	Proceso	Enunciado de la historia	Estado
FASE 4: Modelado	Selección y técnica de modelado	Como desarrollador, se necesita seleccionar la técnica de modelado más adecuada para el problema de clasificación de acné, optando por una CNN, con la finalidad de aprovechar su capacidad para procesar imágenes y extraer características relevantes que permitan clasificar la gravedad de manera precisa.	Finalizada
	Construcción del modelo	Como desarrollador, se necesita construir el modelo de Deep Learning definiendo su arquitectura, parámetros de entrenamiento, con la finalidad de obtener un sistema capaz de aprender patrones a partir de las imágenes del dataset y generar predicciones sobre la gravedad del acné.	Finalizada

1.1.6. Sprint #5

Sprint	Inicio	Término	Épica	Historia	
SPRINT 5 Cumplir con las actividades de la fase 5 de CRISP-DM	Lun 20/09/2025	Vie 27/10/2025	Evaluación	24	Evaluación de resultados

Desarrollo del Sprint 5:

En este sprint se desarrollará la Fase 5 del modelo CRISP-DM, correspondiente a la evaluación. Durante esta etapa se analizará el

rendimiento del modelo construido en la fase de modelado, utilizando métricas de clasificación como precisión, f1-score y matriz de confusión. El objetivo será verificar que dicho modelo cumpla con los requerimientos planteados en el proyecto y garantice un desempeño adecuado para la tarea de clasificación de la gravedad del acné vulgar

- **Evaluación de resultados:**

La evaluación del modelo de clasificación de gravedad de acné se realizó utilizando múltiples métricas y visualizaciones para obtener una comprensión integral del rendimiento alcanzado. Los resultados obtenidos demuestran la efectividad del enfoque utilizado con EfficientNet_V2_S para esta aplicación.

- *Métricas de rendimiento:*

  El modelo alcanzó una precisión global del **92.70%** sobre el conjunto de prueba, representando un rendimiento sólido para la clasificación automática de severidad de acné. Esta métrica indica que el sistema clasifica correctamente aproximadamente 9 de cada 10 imágenes dermatológicas, lo cual es prometedor.

```
Precisión de Validación: 0.9270 (92.70%)

Reporte de Clasificación - Conjunto de Validación:
==========================================================
              precision    recall  f1-score   support

       Leve       0.87      0.96      0.91        82
   Moderado       0.95      0.88      0.91       101
     Severo       1.00      0.96      0.98        50

   accuracy                           0.93       233
  macro avg       0.94      0.93      0.94       233
weighted avg      0.93      0.93      0.93       233
```

Figura 45 - Resultados obtenidos con el modelo

- Análisis de métricas:

Clase	Precisión	Recall	F1-Score	Support
Leve	87%	96%	0.91	82
	Esta clase muestra un excelente equilibrio con el recall más alto (96%), demostrando que el modelo identifica efectivamente la gran mayoría de casos leves. Aunque la precisión es ligeramente menor que otras clases.			
Moderado	95%	88%	0.91	101

	Esta clase presenta un desempeño muy sólido con la segunda precisión más alta, lo que significa que cuando el modelo predice acné moderado, es altamente confiable. El recall de 88% indica que detecta la mayoría de estos casos, aunque existe un margen de mejora.			
Severo	100%	96%	98%	50
	Esta clase exhibe el desempeño más excepcional con precisión perfecta y recall muy elevado. Cuando el modelo predice acné severo, la predicción es definitivamente correcta.			

- *Matriz de confusión:*

La matriz de confusión proporciona resultados detallados sobre los patrones de error:

- ***Acné Leve:*** 79 clasificaciones correctas de 82 casos totales, con 3 confusiones hacia la clase moderada. La mínima tasa de error demuestra la claridad diferenciadora de características leves.
- ***Acné Moderado:*** 89 clasificaciones correctas de 101 casos, con 12 confusiones hacia leve y solo 0 hacia severo. Esta distribución sugiere que los casos moderados frecuentemente comparten características con la severidad leve, pero el modelo raramente confunde la clase moderada con la más severa.
- ***Acné Severo:*** 48 clasificaciones correctas de 50 casos, con 2 confusiones hacia moderado. La robustez en esta categoría es particularmente valiosa, demostrando que los casos severos son diferenciados claramente del resto.

El patrón de errores muestra transiciones lógicas principalmente entre clases adyacentes en severidad, sin saltos abruptos entre categorías extremas (leve directamente a severo). Esta coherencia clínica valida que

el modelo aprende gradaciones realistas de severidad, reflejando la progresión natural de la enfermedad.

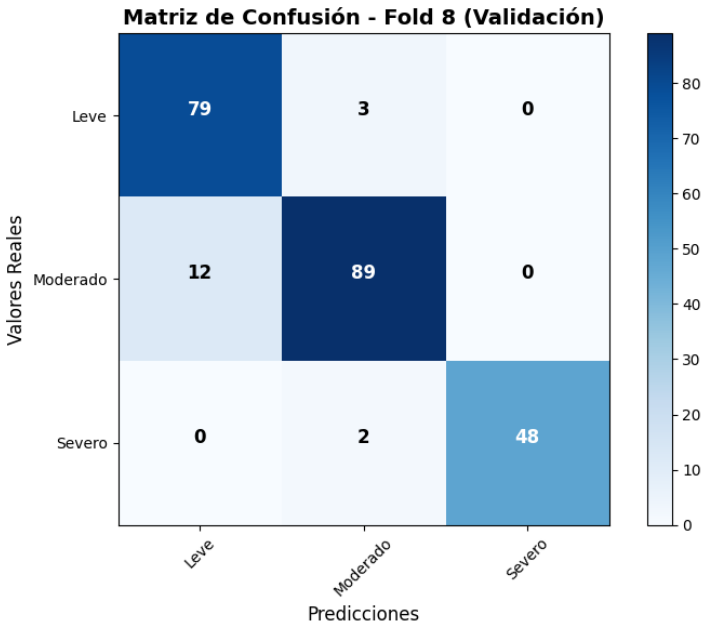


Figura 46 - Matriz de confusión

- *Curvas de entrenamiento:*

Las curvas de aprendizaje revelan el comportamiento del modelo durante el entrenamiento:

Curva de Pérdida:

El modelo exhibe un descenso pronunciado en las primeras 4 épocas (de 0.19 a aproximadamente 0.12), seguido de convergencia estable. Las curvas de entrenamiento y validación mantienen una trayectoria similar sin divergencia significativa, indicando ausencia de sobreajuste severo. A partir de la época 6, ambas curvas se estabilizan con mínima variación, sugiriendo que el modelo ha aprendido representaciones robustas.

Curva de Accuracy:

El modelo muestra mejora progresiva desde ~90% en la época inicial hasta ~95% en el entrenamiento y ~90% en validación en la época final. La convergencia gradual y estable después de la época 5 refleja que el modelo ha encontrado patrones significativos en los datos. La cercanía entre las curvas de entrenamiento y validación confirma una excelente generalización, sin evidencia de memorización o desadaptación.

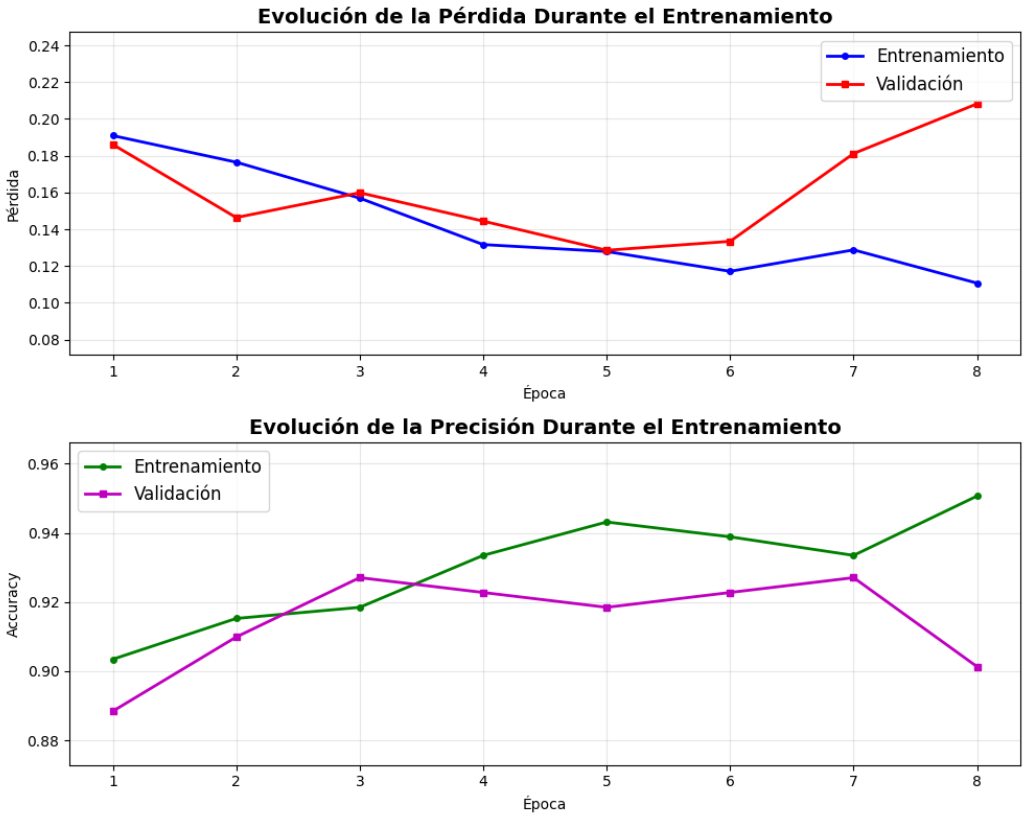


Figura 47 - Curvas de entrenamiento

Sprint Review 5:

Épica	Proceso	Enunciado de la historia	Estado
FASE 5: Evaluación	Evaluación de resultados	Como desarrollador, se necesita evaluar el desempeño del modelo utilizando métricas de precisión, exactitud, sensibilidad y F1-score sobre los subconjuntos de test, con la finalidad de determinar la capacidad del modelo para clasificar correctamente la gravedad del acné.	Finalizada

1.1.7. Sprint #6

Sprint	Inicio	Término	Épica	Historia	
SPRINT 6 Cumplir con las actividades de la fase 6 de CRISP-DM	Lun 27/10/2025	Vie 03/11/2025	Despliegue	25	Plan de despliegue

				26	Monitoreo y mantenimiento
				27	Revisión del proyecto

Desarrollo del Sprint 6:

En este sprint se desarrollará la Fase 6 del modelo CRISP-DM, correspondiente al despliegue. Durante esta etapa se integrará el modelo seleccionado en la aplicación web desarrollada, garantizando su correcto funcionamiento en un entorno real de uso. Para ello, se implementarán los servicios necesarios que permitan la clasificación automática de imágenes de acné vulgar, así como la interacción fluida con la interfaz de usuario.

- **Plan de despliegue:**

En esta actividad se implementará el modelo seleccionado en el entorno de la aplicación web, asegurando su correcta integración con los módulos existentes. Para ello, se desarrollarán APIs que permitan la comunicación entre la interfaz de usuario y el modelo de deep learning, posibilitando la recepción de imágenes por parte del usuario, su procesamiento por el modelo y la devolución de la clasificación de la gravedad del acné vulgar.

Se definirán los endpoints necesarios para el envío de imágenes y la obtención de resultados, garantizando que las solicitudes sean procesadas de manera eficiente y segura.

- *Implementación de APIs con FastAPI:*

  Para la integración del modelo de clasificación de acné vulgar en la aplicación web, se desarrollaron APIs utilizando FastAPI, lo que permitió estructurar y documentar de manera eficiente los servicios de predicción. Se definió una arquitectura basada en endpoints REST,

siendo el principal */predict*, encargado de recibir la imagen facial, procesarlas y devolver la clasificación junto con información detallada sobre la predicción.

La API garantiza la validación de la calidad de la imagen y la presencia de un rostro, con opción de omitir estas verificaciones para pruebas internas mediante el parámetro *skip_quality_check*. Además, se implementó un esquema de respuesta definido por la clase *PredictionResponse*, que proporciona información completa de la predicción:

- Clase predicha y etiqueta correspondiente.
- Confianza del modelo en la predicción, tanto en valor numérico como en porcentaje.
- Probabilidades de cada categoría de severidad del acné.
- Tiempo de procesamiento de la imagen.



```python
class PredictionResponse(BaseModel):
    """Respuesta de predicción individual"""
    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "success": True,
                "prediccion_class": 1,
                "prediccion_label": "Moderate Acne",
                "confianza": 0.8567,
                "confianza_porcentaje": "85.67%",
                "probabilidades": {
                    "Leve": 0.1234,
                    "Moderado": 0.8567,
                    "Severo": 0.0156,
                    "Muy Severo": 0.0043
                },
                "tiempo_procesamiento": 234.5
            }
        }
    )

    success: bool
    prediccion_class: int
    prediccion_label: str
    confianza: float
    confianza_porcentaje: str
    probabilidades: dict
    tiempo_procesamiento: float
```

Figura 48 - Esquema definido para la respuesta del API

El uso de FastAPI permite que las APIs estén documentadas automáticamente, facilitando su uso, pruebas y mantenimiento, así como la integración con la aplicación web desarrollada, asegurando un flujo

seguro y eficiente entre la interfaz de usuario y el modelo de deep learning.

- *Descripción de la API /predict*

La API */predict* es el endpoint principal del servicio de predicción del modelo de clasificación de la gravedad del acné vulgar. Esta API permite que la aplicación web envíe la imagen fácil del paciente para obtener la predicción del nivel de severidad del acné.

Características principales:

Método: POST

Entrada:

- **file (obligatorio):** archivo de imagen que contiene al menos un rostro visible.
- **skip_quality_check** (opcional, booleano): permite omitir las verificaciones de calidad de imagen para pruebas internas.

Procesamiento:

- Validación del archivo recibido (tipo, tamaño y contenido).
- Verificación de la presencia de un rostro en la imagen.
- Verificación de calidad de la imagen adjuntada.
- Transformación de la imagen en un tensor compatible con el modelo.
- Ejecución de la predicción mediante el modelo EfficientNet_V2_S.
- Cálculo de las probabilidades de cada clase y confianza de la predicción.
- Medición del tiempo total de procesamiento.



```python
@app.post("/predict", response_model=PredictionResponse, tags=["Prediction"])
async def predict_single(
    file: UploadFile = File(..., description="Imagen para clasificar"),
    skip_quality_check: bool = False
):
    """
    Clasificar una sola imagen

    Sube una imagen y obtén la clasificación de severidad del acné.
    La imagen debe contener al menos un rostro visible.

    Args:
        file: Archivo de imagen
        skip_quality_check: Si es True, omite las validaciones de calidad de imagen (solo para testing)
    """
    start_time = datetime.utcnow()

    try:
        # Validaciones
        if model is None or mtcnn is None:
            raise HTTPException(
                status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
                detail="Modelo o detector de rostros no disponible"
            )

        if not validate_image(file):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Tipo de archivo no válido: {file.content_type}"
            )

        # Procesar imagen
        image_bytes = await file.read()
        if len(image_bytes) == 0:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Archivo vacío"
            )

        # Validar que la imagen contenga un rostro
        # Si skip_quality_check es True, temporalmente deshabilitar validaciones de calidad
        original_quality_setting = IMAGE_QUALITY_CONFIG.get("enable_quality_checks", True)
        if skip_quality_check:
            IMAGE_QUALITY_CONFIG["enable_quality_checks"] = False

        try:
            face_valid, face_error_msg = validate_face_in_image(image_bytes)
            if not face_valid:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail=face_error_msg
                )
        finally:
            # Restaurar configuración original
            IMAGE_QUALITY_CONFIG["enable_quality_checks"] = original_quality_setting

        image_tensor = process_image(image_bytes)

        # Predecir
        predicted_class, confidence, probabilities = predict(image_tensor)

        # Tiempo de procesamiento
        processing_time = (datetime.utcnow() - start_time).total_seconds() * 1000

        return PredictionResponse(
            success=True,
            prediccion_class=predicted_class,
            prediccion_label=CLASS_NAMES[predicted_class],
            confianza=round(confidence, 4),
            confianza_porcentaje=f"{confidence*100:.2f}%",
            probabilidades=probabilities,
            tiempo_procesamiento=round(processing_time, 2)
        )

    except HTTPException:
        raise
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=str(e)
        )
    except Exception as e:
        logger.error(f"Error en predicción: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Error interno: {str(e)}"
        )
```

Figura 49 - API /predict

- **Monitoreo y mantenimiento**

La aplicación web será sometida a un seguimiento continuo, implementando actualizaciones y ajustes de manera periódica, fundamentados en las observaciones y comentarios proporcionados por los usuarios, con el objetivo de mantener su correcto funcionamiento y optimizar su desempeño.

- **Revisión del proyecto:**

Al finalizar el desarrollo y las pruebas, se llevará a cabo un análisis detallado de la aplicación web para identificar áreas de mejora. Se evaluarán tanto los aspectos técnicos como la precisión y el desempeño del modelo de clasificación de gravedad de acné integrado en el sistema. Además, se definirán los pasos futuros, que podrían incluir la optimización del modelo y la mejora de la interfaz de usuario de la aplicación web, con el fin de ofrecer una experiencia más intuitiva y eficiente.

Sprint Review 6:

Épica	Proceso	Enunciado de la historia	Estado
FASE 6: Despliegue	Plan de despliegue	Como desarrollador, se necesita definir un plan para integrar el modelo de clasificación de acné en la aplicación web, con la finalidad de asegurar que el sistema pueda ser utilizado por el consultorio de manera efectiva y confiable.	Finalizada
	Monitoreo y mantenimiento	Como desarrollador, se necesita implementar mecanismos de monitoreo del modelo en producción, registrando su desempeño y posibles errores, con la finalidad de garantizar su correcto funcionamiento y permitir ajustes cuando sea necesario.	Finalizada
	Revisión del proyecto	Como analista, se necesita revisar todo el proyecto, evaluando las etapas de desarrollo en general, con la finalidad de identificar mejoras, documentar lecciones aprendidas y asegurar que los objetivos del sistema hayan sido alcanzados.	Finalizada

1.1.8. Sprint #7

Sprint	Inicio	Término	Épica	Historia

SPRINT 7	Lun 03/11/2025	Vie 17/11/2025	Evaluación de gravedad acné e historial evaluaciones.	28	Evaluación de gravedad de acné mediante una imagen facial
				29	Registro de evaluación de gravedad de paciente
				30	Generación de reporte de evaluaciones

Desarrollo del Sprint 7:

En este sprint se implementarán las funcionalidades finales del sistema para la gestión de evaluaciones de acné, consolidando los desarrollos realizados en los sprints anteriores. Se desarrollará el módulo de evaluación de gravedad del acné a partir de imágenes faciales, utilizando el modelo de clasificación previamente entrenado. Además, se permitirá registrar y almacenar los resultados de cada paciente de forma organizada, asociando la información a su historial de evaluaciones. También se habilitará la generación de reportes detallados de las evaluaciones, con opciones de descarga, facilitando la revisión médica y el seguimiento del tratamiento. Con estas funcionalidades, se integrará un flujo completo y operativo dentro de la aplicación, asegurando que los distintos módulos interactúen de manera coherente y eficiente.

- **Evaluación de gravedad de acné mediante una imagen facial:**
    - *Funciones implementadas:*

Esta función maneja la evaluación de imágenes médicas utilizando una API de predicción de inteligencia artificial. Cuando el usuario sube una imagen a evaluar, la función primero valida que el paciente exista en el sistema y que se hayan enviado en formato Base64. Luego, busca los datos del paciente y obtiene la URL del backend desde la configuración. La función envía la imagen a evaluar a la función 'predecirImagen' que se encarga de comunicarse con el backend y espera la respuesta. Si la predicción es exitosa, prepara los datos del resultado incluyendo la información del paciente (id, nombre completo, DNI, edad, género), la imagen evaluada, el resultado de la predicción y la fecha actual de evaluación. Finalmente, renderiza la vista 'evaluation/prediction' mostrando todos estos datos. Si ocurre un error durante el proceso, ya sea de

comunicación con la API o interno del servidor, captura el error, lo registra en el log y redirige al usuario de vuelta mostrando un mensaje descriptivo del error ocurrido.



```php
/**
     * Evaluar imagen(es) usando la API de predicción
     */
    public function predecir(Request $request)
    {
        $request->validate([
            'paciente_id' => 'required|exists:paciente,id',
            'imagenes' => 'required|array|min:1|max:3',
            'imagenes.*' => 'required|string' // Base64 strings
        ]);

        try {
            $paciente = Paciente::findOrFail($request->paciente_id);
            $imagenes = $request->imagenes;
            $backendUrl = config('app.backend_url');

            // Determinar si usar predicción simple o batch
            if (count($imagenes) === 1) {
                // Predicción simple
                $resultado = $this->predecirImagen($imagenes[0], $backendUrl);
            }

            if (!$resultado['success']) {
                // Extraer el mensaje de detail del JSON de error si existe
                $errorMessage = $resultado['message'];
                try {
                    // Si el mensaje contiene JSON, intentar extraer el detail
                    if (strpos($errorMessage, 'Error en la API de predicción:') === 0) {
                        $jsonPart = str_replace('Error en la API de predicción: ', '', $errorMessage);
                        $errorData = json_decode($jsonPart, true);
                        if (isset($errorData['detail'])) {
                            $errorMessage = $errorData['detail'];
                        }
                    }
                } catch (\Exception $e) {
                    // Si no se puede parsear, usar el mensaje original
                }

                return redirect()->back()->withErrors([
                    'prediccion' => $errorMessage
                ]);
            }

            // Preparar datos para la vista de resultados
            $resultadoConPaciente = [
                'success' => true,
                'paciente' => [
                    'id' => $paciente->id,
                    'nombre' => $paciente->nombres . ' ' . $paciente->apellidos,
                    'dni' => $paciente->dni,
                    'edad' => $paciente->edad,
                    'genero' => $paciente->genero
                ],
                'imagenes' => $imagenes,
                'prediccion' => $resultado['data'],
                'fecha_evaluacion' => now()->toDateTimeString()
            ];

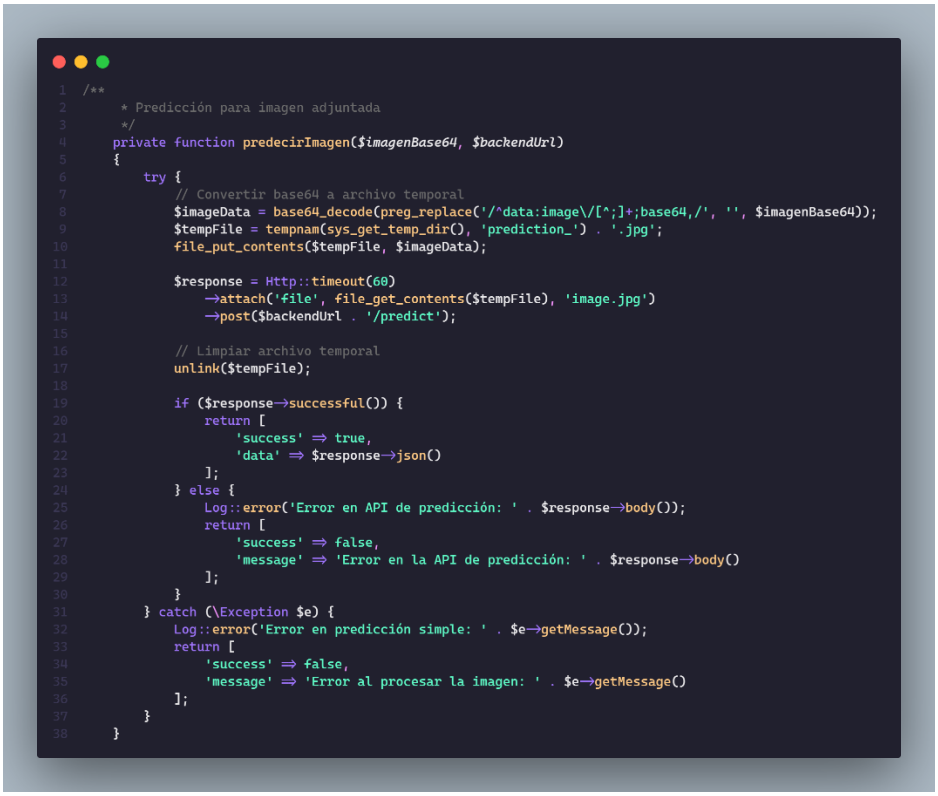
            return Inertia::render('evaluation/prediction', $resultadoConPaciente);

        } catch (\Exception $e) {
            Log::error('Error en predicción: ' . $e->getMessage());
            return redirect()->back()->withErrors([
                'prediccion' => 'Error interno del servidor: ' . $e->getMessage()
            ]);
        }
    }
```

Figura 50 - Función de predicción de gravedad de acné

Esta función privada maneja la predicción de la imagen que envía el usuario. Primero, convierte la imagen desde formato Base64 a datos binarios, eliminando el encabezado del formato. Luego, crea un archivo temporal en el sistema con extensión .jpg y guarda los datos de la imagen en él. A continuación, envía una petición HTTP POST a la ruta '/predict' del backend configurado, adjuntando el archivo temporal como 'image.jpg' con un timeout de 60 segundos para esperar la respuesta. Una vez completada la petición,

elimina el archivo temporal del sistema para liberar espacio. Si la API responde exitosamente, retorna un array con 'success' en true y los datos de la predicción en formato JSON. Si la API devuelve un error, registra el error en el log y retorna un array con 'success' en false y el mensaje de error recibido.



```php
/**
     * Predicción para imagen adjuntada
     */
    private function predecirImagen($imagenBase64, $backendUrl)
    {
        try {
            // Convertir base64 a archivo temporal
            $imageData = base64_decode(preg_replace('/^data:image\/[^;]+; base64,/', '', $imagenBase64));
            $tempFile = tempnam(sys_get_temp_dir(), 'prediction_') . '.jpg';
            file_put_contents($tempFile, $imageData);

            $response = Http::timeout(60)
                ->attach('file', file_get_contents($tempFile), 'image.jpg')
                ->post($backendUrl . '/predict');

            // Limpiar archivo temporal
            unlink($tempFile);

            if ($response->successful()) {
                return [
                    'success' => true,
                    'data' => $response->json()
                ];
            } else {
                Log::error('Error en API de predicción: ' . $response->body());
                return [
                    'success' => false,
                    'message' => 'Error en la API de predicción: ' . $response->body()
                ];
            }
        } catch (\Exception $e) {
            Log::error('Error en predicción simple: ' . $e->getMessage());
            return [
                'success' => false,
                'message' => 'Error al procesar la imagen: ' . $e->getMessage()
            ];
        }
    }
```

Figura 51 - Función de envío de imagen al backend

- *Integración con el backend:*

Esta función maneja el proceso completo de evaluación de un paciente con la imagen capturada. Primero, verifica que se haya seleccionado un paciente y que exista la una imagen adjuntada; si falta alguno de estos requisitos, muestra un mensaje de error "Debe seleccionar un paciente y agregar una imagen" y detiene el proceso. Si las validaciones son correctas, activa un indicador de evaluación en progreso y envía una petición POST a la ruta '/evaluacion/predecir' con el ID del paciente seleccionado y la imagen seleccionada. Cuando la petición es exitosa, Inertia maneja automáticamente la redirección a la página de resultados. Si ocurre un error durante la petición, se muestra al usuario mediante una notificación.

```javascript
const handleEvaluatePatient = async () => {
        if (!selectedPatient || capturedImages.length === 0) {
            toast.error('Debe seleccionar un paciente y agregar al menos una imagen');
            return;
        }

        setIsEvaluating(true);

        try {
            router.post(
                '/evaluacion/predecir',
                {
                    paciente_id: selectedPatient.id,
                    imagenes: capturedImages,
                },
                {
                    onSuccess: () => {
                        // La redirección se maneja automáticamente por Inertia
                    },
                    onError: (errors) => {
                        console.error('Error:', errors);

                        let errorMessage = 'Error al procesar la evaluación';

                        if (errors.prediccion) {
                            errorMessage = errors.prediccion;
                        } else if (typeof errors === 'string') {
                            errorMessage = errors;
                        } else if (errors.message) {
                            errorMessage = errors.message;
                        }

                        toast.error(errorMessage);
                    },
                    onFinish: () => {
                        setIsEvaluating(false);
                    },
                },
            );
        } catch (error) {
            console.error('Error:', error);
            toast.error('Error de conexión al procesar la evaluación');
            setIsEvaluating(false);
        }
    };
```

Figura 52 - Uso del API de predicción en el frontend

- *Funcionalidad en uso:*

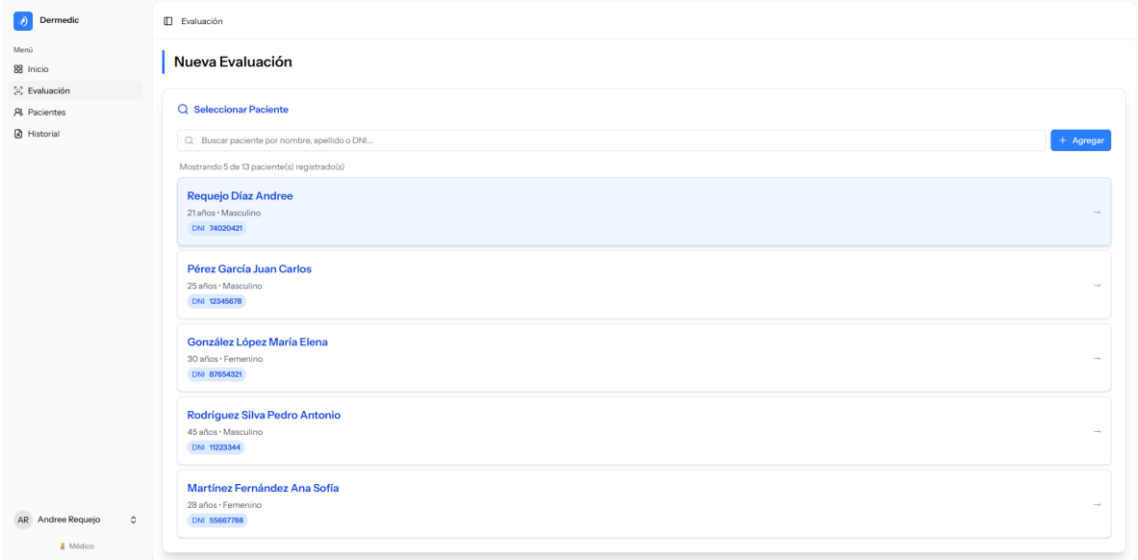


Figura 53 - Selección de paciente para predicción

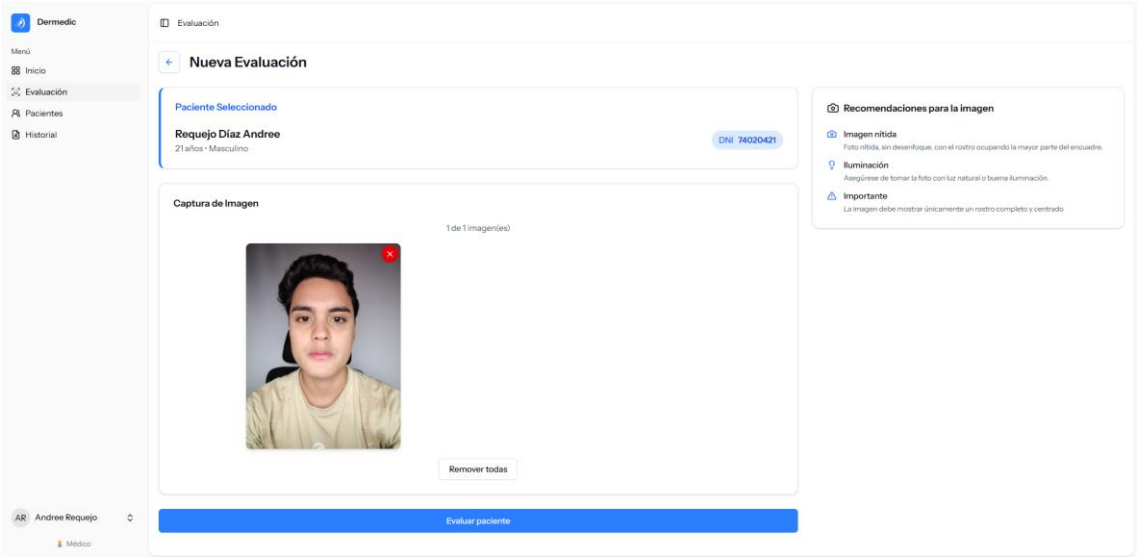


Figura 54 - Adjuntar imagen para predicción

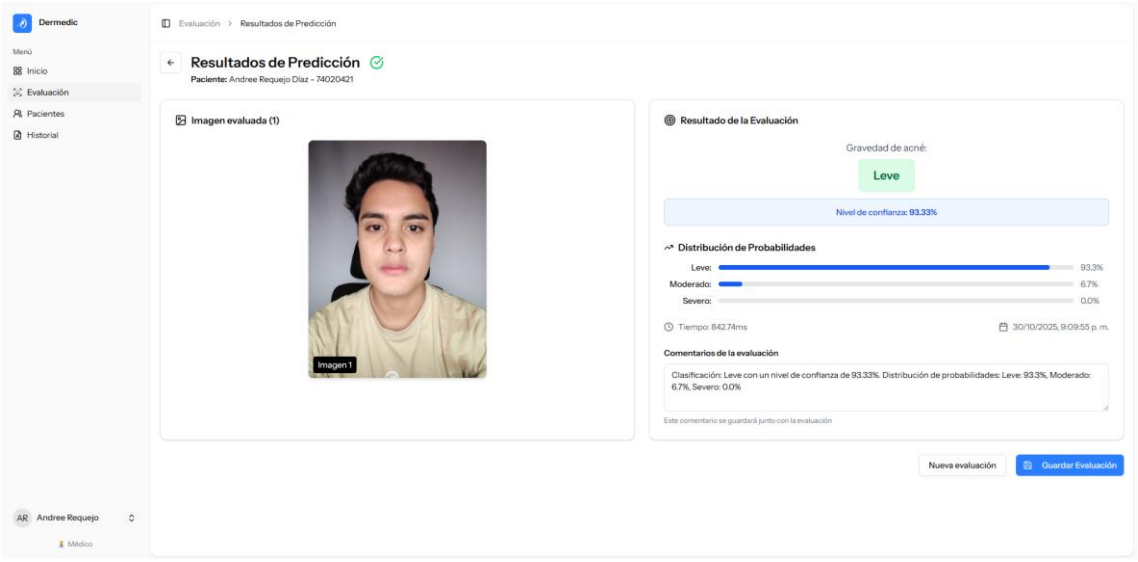


Figura 55 - Interfaz de predicción de gravedad de acné

- **Registro de evaluación de gravedad de paciente:**
  - *Funciones implementadas:*

Esta función maneja la creación y almacenamiento de una nueva evaluación en el sistema. Primero, valida que todos los datos recibidos cumplan con las reglas establecidas: el paciente debe existir en la base de datos, la clasificación debe ser uno de los valores permitidos. Una vez validados los datos, inicia una transacción en la base de datos para garantizar la consistencia de los datos. Prepara un array con la información de la evaluación incluyendo el ID del paciente, la clasificación, el comentario, la fecha y hora actual, la hora en formato de tiempo, la confianza, el tiempo de procesamiento, y las probabilidades convertidas a formato JSON si existen (o null si no). Luego, crea el registro de la evaluación en la base de datos. A continuación, limpia el formato base64 eliminando el encabezado y crea un registro de imagen asociado a la

evaluación con el contenido en base64. Después de guardar todos los datos exitosamente, confirma la transacción en la base de datos y redirige al usuario a la página de historial mostrando un mensaje de éxito. Si ocurre algún error durante el proceso, revierte todos los cambios realizados en la base de datos, registra el error en el log del sistema y redirige al usuario de vuelta mostrando un mensaje de error junto con los detalles técnicos del problema.



```php
/**
     * Crear una nueva evaluación
     */
    public function store(Request $request)
    {
        $request->validate([
            'paciente_id' => 'required|exists:paciente,id',
            'clasificacion' => 'required|in:Ausente,Leve,Moderado,Severo',
            'comentario' => 'nullable|string',
            'imagenes' => 'required|array|min:1|max:3',
            'imagenes.*' => 'required|string', // Base64 strings
            // Campos opcionales para predicción automática
            'es_prediccion_automatica' => 'nullable|boolean',
            'confianza' => 'nullable|numeric|between:0,1',
            'tiempo_procesamiento' => 'nullable|numeric',
            'probabilidades' => 'nullable|array'
        ]);

        try {
            DB::beginTransaction();

            // Preparar datos de la evaluación
            $evaluacionData = [
                'paciente_id' => $request->paciente_id,
                'clasificacion' => $request->clasificacion,
                'comentario' => $request->comentario,
                'fecha' => now()->toDateString(),
                'hora' => now()->toTimeString(),
                'confianza' => $request->confianza,
                'tiempo_procesamiento' => $request->tiempo_procesamiento,
                'probabilidades' => $request->probabilidades ? json_encode($request->probabilidades) : null
            ];

            // Crear la evaluación
            $evaluacion = Evaluacion::create($evaluacionData);

            // Crear las imágenes asociadas
            foreach ($request->imagenes as $imagenBase64) {
                // Remover el prefijo data:image si existe
                $imagenBase64 = preg_replace('/^data:image\/[^;]+; base64,/', '', $imagenBase64);

                Imagen::create([
                    'evaluacion_id' => $evaluacion->id,
                    'contenido_base64' => $imagenBase64
                ]);
            }

            DB::commit();

            // Retornar redirección de Inertia en lugar de JSON
            return redirect()->route('historial')
                ->with('success', 'Evaluación guardada exitosamente');

        } catch (\Exception $e) {
            DB::rollBack();
            Log::error('Error al crear evaluación: ' . $e->getMessage());

            // Retornar con errores en lugar de JSON
            return redirect()->back()
                ->withErrors(['general' => 'Error al guardar la evaluación: ' . $e->getMessage()])
                ->withInput();
        }
    }
```

Figura 56 - Función de registro de evaluación a paciente

- *Integración con el backend:*

Esta función maneja el guardado de una evaluación después de que se ha realizado la predicción automática. Cuando el usuario hace clic en "Guardar evaluación", la función activa un indicador de guardado en progreso e inicializa las variables para clasificación, comentario, confianza, tiempo de procesamiento y probabilidades. Luego, envía una petición POST a la ruta '/evaluaciones' con todos los datos: el ID del paciente, la clasificación, el comentario, la imagen, la confianza, el tiempo de procesamiento y las probabilidades. Si la evaluación se guarda exitosamente, muestra un mensaje de éxito y el controlador maneja automáticamente la redirección. Si ocurre un error durante el guardado muestra un mensaje de error.



```
const handleSave = () => {
        setIsSaving(true);

        let clasificacion = '';
        let comentario = '';
        let confianza = 0;
        let tiempo_procesamiento = 0;
        let probabilidades = null;

        if (isSimplePrediction) {
            const pred = prediccion as PrediccionSimple;
            clasificacion = pred.prediccion_label;
            confianza = pred.confianza;
            tiempo_procesamiento = pred.tiempo_procesamiento;
            comentario = `Evaluación automática con ${pred.confianza_porcentaje} de confianza`;
            probabilidades = pred.probabilidades;
        }

        // Usar la ruta POST /evaluaciones que mapea al método store
        router.post(
            '/evaluaciones',
            {
                paciente_id: paciente.id,
                clasificacion,
                comentario,
                imagenes,
                es_prediccion_automatica: true,
                confianza,
                tiempo_procesamiento,
                probabilidades,
            },
            {
                onSuccess: () => {
                    toast.success('Evaluación guardada exitosamente');
                    // No necesitamos redirigir manualmente, el controlador lo hace automáticamente
                },
                onError: (errors) => {
                    console.error('Errores:', errors);
                    // Mostrar el error específico si existe
                    if (errors.general) {
                        toast.error(errors.general);
                    } else {
                        toast.error('Error al guardar la evaluación');
                    }
                },
                onFinish: () => {
                    setIsSaving(false);
                },
            },
        );
    };
```

Figura 57 - Consumo de API de registro de evaluación

- *Funcionalidad en uso:*

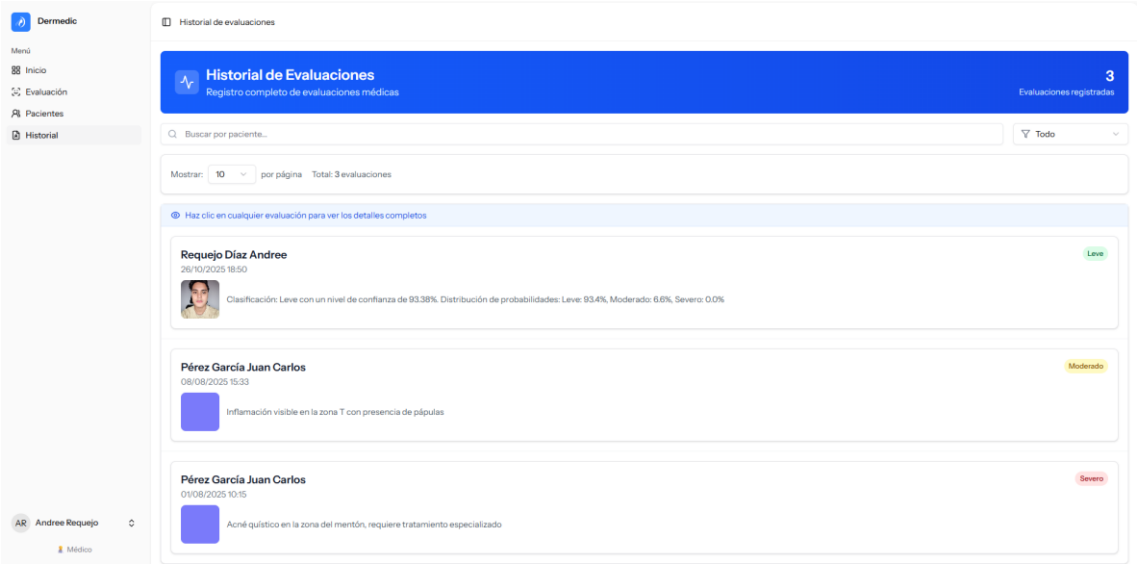


Figura 58 - Visualización del registro de la evaluación realizada

- **Generación de reporte de evaluaciones:**
  - *Funciones implementadas:*

Esta función genera el reporte completo de un paciente con todas sus evaluaciones registradas. Primero, busca al paciente en la base de datos por su ID, incluyendo todas sus evaluaciones ordenadas por fecha de forma descendente (las más recientes primero). Si el paciente no tiene evaluaciones registradas, renderiza la vista 'reports/ReportePaciente' mostrando únicamente los datos básicos del paciente (ID, nombres, apellidos, edad, género, teléfono y DNI), con un array vacío de evaluaciones y un indicador del modal en true para mostrar un modal informativo de que no se tienen evaluaciones registradas para el paciente. Si el paciente tiene evaluaciones, formatea cada evaluación preparando los datos para el frontend: la fecha de la evaluación, el resultado de la clasificación, el comentario, y procesa la imagen asociada convirtiéndola al formato URI seguido del contenido en base64 para que pueda mostrarse directamente en el navegador. Finalmente, renderiza la vista 'reports/ReportePaciente' enviando toda la información del paciente junto con el array de evaluaciones formateadas y el indicador del modal en false para no mostrar el modal.

```php
/**
     * Mostrar el reporte del paciente con sus evaluaciones
     */
    public function reporte($id)
    {
        $paciente = Paciente::with(['evaluaciones' => function($q) {
            $q->orderBy('fecha', 'desc');
        }])->findOrFail($id);

        // Si no tiene evaluaciones, mostrar solo el modal y no abrir el reporte
        if ($paciente->evaluaciones->isEmpty()) {
            return Inertia::render('reports/ReportePaciente', [
                'paciente' => [
                    'id' => $paciente->id,
                    'nombre' => $paciente->nombres,
                    'apellido' => $paciente->apellidos,
                    'edad' => $paciente->edad,
                    'genero' => $paciente->genero,
                    'telefono' => $paciente->telefono,
                    'dni' => $paciente->dni,
                ],
                'evaluaciones' => [],
                'showModalNoEvaluaciones' => true,
            ]);
        }

        // Formatear datos para el frontend
        $evaluaciones = $paciente->evaluaciones->map(function($ev) {
            return [
                'id' => $ev->id,
                'fecha' => $ev->fecha ? $ev->fecha->format('d/m/Y') : '',
                'resultado' => $ev->clasificacion,
                'comentario' => $ev->comentario,
                'imagenes' => $ev->imagenes->map(function($img) {
                    return '' . $img->contenido_base64;
                })->toArray(),
            ];
        });

        return Inertia::render('reports/ReportePaciente', [
            'paciente' => [
                'id' => $paciente->id,
                'nombre' => $paciente->nombres,
                'apellido' => $paciente->apellidos,
                'edad' => $paciente->edad,
                'genero' => $paciente->genero,
                'telefono' => $paciente->telefono,
                'dni' => $paciente->dni,
            ],
            'evaluaciones' => $evaluaciones,
            'showModalNoEvaluaciones' => false,
        ]);
    }
```

Figura 59 - Función de reporte de evaluaciones

- *Funcionalidad en uso:*

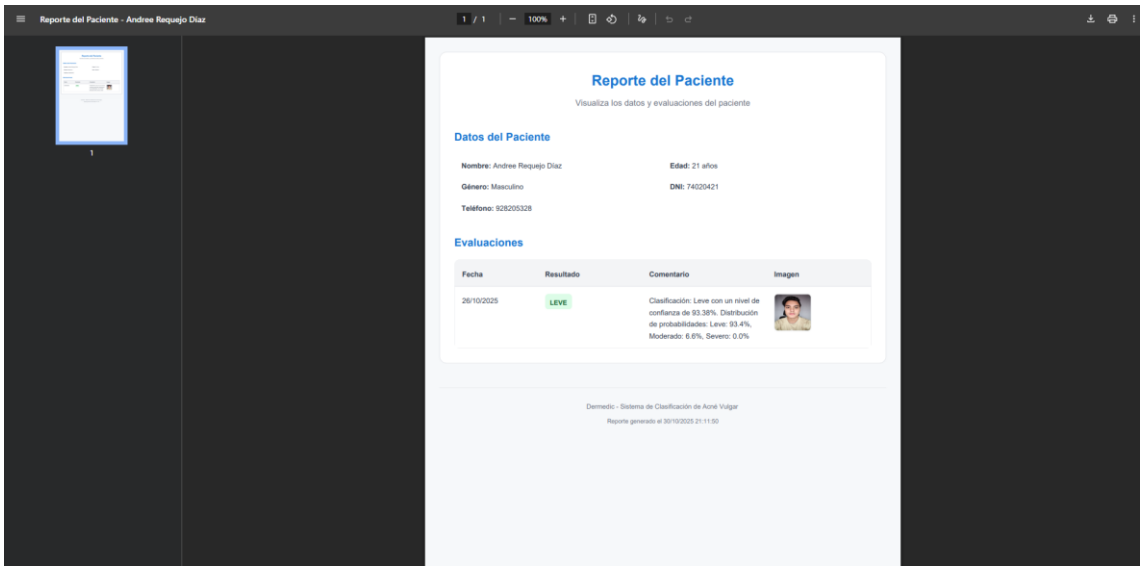


Figura 60 – Reporte generado por el sistema

Sprint Review 7:

Épica	Proceso	Enunciado de la historia	Estado
Evaluación de gravedad acné e historial evaluaciones.	Evaluación de gravedad de acné mediante una imagen facial	Como médico, se necesita evaluar la gravedad del acné de un paciente a partir de una imagen facial cargada en el sistema, con la finalidad de obtener una clasificación automática que apoye el diagnóstico clínico.	Finalizada
	Registro de evaluación de gravedad de paciente	Como médico, se necesita registrar en el sistema los resultados de la evaluación de la gravedad del acné de cada paciente, con la finalidad de llevar un historial clínico que facilite el seguimiento de su evolución.	Finalizada
	Generación de reporte de evaluaciones	Como médico, se necesita generar un reporte con el historial de evaluaciones de cada paciente, con la finalidad de contar con documentación estructurada que respalde el proceso diagnóstico y sirva de apoyo en la toma de decisiones médicas.	Finalizada

1.2. En base a los objetivos del proyecto

1.2.1. Definir el algoritmo de deep learning más adecuado para la clasificación de la gravedad del acné vulgar a partir de imágenes faciales mediante análisis comparativo.

El primer objetivo propuesto consiste en seleccionar mediante un análisis comparativo experimental, el algoritmo de deep learning más adecuado para clasificar la gravedad del acné vulgar en imágenes faciales, priorizando la precisión diagnóstica, robustez y viabilidad de despliegue en una aplicación web. Para esto, se revisaron trabajos recientes sobre clasificación de acné con CNN para identificar arquitecturas comunes.

Las arquitecturas seleccionadas fueron las siguientes:

- **VGG16**, por su eficacia comprobada en tareas de clasificación de imágenes y su uso frecuente en estudios médicos relacionados con acné.
- **ResNet50**, otra arquitectura muy usada para este tipo de trabajos que permite entrenar redes más profundas reduciendo el problema del desvanecimiento del gradiente.
- **EfficientNetV2-S**, una arquitectura más reciente que optimiza la eficiencia del modelo mediante diversas técnicas, lo que la hace atractiva para implementaciones prácticas.

Todos los modelos fueron entrenados bajo las mismas condiciones:

- Optimizador Adam, tasa de aprendizaje inicial 0.0001.
- Tamaño de lote (batch size) de 32 imágenes.
- Early stopping para evitar sobreajuste.
- Métricas de evaluación: **exactitud (accuracy)**, **precisión**, **sensibilidad (recall)** y **F1-Score**.

Los experimentos fueron ejecutados en un entorno de cómputo local con las siguientes especificaciones de hardware y software:

- **Procesador:** AMD Ryzen 5 3600 (6 núcleos, 12 hilos, 3.6 GHz)
- **Memoria RAM:** 16 GB DDR4
- **Tarjeta gráfica:** NVIDIA GeForce RTX 4060 (8 GB VRAM)
- **Sistema operativo:** Windows 11 (64 bits)
- **Entorno de desarrollo:** Python 3.10, PyTorch 2.7, CUDA 12

Los resultados obtenidos se muestran en la siguiente tabla:

Modelo	Exactitud	Precisión	Sensibilidad	F1-Score	Tamaño del modelo	Tiempos de inferencia (ms)
VGG16	84%	86%	84%	85%	468 MB	3.10 ms
ResNet50	79.4%	79%	79%	80%	97 MB	10.20 ms
EfficientNetV2-S	87%	88%	86%	87%	83 MB	5 ms

Los resultados evidencian que EfficientNetV2-S superó a las arquitecturas VGG16 y ResNet50 en todas las métricas de desempeño. Además, el tamaño reducido del modelo y su menor tiempo de inferencia lo convierten en una opción más eficiente para su posterior integración en una aplicación web, sin comprometer la calidad del diagnóstico automatizado.

El objetivo se cumplió satisfactoriamente al identificar, mediante un análisis comparativo, que la arquitectura **EfficientNetV2-S** representa la opción más adecuada para la clasificación de la gravedad del acné vulgar. Dicha selección se basa en su mayor precisión, mejor equilibrio entre rendimiento y eficiencia computacional, y su viabilidad de implementación práctica dentro del sistema propuesto. Los resultados experimentales y evidencias de entrenamiento (curvas de aprendizaje, matrices de confusión y registros de validación) se pueden observar en el **Anexo 01**.

1.2.2. Desarrollar un modelo basado en deep learning para la clasificación automática de la gravedad del acné vulgar en imágenes faciales.

Con base en la arquitectura seleccionada en el objetivo anterior (**EfficientNetV2-S**), se desarrolló un modelo de deep learning especializado para la clasificación automática de la gravedad del acné vulgar. El proceso de desarrollo comprendió varias etapas que permitieron optimizar el rendimiento del modelo y asegurar su capacidad de generalización. En el Sprint 4 se puede observar el desarrollo total de la arquitectura seleccionada

Se partió del modelo preentrenado con el conjunto de datos ImageNet, al cual se le adaptó una arquitectura personalizada que permitió ajustar las salidas a las 3 categorías definidas para la gravedad del acné: leve, moderado

y grave. La capa clasificadora original fue reemplazada por un bloque denso compuesto por varias capas lineales con unidades decrecientes, acompañadas de funciones de activación y técnicas de regularización como dropout. Esta estructura se diseñó para mejorar la capacidad del modelo de identificar patrones dermatológicos relevantes y reducir el sobreajuste durante el entrenamiento.

Los resultados del entrenamiento inicial se mostraron en el objetivo anterior, dando resultados muy buenos con un accuracy del 87%, sin embargo, para poder aumentar aún más su capacidad de generalización el proceso de entrenamiento se desarrolló bajo un esquema **incremental** con validación cruzada por cinco folds. Cada fold utilizó diferentes subconjuntos del total del dataset, asegurando que el modelo se expusiera a imágenes distintas en cada fase de entrenamiento. Este procedimiento permitió reforzar su capacidad de generalización, ya que los pesos obtenidos en un entrenamiento se tomaron como base para el siguiente, generando un aprendizaje acumulativo.

El modelo se entrenó utilizando lotes pequeños de datos provenientes del dataset **ACNE-04** para optimizar el aprendizaje y permitir un mejor ajuste de los parámetros. Se estableció un número moderado de épocas por fold, junto con una tasa de aprendizaje reducida, con el propósito de garantizar un entrenamiento estable y evitar el sobreajuste. Para el primer entrenamiento se utilizaron 15 épocas y 8 para el siguiente entrenamiento incremental.

Como técnica de regularización adicional se aplicó *label smoothing*, lo que permitió suavizar las etiquetas de las clases y disminuir la confianza excesiva del modelo en sus predicciones. El optimizador empleado fue Adam, y las métricas principales de evaluación fueron la exactitud (*accuracy*), precisión, exhaustividad (*recall*) y F1-Score, con el fin de garantizar una evaluación equilibrada entre todas las clases.

Para mejorar la robustez del modelo frente a variaciones reales en las imágenes, se aplicaron técnicas de *aumento de datos (data augmentation)*. Entre ellas se incluyeron rotaciones aleatorias, volteos horizontales y verticales, y transformaciones elásticas, que simularon variaciones naturales en la orientación y en la estructura facial. Durante la fase de validación y

prueba se emplearon únicamente transformaciones de normalización y redimensionamiento, con el fin de evaluar el rendimiento del modelo sin distorsionar las muestras originales.

A lo largo de los ciclos de entrenamiento, se observó una tendencia progresiva de mejora en las métricas de evaluación, reflejando la efectividad del aprendizaje incremental. El modelo logró alcanzar niveles de precisión y F1-score superiores al 87 % inicial, evidenciando una adecuada capacidad para distinguir entre los diferentes grados de severidad del acné. Este modelo **constituye el componente central del sistema de clasificación** de gravedad del acné vulgar que se integrará posteriormente en la aplicación web desarrollada como parte del siguiente objetivo.

1.2.3. Desarrollar una aplicación web responsive que integre el modelo de clasificación de gravedad del acné vulgar, para facilitar su uso por parte del especialista.

Con el propósito de facilitar el uso del modelo de clasificación desarrollado, se implementó una aplicación web responsive que permite a los especialistas subir imágenes faciales de pacientes y obtener de manera automática la predicción del grado de gravedad del acné vulgar.

La aplicación fue construida utilizando el framework Laravel en el lado del servidor y React con Inertia.js en el lado del cliente, tecnologías que permiten una integración fluida entre el backend y frontend. Este enfoque permitió mantener una única base de código y reducir el tiempo de carga. El sistema se estructuró en 3 módulos principales: gestión de pacientes, historial de evaluaciones y el módulo de clasificación de gravedad de acné.

El modelo de clasificación desarrollado con **EfficientNetV2-S** fue exportado y desplegado como un servicio en el backend, de modo que las predicciones se ejecutan de forma automática tras la carga de una imagen. Para lograr esta integración se realizó **mediante API REST** hecha con FastAPI, que permite enviar las imágenes al servidor, procesarlas con el modelo y retornar la clase predicha junto con su probabilidad.

La aplicación fue sometida a pruebas funcionales y de rendimiento para asegurar su correcto funcionamiento. Durante las pruebas, se verificó que la integración del modelo no afectara el tiempo de respuesta del sistema y que los resultados de clasificación sean consistentes. Asimismo, se comprobó la

correcta visualización y adaptabilidad de la interfaz en diferentes tamaños de pantalla y navegadores.

El objetivo se cumplió satisfactoriamente, logrando desarrollar una aplicación web completa, moderna y adaptable, que integra de forma eficiente el modelo de deep learning entrenado para la clasificación de la gravedad del acné vulgar. Permitiendo a los especialistas realizar evaluaciones automáticas, rápidas y confiables, contribuyendo al apoyo diagnóstico y al seguimiento clínico de los pacientes.

1.2.4. Evaluar el rendimiento del sistema desarrollado mediante métricas de desempeño como exactitud, precisión, sensibilidad y F1-Score.

Para comprobar la efectividad del sistema propuesto, se realizó una evaluación exhaustiva del modelo de clasificación basado en la arquitectura **EfficientNetV2-S**, utilizando métricas ampliamente empleadas en el ámbito del deep learning para tareas de clasificación multiclase. El análisis se centró en determinar la capacidad del modelo para identificar correctamente los distintos grados de severidad del acné vulgar a partir de imágenes faciales.

El proceso de evaluación se llevó a cabo utilizando un conjunto de imágenes independiente, no empleado durante las fases de entrenamiento ni validación. Este conjunto permitió estimar el rendimiento real del modelo ante datos no vistos previamente, garantizando una medición imparcial de su capacidad de generalización.

Las métricas consideradas fueron:

- **Exactitud (Accuracy):** proporción de predicciones correctas sobre el total de muestras.
- **Precisión (Precisión):** proporción de verdaderos positivos respecto a las predicciones positivas realizadas.
- **Sensibilidad o exhaustividad (Recall):** capacidad del modelo para identificar correctamente los casos positivos.
- **F1-Score:** medida armónica entre precisión y sensibilidad, que refleja el equilibrio general del modelo.

Los resultados promedio alcanzados por el modelo fueron los siguientes:

Métrica	Valor (%)

Accuracy	93%
Precision	94%
Recall	93%
F1-Score	94%

Estos valores reflejan un **alto nivel de desempeño**, especialmente en la capacidad del modelo para identificar correctamente los distintos grados de severidad del acné. El análisis de la matriz de confusión evidenció que la mayoría de los errores de clasificación se produjeron entre las clases moderado y leve, dada su similitud visual. El patrón de errores muestra transiciones lógicas principalmente entre clases adyacentes en severidad, sin saltos abruptos entre categorías extremas (leve directamente a severo). Para observar más a detalle los resultados obtenidos se puede observar las evidencias adjuntadas en el **Anexo 02**.

El objetivo se cumplió satisfactoriamente, ya que se logró evaluar cuantitativamente el rendimiento del sistema mediante métricas reconocidas en el campo del aprendizaje profundo. Los resultados obtenidos demuestran que el modelo implementado alcanza un alto nivel de precisión y generalización, lo que valida su uso para la clasificación automática de la gravedad del acné vulgar.

1.3. Anexos

1.3.1. Anexo 01:

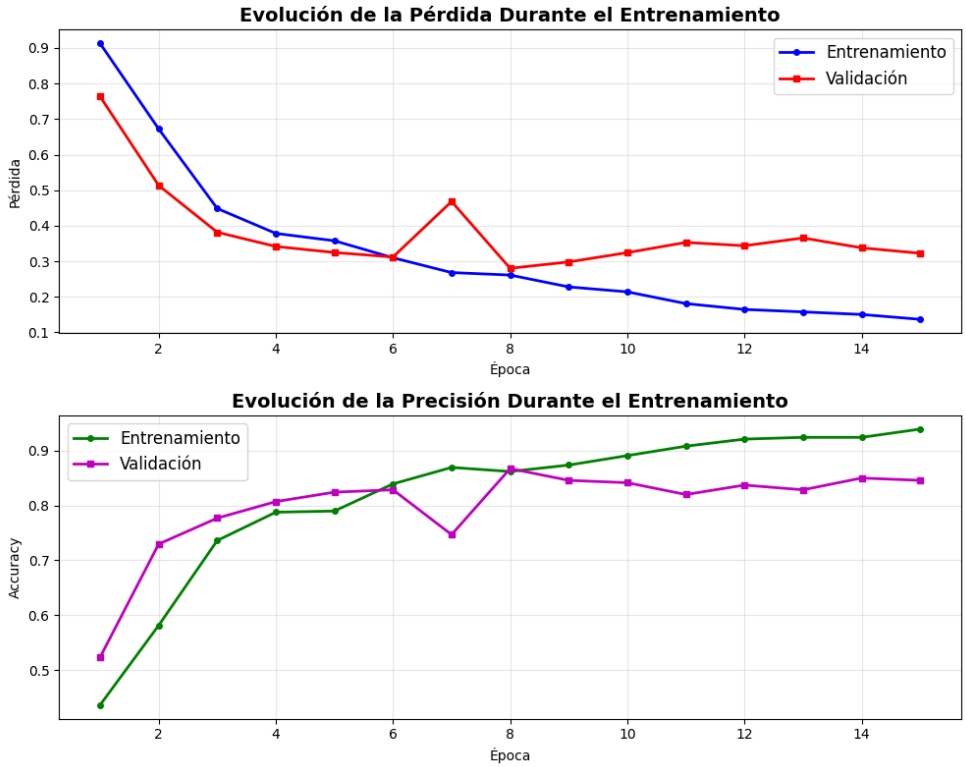


Figura 61 - Curvas de aprendizaje de EfficientNet

```
==================================================
ESTADÍSTICAS DE ENTRENAMIENTO
==================================================
Mejor accuracy de entrenamiento: 0.9388
Mejor accuracy de validación: 0.8670
Pérdida final de entrenamiento: 0.1365
Pérdida final de validación: 0.3225
==================================================
```

Figura 62 - Estadísticas numéricas de curvas de entrenamiento EfficientNet

```
============================================================
Reporte de clasificación - Validación:
============================================================
              precision    recall  f1-score   support

       Leve       0.87      0.91      0.89        82
   Moderado       0.84      0.86      0.85       101
     Severo       0.93      0.80      0.86        50

   accuracy                           0.87       233
  macro avg       0.88      0.86      0.87       233
weighted avg       0.87      0.87      0.87       233

============================================================
```

Figura 63 - Métricas de rendimiento de EfficientNet

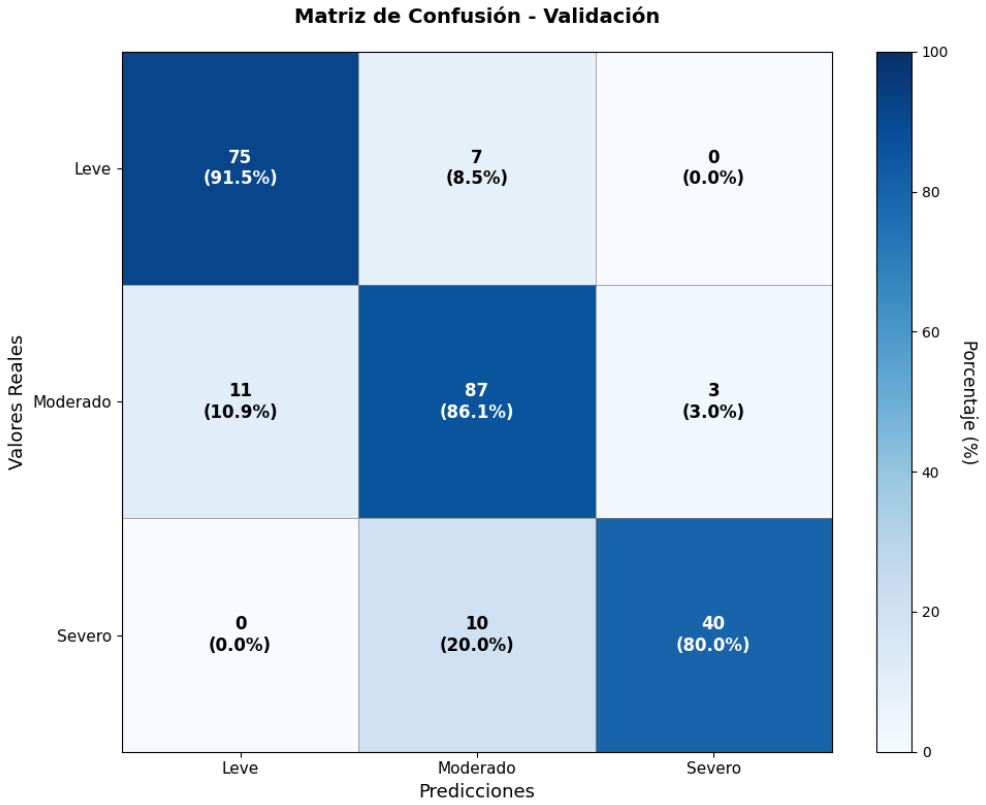


Figura 64 - Matriz de confusión de EfficientNet

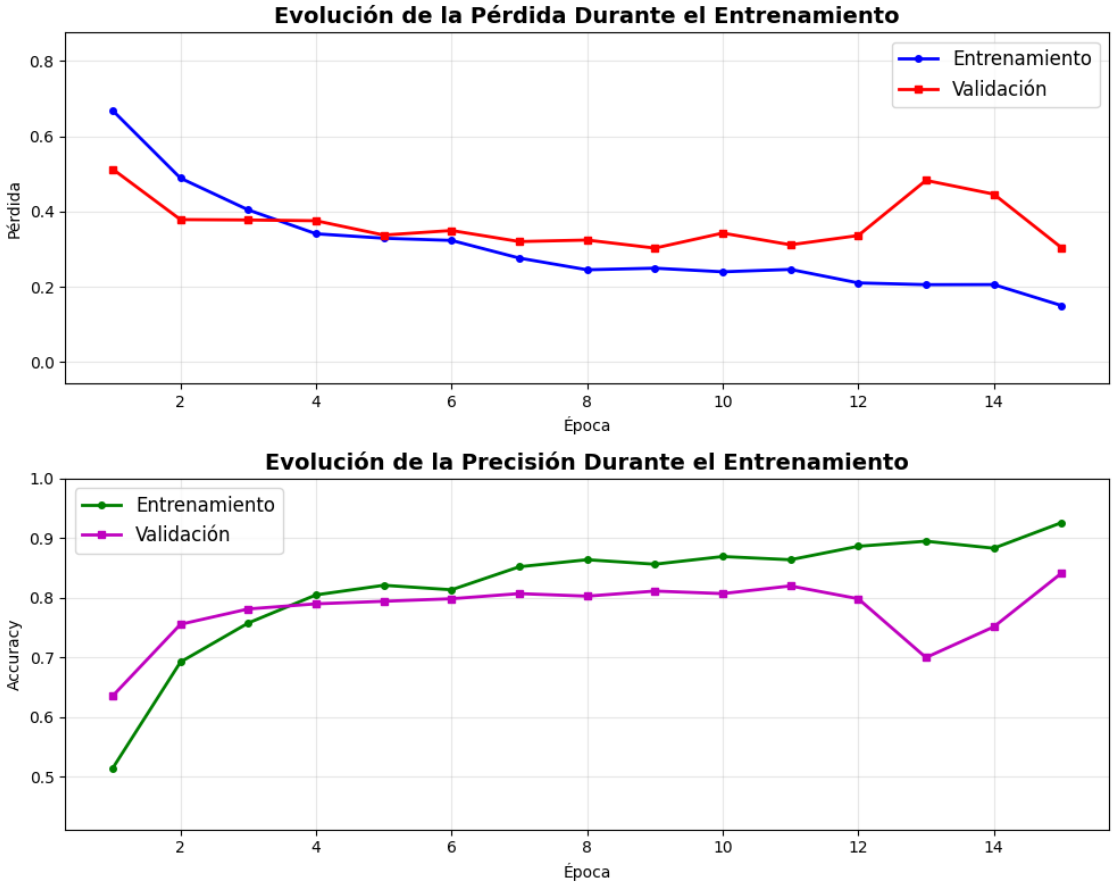


Figura 65 - Curva de aprendizaje de VGG16

```
==================================================
ESTADÍSTICAS DE ENTRENAMIENTO
==================================================
Accuracy de entrenamiento: 0.9260
Accuracy de validación: 0.8412
Pérdida de entrenamiento: 0.1505
Pérdida de validación: 0.3041
==================================================
```

Figura 66 - Estadísticas numéricas de curva de entrenamiento de VGG16

```
============================================================
Reporte de clasificación - Validación:
============================================================
              precision    recall  f1-score   support

       Leve       0.88      0.79      0.83        82
   Moderado       0.79      0.86      0.82       101
     Severo       0.90      0.88      0.89        50

   accuracy                           0.84       233
  macro avg       0.86      0.84      0.85       233
weighted avg      0.84      0.84      0.84       233

============================================================
```

Figura 67 - Métricas de rendimiento de VGG16

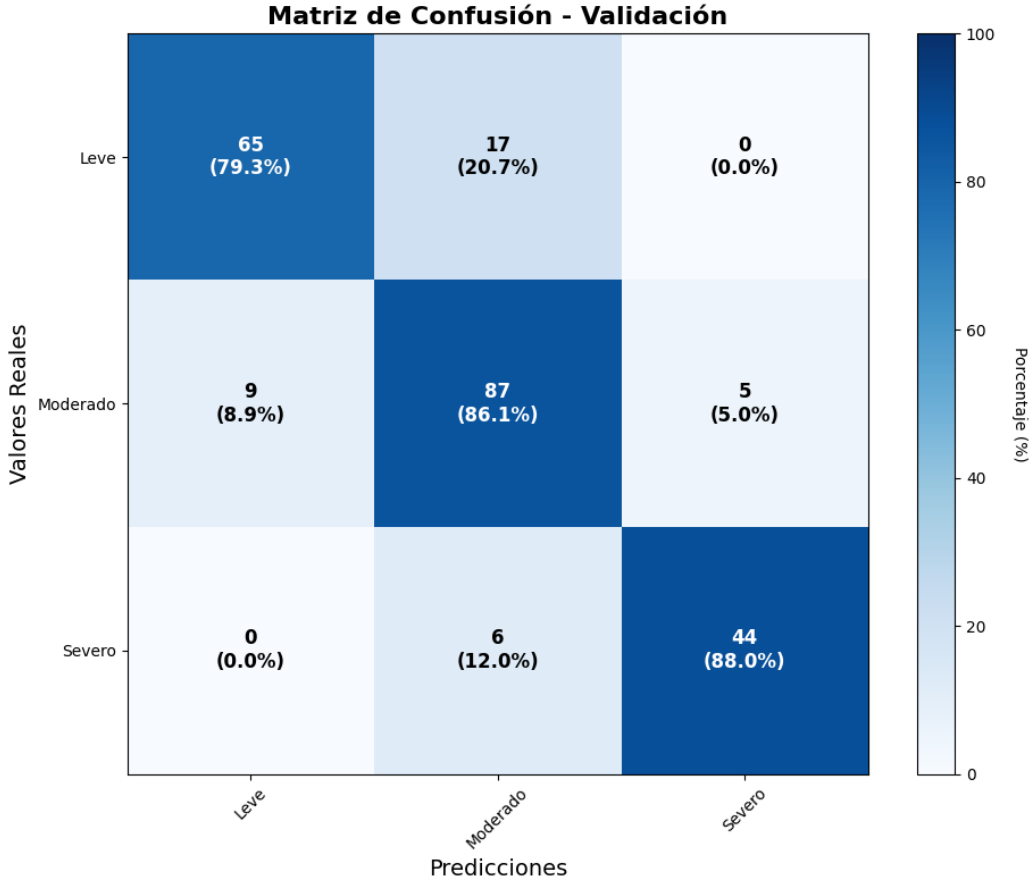


Figura 68 - Matriz de confusión VGG16

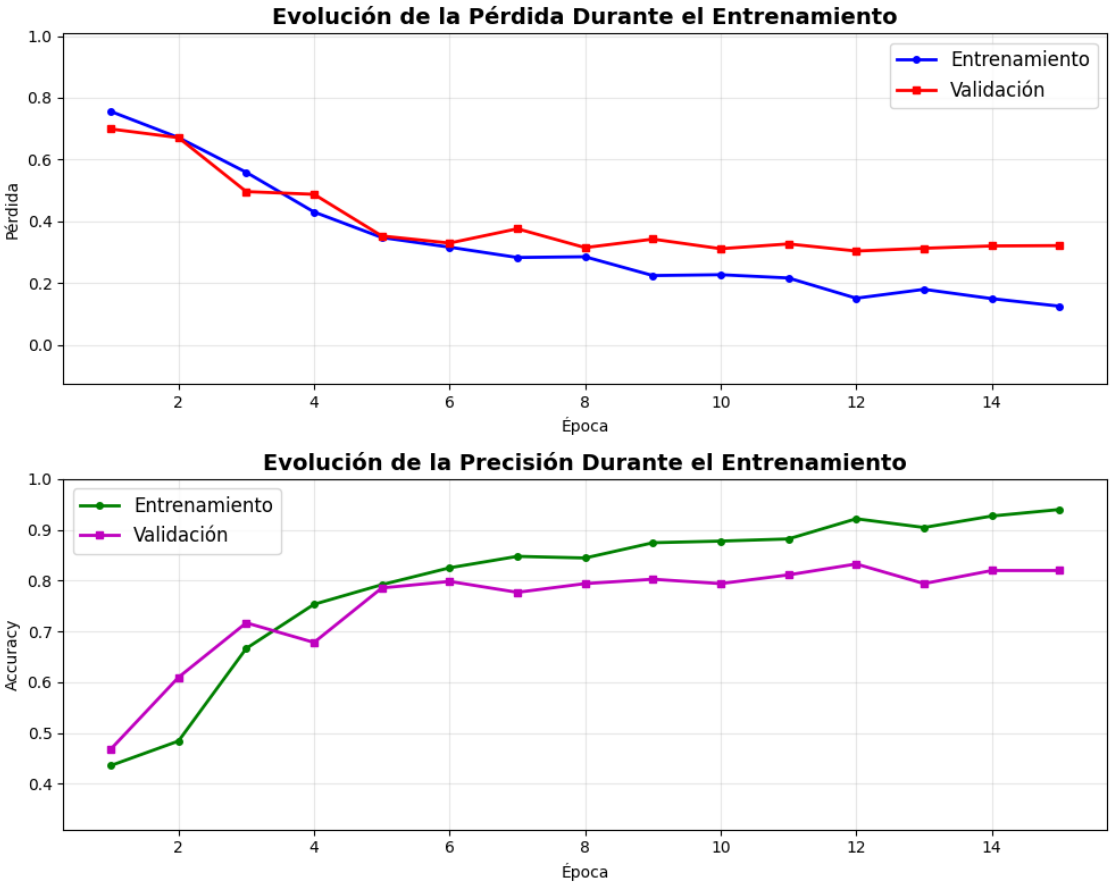


Figura 69 - Curva de aprendizaje de Resnet50

```
==================================================
ESTADÍSTICAS DE ENTRENAMIENTO
==================================================
Accuracy de entrenamiento: 0.9399
Accuracy de validación: 0.8326
Pérdida de entrenamiento: 0.1259
Pérdida de validación: 0.3219
==================================================
```

Figura 70 - Estadísticas numéricas de curva de entrenamiento de Resnet50

```
============================================================
Reporte de clasificación - Validación:
============================================================
              precision    recall  f1-score   support

       Leve       0.84      0.79      0.82        82
    Moderado      0.79      0.84      0.81       101
     Severo       0.92      0.88      0.90        50

   accuracy                           0.83       233
  macro avg       0.85      0.84      0.84       233
weighted avg      0.83      0.83      0.83       233

============================================================
```

Figura 71 - Métricas de rendimiento de Resnet50

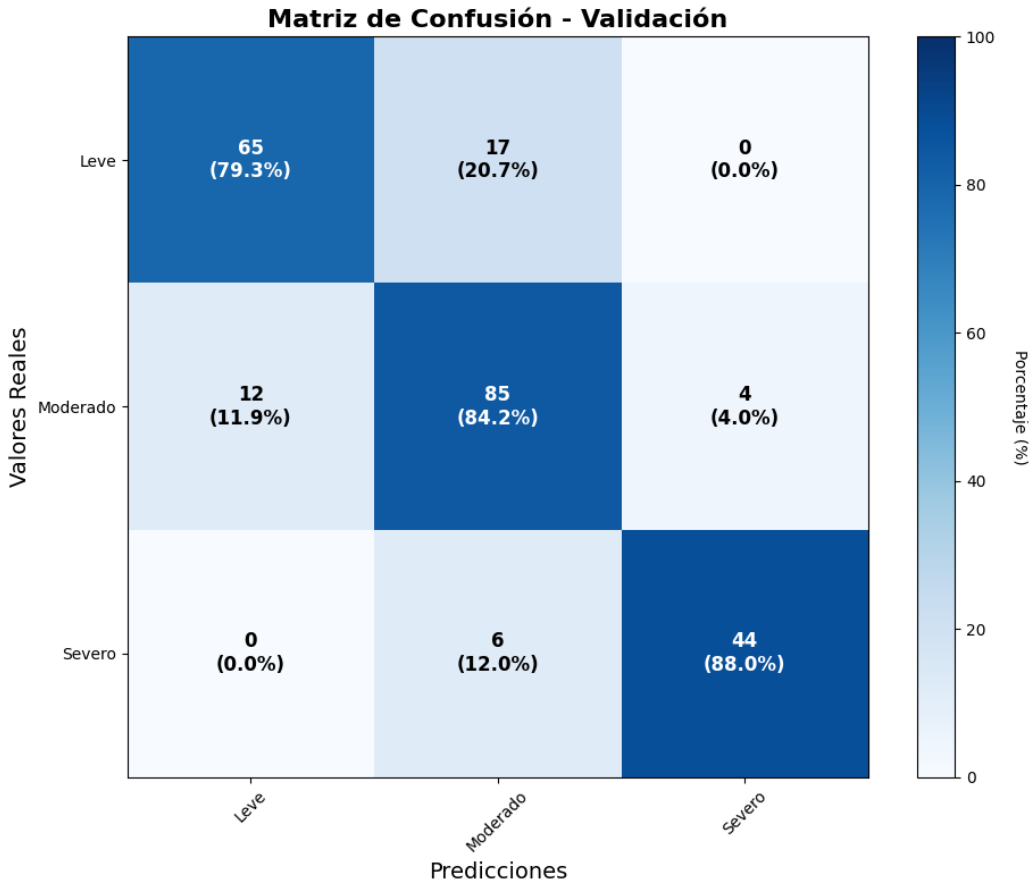


Figura 72 - Matriz de confusión Resnet50

1.3.2. Anexo 02:

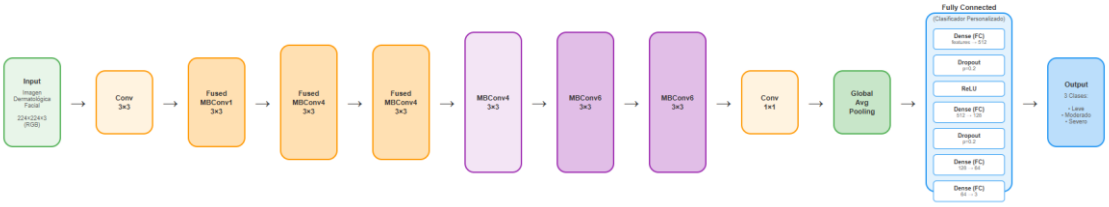


Figura 73 - Arquitectura personalizada EfficientNetV2_S

1.3.3. Anexo 03:

Precisión de Validación: 0.9270 (92.70%)

Reporte de Clasificación - Conjunto de Validación:

	precision	recall	f1-score	support
Leve	0.87	0.96	0.91	82
Moderado	0.95	0.88	0.91	101
Severo	1.00	0.96	0.98	50
accuracy			0.93	233
macro avg	0.94	0.93	0.94	233
weighted avg	0.93	0.93	0.93	233

Figura 74 - Métricas de entrenamiento finales del modelo EfficientNetV2-S

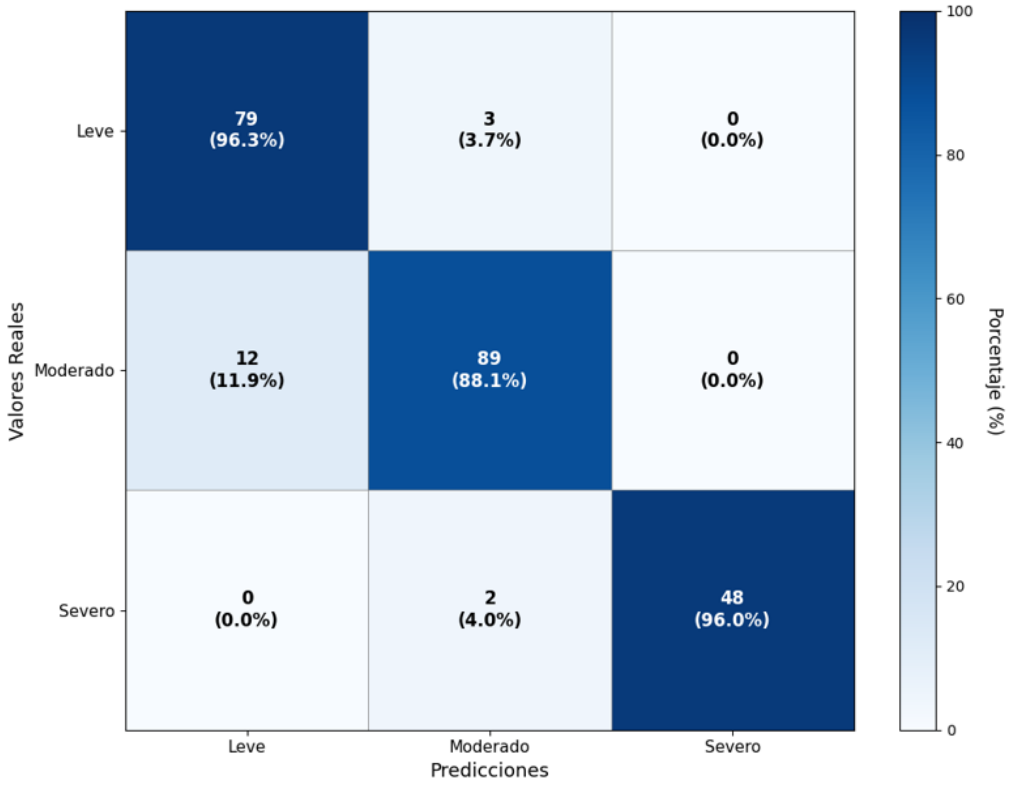


Figura 75 - Matriz de confusión final de EfficientNetV2-S

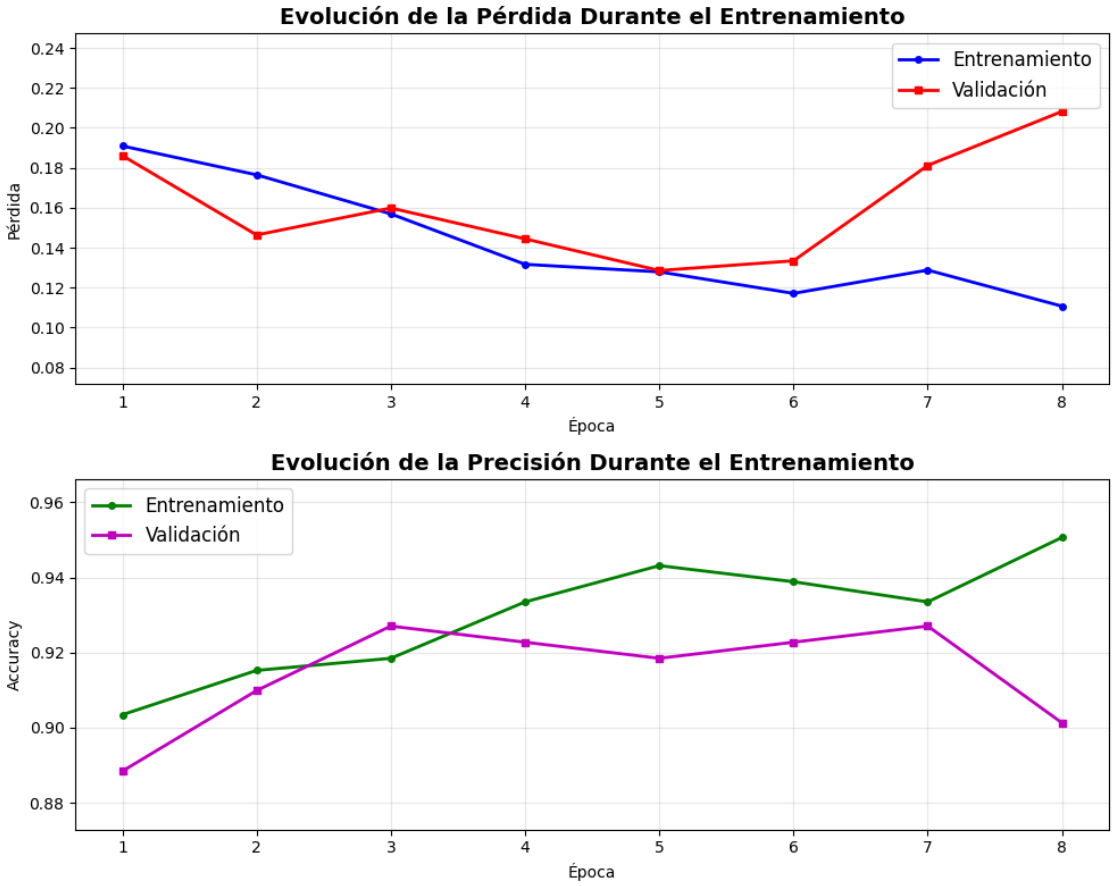


Figura 76 - Curvas de entrenamiento finales del modelo EfficientNetV2-S

1.3.4. Anexo 04:

Las pruebas se realizaron aplicando dos enfoques complementarios:

- **Pruebas de Caja Negra:**

  Se basaron en los requerimientos funcionales definidos. Se diseñaron casos con entradas válidas e inválidas para verificar el comportamiento esperado del sistema.

- **Pruebas de Caja Blanca:**

  Se ejecutaron sobre componentes internos del sistema, verificando el flujo lógico de las funciones principales, las condiciones y sentencias de control, así como la correcta integración entre módulos.

II. Pruebas de Software

2.1. Pruebas de caja negra

ID	Módulo	Caso de Prueba	Descripción	Datos de Entrada	Resultado Esperado	Resultado Obtenido	Estado
CN-01	Autenticación	Inicio de sesión con credenciales válidas	Verificar que el usuario pueda acceder correctamente al sistema con datos válidos.	Correo: andree09requejo@gmail.com, Contraseña: 159357An@	El sistema permite el acceso y redirige al panel principal.	Acceso exitoso.	Aprobado
CN-02	Autenticación	Inicio de sesión con credenciales incorrectas	Comprobar que el sistema rechace el acceso con datos inválidos.	Usuario: dermedic Contraseña: abc123	El sistema muestra mensaje “Credenciales incorrectas”.	Mensaje mostrado correctamente.	Aprobado

CN-03	Gestión de pacientes	Registro de paciente nuevo	Validar que se registre un paciente con datos válidos.	Nombre: Juan Pérez, DNI: 74125896, Edad: 24	El sistema guarda los datos y muestra mensaje de confirmación.	Paciente registrado exitosamente.	Aprobado
CN-04	Gestión de pacientes	Registro de paciente duplicado	Verificar restricción ante intento de duplicar un paciente.	DNI existente: 74125896	El sistema muestra error “El dni ingresado ya existe”.	Error mostrado correctamente.	Aprobado
CN-05	Gestión de pacientes	Eliminación de un paciente con evaluaciones realizadas	Verificar restricción ante el intento de eliminar un paciente con evaluaciones realizadas	Solicitud de eliminación de un paciente que tiene evaluaciones asociadas.	El sistema muestra error “El paciente tiene evaluaciones registradas”	Error mostrado correctamente.	Aprobado

CN-06	Clasificación	Clasificación de imagen válida	Validar que el modelo devuelva correctamente la gravedad del acné.	Imagen válida de paciente	El sistema devuelve nivel de severidad	Resultado correcto.	Aprobado
CN-07	Clasificación	Imagen sin rostro detectado	Evaluar respuesta del sistema ante imagen inválida para el modelo.	Imagen sin rostro humano	El sistema muestra mensaje “No se detecta un rostro válido”.	Mensaje mostrado correctamente.	Aprobado
CN-08	Evaluación	Registro de evaluación en base de datos	Verificar que el resultado de clasificación se guarde correctamente.	Resultado de clasificación, comentario clínico e imagen ingresada	Evaluación almacenada correctamente en la base de datos.	Registro correcto.	Aprobado
CN-09	Historial de evaluaciones	Consulta de evaluaciones previas de un paciente	Validar que el sistema muestre el historial de evaluaciones de un paciente.	DNI: 74125896	Lista de evaluaciones previas visibles correctamente.	Historial de evaluaciones mostrado.	Aprobado

CN-10	Reportes	Generación de reporte de evaluaciones de un paciente	Verificar que el sistema genera correctamente el reporte de evaluaciones asociadas a un paciente seleccionado.	Selección de un paciente con evaluaciones registradas y solicitud de generación de reporte.	El sistema genera y descarga un archivo PDF con las evaluaciones del paciente	El reporte se genera correctamente mostrando las evaluaciones correspondientes sin errores.	Aprobado

2.1.1. Prueba CN – 01: Inicio de sesión con credenciales válidas

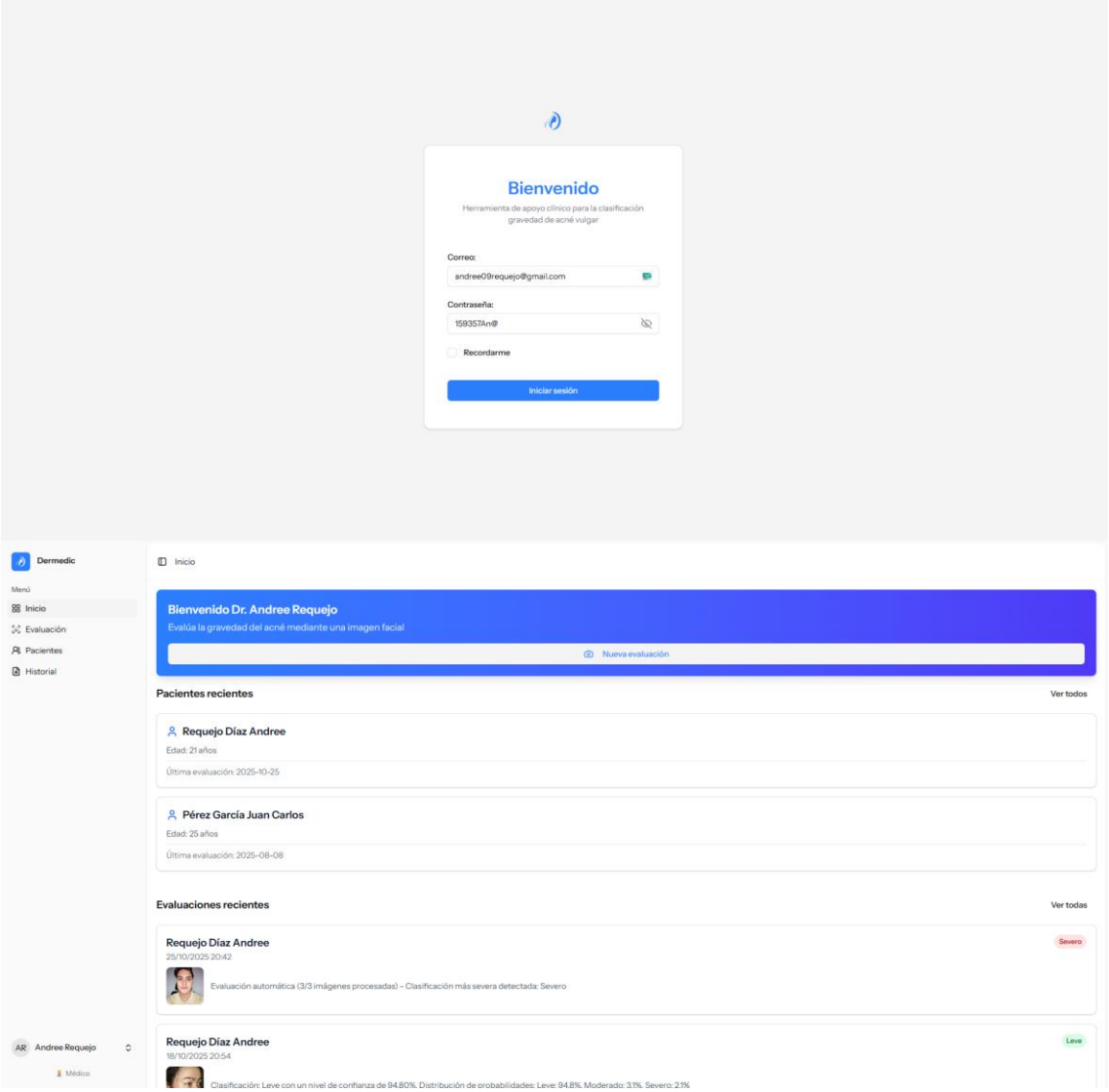
Datos de entrada:

- Correo: andree09requejo@gmail.com
- Contraseña: 159357An@

Procedimiento:

1. Ingresar al formulario de inicio de sesión.
2. Introducir las credenciales válidas.
3. Hacer clic en “Iniciar sesión”.

Resultado esperado:
El sistema permite el acceso y redirige al panel principal.
Resultado obtenido:
Acceso exitoso al sistema; se visualiza el panel principal.
Evidencia:



2.1.2. Prueba CN – 02: Inicio de sesión con credenciales incorrectas

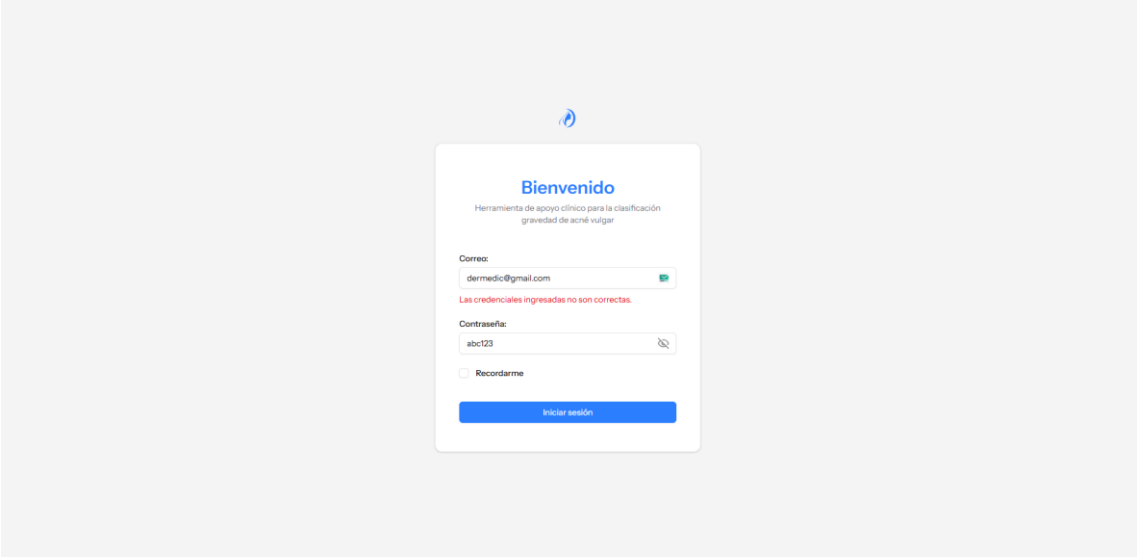
Módulo: Autenticación
Descripción: Comprobar que el sistema rechace el acceso con datos inválidos.
Datos de entrada:

- Usuario: dermedic@gmail.com
- Contraseña: abc123

Procedimiento:

1. Ingresar al formulario de inicio de sesión.
2. Introducir credenciales inválidas.
3. Hacer clic en el botón “Iniciar sesión”.

Resultado esperado:
El sistema muestra el mensaje “Credenciales incorrectas”.
Resultado obtenido:
Mensaje mostrado correctamente y sin acceso al sistema.
Evidencia:



2.1.3. Prueba CN – 03: Registro de paciente nuevo

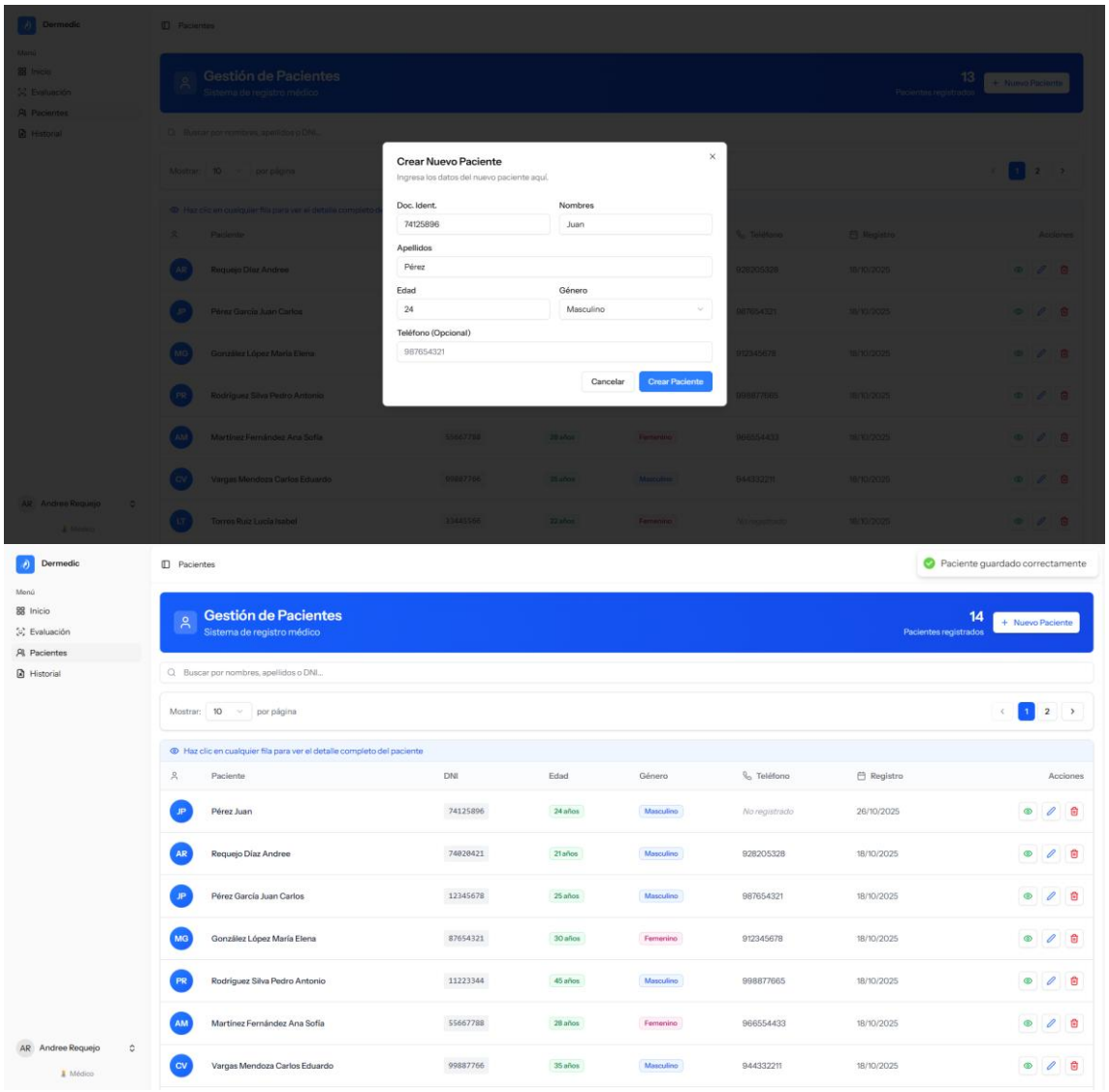
Módulo: Gestión de pacientes
Descripción: Validar que se registre un paciente con datos válidos.
Datos de entrada:

- Nombre: Juan
- Apellidos: Pérez
- DNI: 74125896
- Edad: 24
- Género: Masculino

Procedimiento:

1. Ingresar al módulo “Pacientes”.
2. Seleccionar la opción “Nuevo paciente”.
3. Completar los campos obligatorios y guardar.

Resultado esperado:
El sistema guarda los datos y muestra mensaje de confirmación.
Resultado obtenido:
Paciente registrado exitosamente y visible en la lista de pacientes.
Evidencia:



2.1.4. Prueba CN – 04: Registro de paciente duplicado

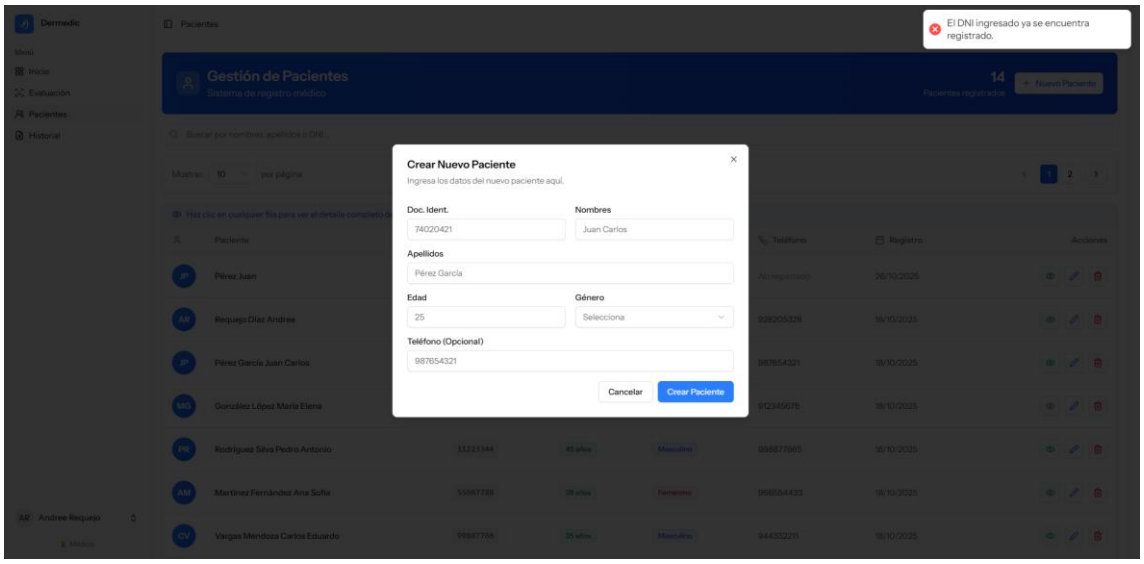
Módulo: Gestión de pacientes
Descripción: Verificar restricción ante intento de duplicar un paciente.
Datos de entrada:

- DNI existente: 74125896

Procedimiento:

1. Intentar registrar un nuevo paciente con el mismo DNI que uno existente.
2. Guardar el formulario.

Resultado esperado:
El sistema muestra el mensaje de error “El DNI ingresado ya existe”.
Resultado obtenido:
Mensaje de error mostrado correctamente y registro no duplicado.
Evidencia:



2.1.5. Prueba CN – 05: Eliminación de un paciente con evaluaciones realizadas

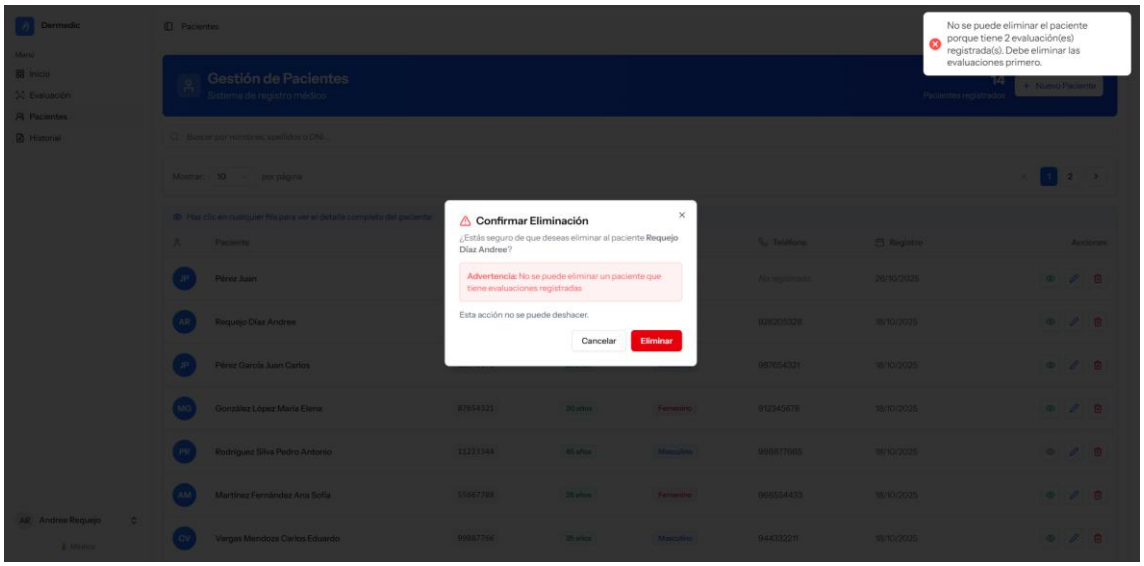
Módulo: Gestión de pacientes
Descripción: Verificar restricción ante el intento de eliminar un paciente con evaluaciones realizadas.
Datos de entrada:

- Solicitud de eliminación de un paciente con evaluaciones asociadas.

Procedimiento:

1. Ingresar al módulo “Pacientes”.
2. Seleccionar un paciente con evaluaciones.
3. Hacer clic en “Eliminar”.

Resultado esperado:
El sistema muestra error “El paciente tiene evaluaciones registradas”.
Resultado obtenido:
Error mostrado correctamente y eliminación impedida.
Evidencia:



2.1.6. Prueba CN – 06: Clasificación de imagen válida

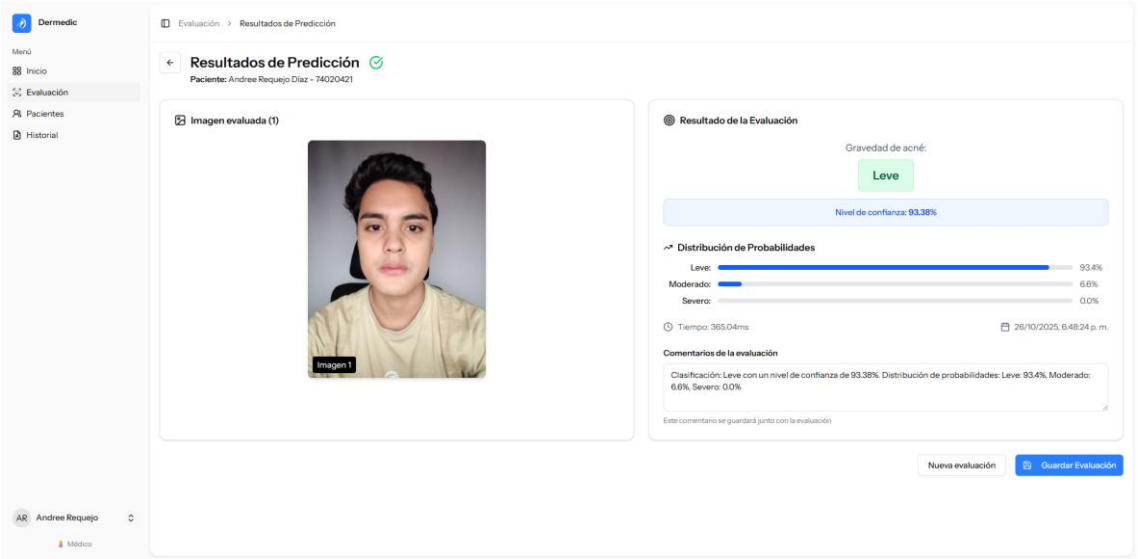
Módulo: Clasificación
Descripción: Validar que el modelo devuelva correctamente la gravedad del acné ante una imagen válida.
Datos de entrada:

- Imagen válida del rostro de un paciente.

Procedimiento:

1. Ingresar al módulo de clasificación.
2. Subir la imagen del paciente.
3. Esperar el resultado del modelo.

Resultado esperado:
El sistema devuelve el nivel de severidad del acné.
Resultado obtenido:
Resultado correcto mostrado con nivel de gravedad y etiqueta correspondiente.
Evidencia:



2.1.7. Prueba CN – 07: Imagen sin rostro detectado

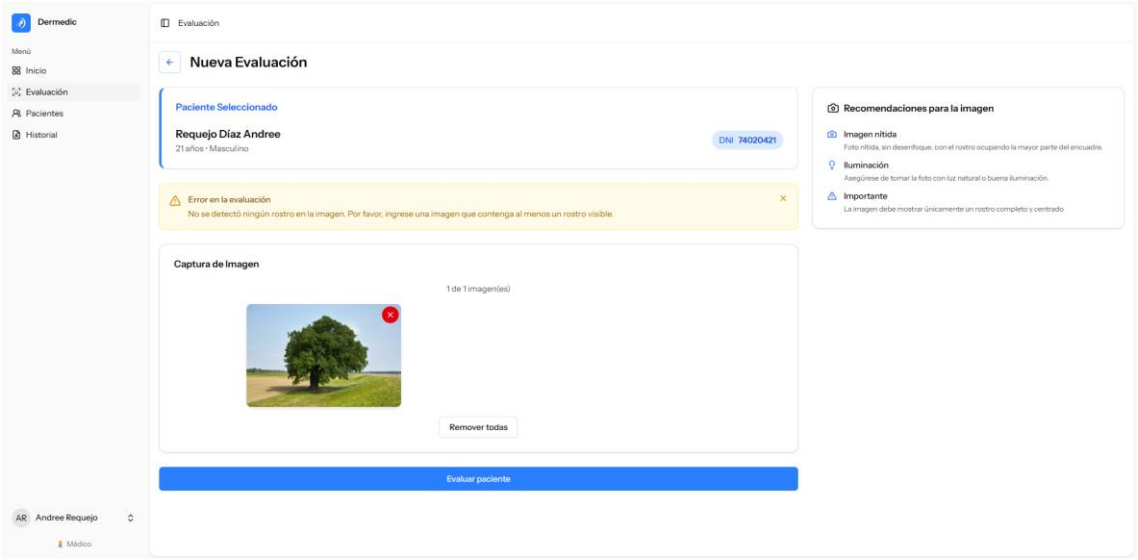
Módulo: Clasificación
Descripción: Evaluar la respuesta del sistema ante una imagen inválida para el modelo.
Datos de entrada:

- Imagen sin rostro humano.

Procedimiento:

1. Cargar una imagen sin rostro en el módulo de clasificación.
2. Iniciar la evaluación.

Resultado esperado:
El sistema muestra el mensaje “No se detecta un rostro válido”.
Resultado obtenido:
Mensaje mostrado correctamente sin procesar la imagen.
Evidencia:



2.1.8. Prueba CN – 08: Registro de evaluación en base de datos

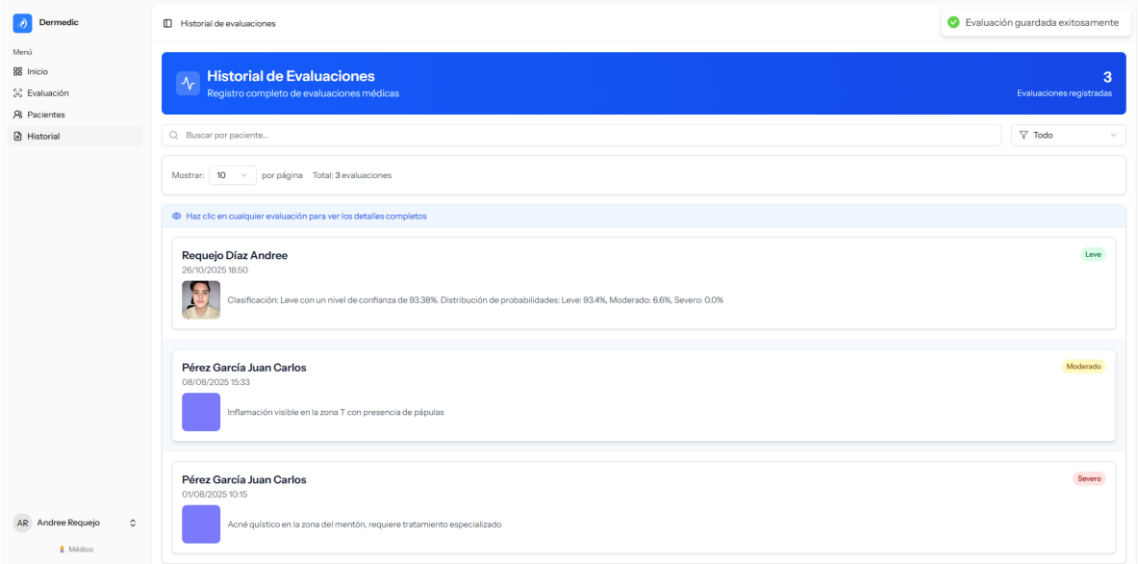
Módulo: Evaluación
Descripción: Verificar que el resultado de clasificación se guarde correctamente en la base de datos.
Datos de entrada:

- Resultado de clasificación, comentario clínico e imagen ingresada.

Procedimiento:

1. Realizar una clasificación de imagen.
2. Ingresar observaciones clínicas.
3. Guardar la evaluación.

Resultado esperado:
La evaluación se almacena correctamente en la base de datos.
Resultado obtenido:
Registro correcto con vinculación entre paciente, imagen y resultado.
Evidencia:



2.1.9. Prueba CN – 09: Consulta de evaluaciones previas de un paciente

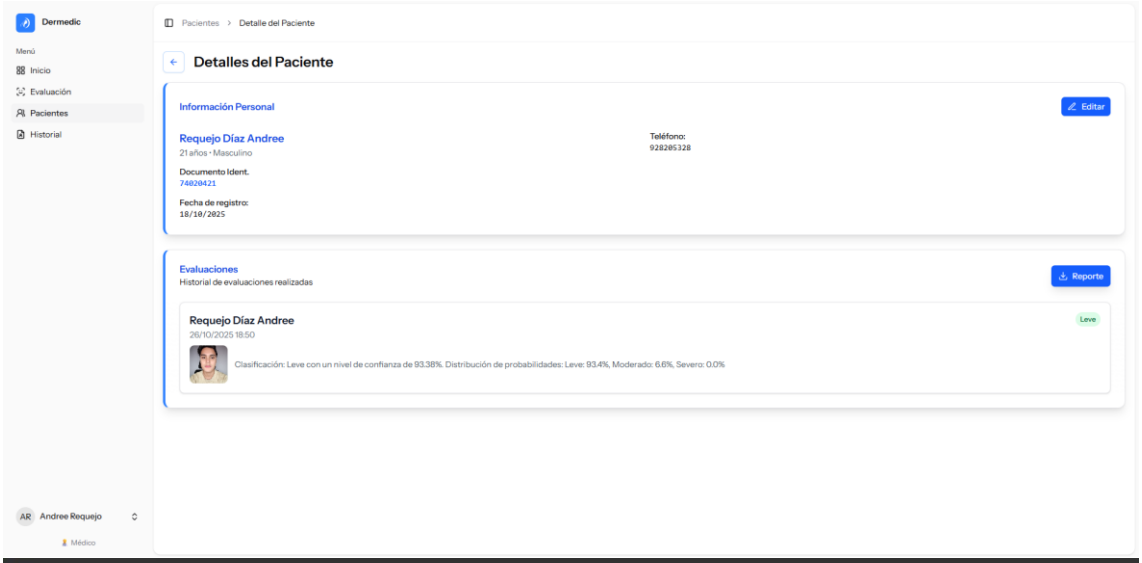
Módulo: Historial de evaluaciones
Descripción: Validar que el sistema muestre el historial de evaluaciones de un paciente.
Datos de entrada:

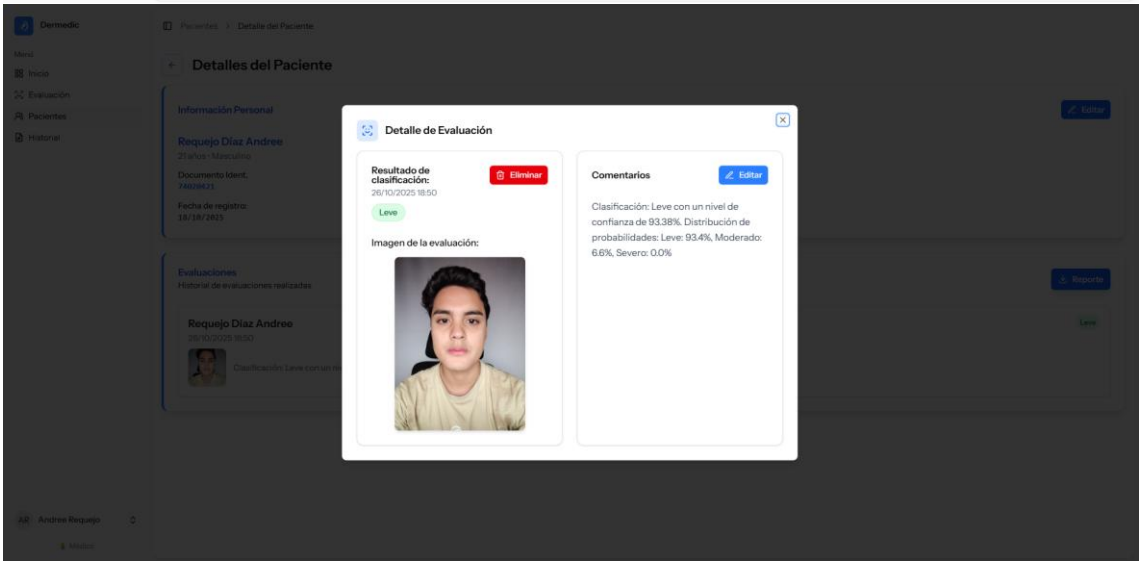
- DNI: 74125896

Procedimiento:

1. Ingresar al módulo “Pacientes”.
2. Buscar el paciente por DNI.
3. Seleccionar el paciente deseado.
4. Visualizar las evaluaciones asociadas al visualizar el detalle del paciente.

Resultado esperado:
Se muestran las evaluaciones previas registradas para el paciente.
Resultado obtenido:
Historial mostrado correctamente con detalles de fecha y gravedad.
Evidencia:





2.1.10. Prueba CN – 10: Generación de reporte de evaluaciones de un paciente

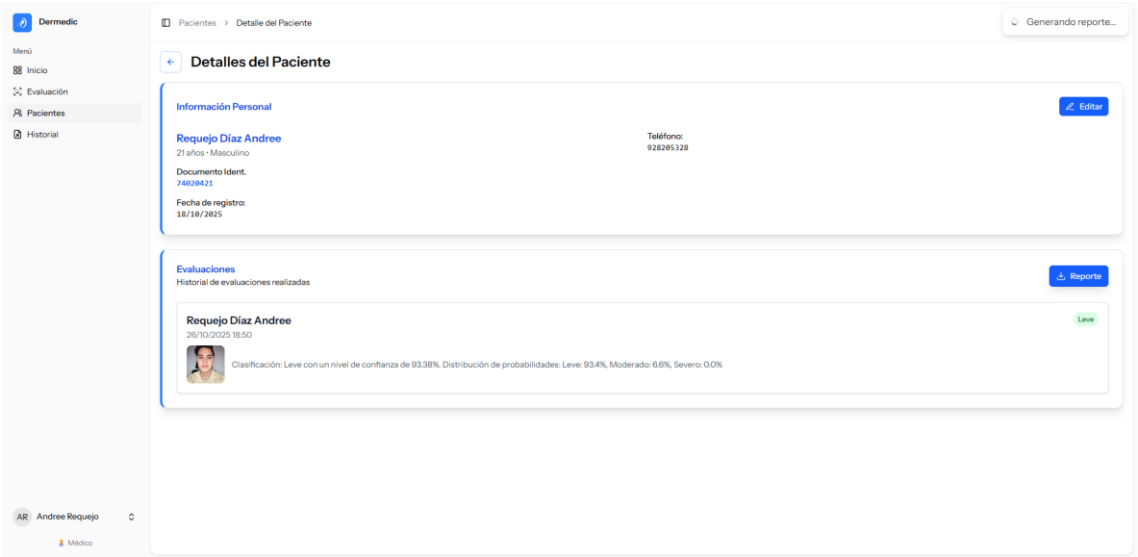
Módulo: Reportes
Descripción:
Verificar que el sistema genera correctamente el reporte de evaluaciones asociadas a un paciente seleccionado.
Datos de entrada:

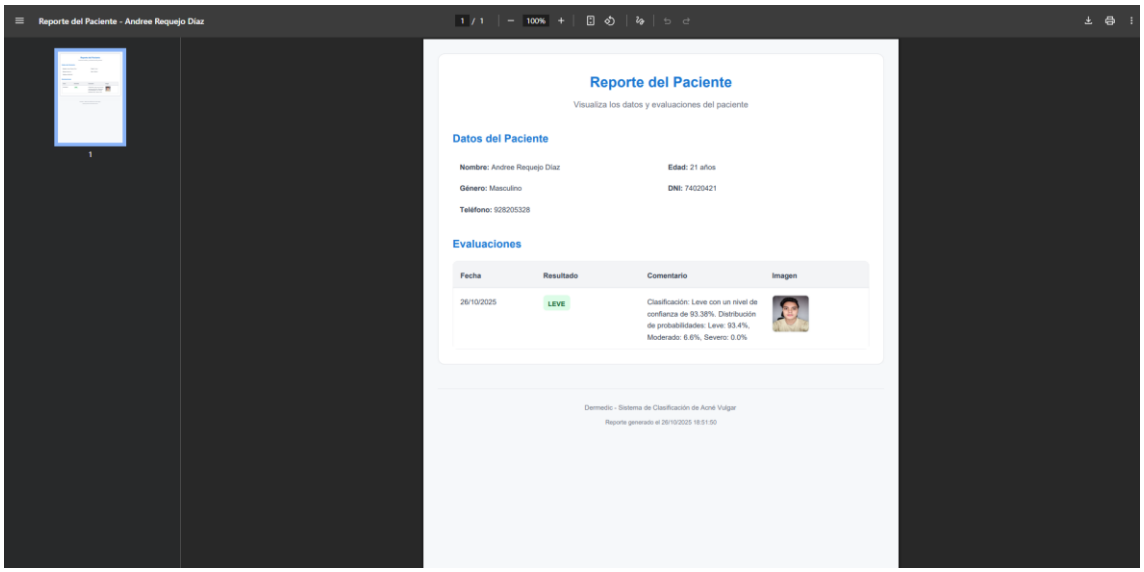
- Paciente con evaluaciones registradas previamente.
- Acción: Solicitud de generación de reporte desde el módulo “Reportes”.

Procedimiento:

1. Ingresar al sistema con credenciales válidas.
2. Acceder al detalle de un paciente y seleccionar el botón de **“Reporte”** desde la sección de evaluaciones del paciente.
3. Esperar la generación y descarga automática del archivo PDF.

Resultado esperado:
El sistema genera un reporte en formato PDF que incluye las evaluaciones registradas del paciente seleccionado.
Resultado obtenido:
El reporte se genera correctamente y contiene la información de las evaluaciones del paciente.
Evidencia:




Reporte del Paciente - Andree Requejo Díaz
1 / 1
100%
1
Reporte del Paciente
Visualiza los datos y evaluaciones del paciente
Datos del Paciente
Nombre: Andree Requejo Díaz
Edad: 21 años
Género: Masculino
DNI: 74020421
Teléfono: 928205328
Evaluaciones
Fecha
Resultado
Comentario
Imagen
26/10/2025
LEVE
Clasificación: Leve con un nivel de confianza de 93.38%. Distribución de probabilidades: Leve: 93.4%, Moderado: 6.6%, Severo: 0.0%
Dermedic - Sistema de Clasificación de Acné Vulgar
Reporte generado el 26/10/2025 18:51:50

2.2. Pruebas de caja blanca

ID	Módulo	Caso de Prueba	Descripción	Datos de Entrada	Resultado Esperado	Resultado Obtenido	Estado
CB-01	Autenticación	Validación del middleware de autenticación	Verificar que el middleware impida el acceso a rutas protegidas sin token o sesión válida.	Petición HTTP a una ruta protegida sin token de autenticación.	Redirección al login.	El middleware bloquea correctamente el acceso no autorizado.	Aprobado
CB-02	Autenticación	Cifrado de contraseñas al registrarse	Comprobar que las contraseñas se almacenen cifradas en la base de datos.	Registro de nuevo usuario con contraseña 159357An@.	En la base de datos, la contraseña se guarda con hash bcrypt.	Se verifica el cifrado correcto mediante bcrypt.	Aprobado
CB-03	Clasificación	Ejecución del modelo CNN	Verificar que la función que carga y ejecuta el modelo CNN devuelva un resultado válido.	Imagen de prueba	El modelo devuelve un json con los resultados de le evaluación	Resultado retornado correctamente	Aprobado
CB-04	Gestión de Pacientes	Validación de reglas de integridad	Verificar que los datos pasen las reglas de validación y se inserten correctamente.	Intentos voluntarios de fallos en el registro del paciente	El sistema retorna mensajes de error por cada fallo ingresado	Mensajes de error mostrados correctamente	Aprobado
CB-05	Gestión de pacientes	Consulta de evaluaciones por paciente	Validar que la consulta SQL solo retorne evaluaciones asociadas al paciente indicado.	ID de paciente: 12.	Devuelve lista con sus evaluaciones y no incluye registros ajenos.	Resultados filtrados correctamente.	Aprobado
CB-06	Gestión de pacientes	Validación de parámetros de búsqueda en el	Comprobar que el método aplica correctamente el filtro	Parámetro de búsqueda (por ejemplo: Andree)	Solo los pacientes con coincidencias son devueltos.	Se muestran correctamente los	Aprobado

		listado de pacientes	de búsqueda cuando se envía el parámetro search.			pacientes coincidentes	
CB-07	Reportes	Generación de reportes sin evaluaciones	Verificar que se redirija con mensaje de error si el paciente no tiene evaluaciones.	Solicitud de reporte en la aplicación	El sistema retorna un mensaje informando que el paciente no tiene evaluaciones	Mensaje mostrado correctamente	Aprobado
CB-08	Reportes	Detección de tipo MIME en imágenes base64	Verificar que la función detecte correctamente la extensión de un archivo base64	Imagen codificada en base64 obtenida desde la bd	El sistema muestra correctamente la imagen dentro del sistema	Imagen correctamente visualizada en el sistema	Aprobado
CB-09	Historial	Filtrado de evaluaciones	Validar que la función correspondiente cargue correctamente las evaluaciones según filtro.	Selección de filtro de severidad	El sistema filtra correctamente las evaluaciones relacionadas al tipo seleccionado	Evaluaciones filtradas correctamente según gravedad	Aprobado
CB-10	Clasificación	Compresión de imagen	Validar el método de compresión se realice correctamente y se retorne un Base64 funcional.	Imagen de prueba	El sistema comprime la imagen correctamente	Imagen comprimida	Aprobado

2.2.1. Prueba CB – 01: Validación del middleware de autenticación

Módulo: Autenticación
Descripción:
Verificar que el middleware de autenticación impida el acceso a rutas protegidas cuando el usuario no cuenta con una sesión activa.
Datos de entrada:

- Petición HTTP sin token de autenticación a una ruta protegida (/dashboard o /pacientes).
- Método de solicitud: GET.

Procedimiento:

1. Ingresar a el navegador sin haber iniciado sesión.
2. Enviar una solicitud **GET** a una ruta protegida del sistema (por ejemplo, /dashboard).
3. Observar la respuesta devuelta por el servidor.
4. Repetir la solicitud iniciando sesión previamente para comparar el comportamiento.

Resultado esperado:
El sistema debe impedir el acceso a la ruta protegida y redirigir automáticamente al formulario de inicio de sesión.
Resultado obtenido:
Al enviar la solicitud sin autenticación, el middleware intercepta correctamente las peticiones no autorizadas y redirecciona automáticamente al formulario de inicio de sesión. Cuando la sesión está activa, el acceso se concede normalmente al panel principal.
Evidencia:

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->redirectGuestsTo(fn (Request $request) => route('login'));
})
```

Figura 77 - Código para manejar redirección para sesiones no autenticadas

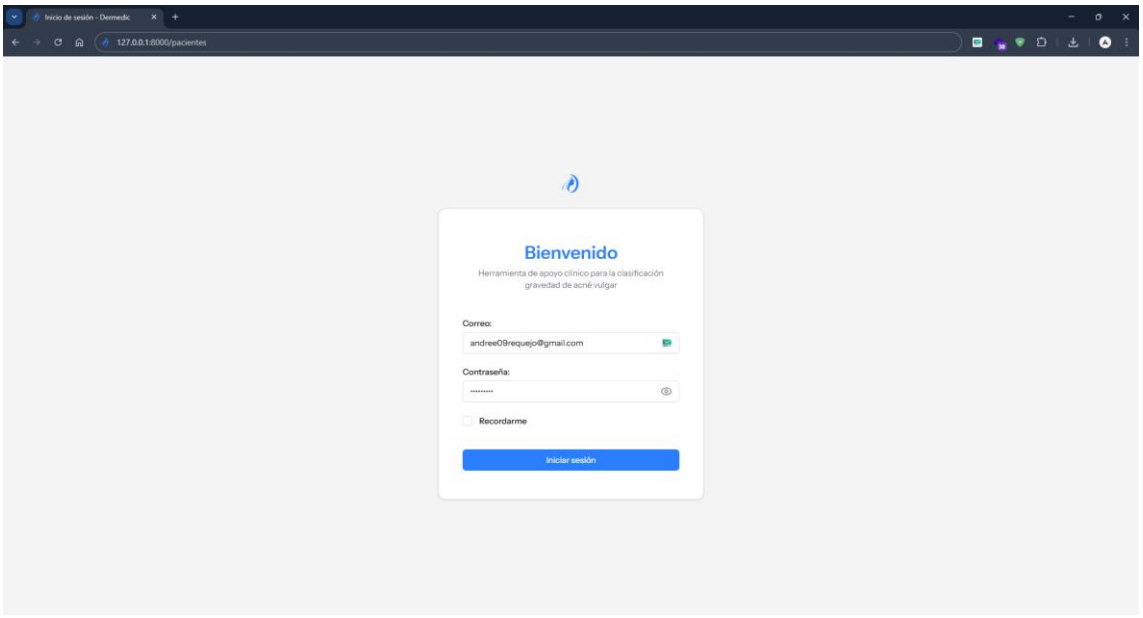


Figura 78 - Redirección a Inicio de sesión

2.2.2. Prueba CB – 02: Cifrado de contraseñas

Módulo: Autenticación
Descripción:
Comprobar que las contraseñas de los usuarios se almacenen de forma cifrada mediante **hash** al momento del registro, garantizando la seguridad de las credenciales.
Datos de entrada:
Seeder con los siguientes datos:

- Nombre: Andree Requejo
- Correo: andree09requejo@gmail.com
- Contraseña: 159357An@

Procedimiento:

1. Crear un nuevo usuario mediante el seeder correspondiente.
2. Acceder a la base de datos.
3. Consultar el campo password de la tabla users para el usuario recién creado.
4. Verificar si la contraseña se encuentra cifrada (no visible en texto plano).

Resultado esperado:
La contraseña se almacena en formato **hash**.
Resultado obtenido:
La contraseña registrada no se muestra en texto plano; se almacena en formato hash.
Evidencia:

```php
public function store(Request $request): RedirectResponse
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|lowercase|email|max:255|unique:'.User::class,
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    event(new Registered($user));

    Auth::login($user);

    return redirect()->intended(route('dashboard', absolute: false));
}
```

Figura 79 - Fragmento de código donde se realiza el hash a la contraseña

dermedic.users: 3 filas en total (exacto)

#	1 id	2 name	3 email	4 email_verified_at	5 password
1	1	Andree Requejo	andree09requejo@gmail.com	2025-10-18 02:14:10	$2y$12$YwGDXhEDbPakLC/GRUlfBen4Uq.pEe4m6y/AP/...

Figura 80 - Evidencia de encriptación de contraseña en base de datos

2.2.3. Prueba CB – 03: Ejecución del modelo CNN

Módulo: Clasificación
Descripción:
Verificar que el modelo de red neuronal convolucional (CNN) cargue correctamente y procese la imagen enviada, retornando un json con los datos asociados a la predicción.
Datos de entrada:
Imagen válida de un paciente (rostro con acné visible).
Procedimiento:

1. Inicializar el backend.
2. Ingresar una imagen de prueba desde postman.
3. Enviar la imagen al backend para su procesamiento.
4. Verificar que el modelo haya cargado correctamente y devuelto un resultado de sin errores.

Resultado esperado:
El modelo CNN procesa la imagen correctamente, devuelve la clasificación correspondiente y no se presentan excepciones durante la ejecución.
Resultado obtenido:
El modelo se carga sin errores y devuelve un resultado de clasificación con nivel de severidad “Moderado” para la imagen analizada.
Evidencia:

```
python main.py

INFO:     Will watch for changes in these directories: ['J:\\1. SEMINARIO DE TESIS II\\Proyecto\\backend_tesis']
INFO:     Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO:     Started reloader process [2672] using StatReload
INFO:     Started server process [21092]
INFO:     Waiting for application startup.
INFO:main:Iniciando servidor...
INFO:services.model_service:Modelo de acné cargado
```

Figura 81 - Log de backend con mensaje de éxito al iniciar el modelo CNN

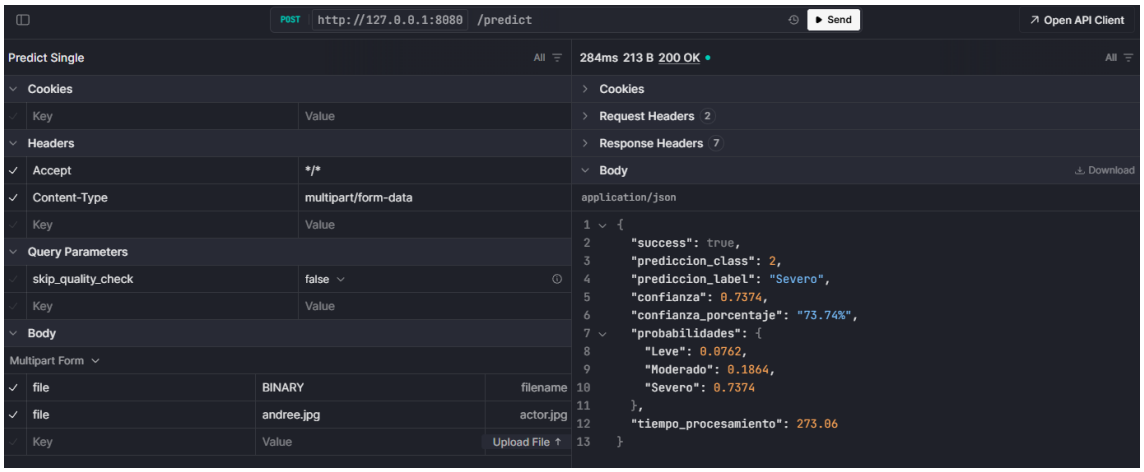


Figura 82 - Resultado de predicción

```python
@app.post("/predict", response_model=PredictionResponse, tags=["Prediction"])
async def predict_single(
    file: UploadFile = File(..., description="Imagen para clasificar"),
    skip_quality_check: bool = False
):
    start_time = datetime.utcnow()
    try:
        # Validaciones
        if not model_service.is_model_ready():
            raise HTTPException(
                status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
                detail="Modelo no disponible"
            )

        if not validate_image(file):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Tipo de archivo no válido: {file.content_type}"
            )

        # Procesar imagen
        image_bytes = await file.read()
        if len(image_bytes) == 0:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Archivo vacío"
            )

        # FLUJO DE VALIDACIÓN:
        # 1. MediaPipe: Detectar presencia y cantidad de rostros (filtro rápido inicial)
        # 2. ONNX Model: Validar si es imagen real vs animada (solo si hay 1 rostro)
        # 3. Validaciones de calidad: Blur, contraste, brillo (solo si es imagen real)
        # 4. Proceder al modelo principal de clasificación de acné

        # Si skip_quality_check es True, temporalmente deshabilitar validaciones de calidad
        original_quality_setting = IMAGE_QUALITY_CONFIG.get("enable_quality_checks", True)
        if skip_quality_check:
            IMAGE_QUALITY_CONFIG["enable_quality_checks"] = False

        try:
            face_valid, face_error_msg, face_confidence, validation_info = validate_face_in_image(
                image_bytes,
                model_service.get_onnx_session(),
                model_service.get_mtcnn()  # Pasar el detector MTCNN
            )

            if not face_valid:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail=face_error_msg
                )
        finally:
            # Restaurar configuración original
            IMAGE_QUALITY_CONFIG["enable_quality_checks"] = original_quality_setting

        image_tensor = process_image(image_bytes)

        # Predecir
        predicted_class, confidence, probabilities = model_service.predict(image_tensor, temperature=0.4)

        # Tiempo de procesamiento
        processing_time = (datetime.utcnow() - start_time).total_seconds() * 1000

        return PredictionResponse(
            success=True,
            prediccion_class=predicted_class,
            prediccion_label=CLASS_NAMES[predicted_class],
            confianza=round(confidence, 4),
            confianza_porcentaje=f"{confidence*100:.2f}%",
            probabilidades=probabilities,
            tiempo_procesamiento=round(processing_time, 2)
        )
    except HTTPException:
        raise
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=str(e)
        )
    except Exception as e:
        logger.error(f"Error en predicción: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Error interno: {str(e)}"
        )
```

Figura 83 - Código que se encarga de manejar la respuesta de una clasificación

2.2.4. Prueba CB – 04: Validación de reglas de integridad

Módulo: Gestión de Pacientes
Descripción:
Verificar que los datos del paciente pasen correctamente las reglas de validación definidas en el backend antes de insertarse en la base de datos, garantizando la integridad de la información almacenada.
Datos de entrada:
Formulario de registro de paciente con los siguientes campos de prueba:

- Nombre: Andree
- Apellidos: “Requejo”
- DNI: “123” (longitud incorrecta)
- Edad: “5” (valor inválido)
- Género: Masculino

Procedimiento:

1. Acceder al formulario de registro de paciente desde la interfaz web.
2. Ingresar los valores mencionados en los campos correspondientes.
3. Enviar el formulario presionando el botón “Registrar”.
4. Observar los mensajes retornados por el sistema.

Resultado esperado:
El sistema no permite el registro del paciente y muestra mensajes de error específicos por cada campo inválido, tales como:

- “El campo nombres es obligatorio.”
- “El DNI debe tener 8 dígitos numéricos.”
- “La edad debe ser un número válido mayor a 10.”

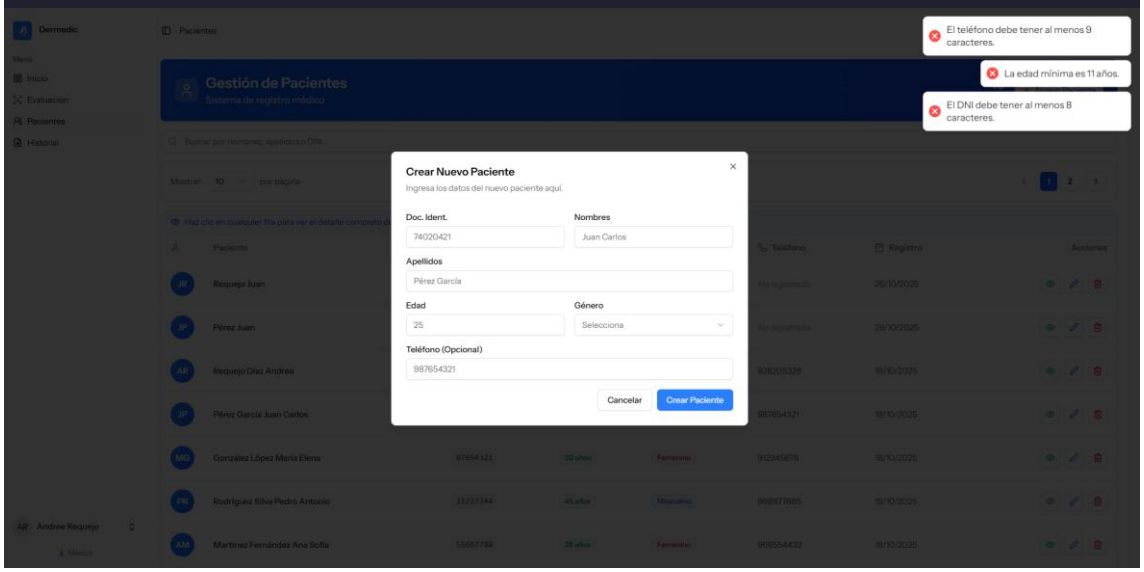
Resultado obtenido:
Los mensajes de error se muestran correctamente en pantalla y el registro no se guarda en la base de datos.
Evidencia:

```php
public function store(Request $request)
{
    $validated = $request->validate([
        'dni' => 'required|string|min:8|max:8|unique:paciente,dni',
        'nombres' => 'required|string|max:255',
        'apellidos' => 'required|string|max:255',
        'edad' => 'required|integer|min:11|max:30',
        'genero' => 'required|in:Masculino,Femenino',
        'telefono' => 'nullable|string|min:9|max:9|unique:paciente,telefono',
    ], [
        'dni.min' => 'El DNI debe tener al menos 8 caracteres.',
        'dni.unique' => 'El DNI ingresado ya se encuentra registrado.',
        'edad.min' => 'La edad mínima es 11 años.',
        'edad.max' => 'La edad máxima es 30 años.',
        'telefono.unique' => 'El teléfono ingresado ya se encuentra registrado.',
        'telefono.min' => 'El teléfono debe tener al menos 9 caracteres.',
        'genero.required' => 'Seleccione el género del paciente.'
    ]);

    // Si el teléfono está vacío, establecerlo como null
    if (empty($validated['telefono'])) {
        $validated['telefono'] = null;
    }

    Paciente::create($validated);

    return redirect()->back()->with('success', 'Paciente creado exitosamente.');
}
```



2.2.5. Prueba CB – 05: Consulta de evaluaciones por paciente

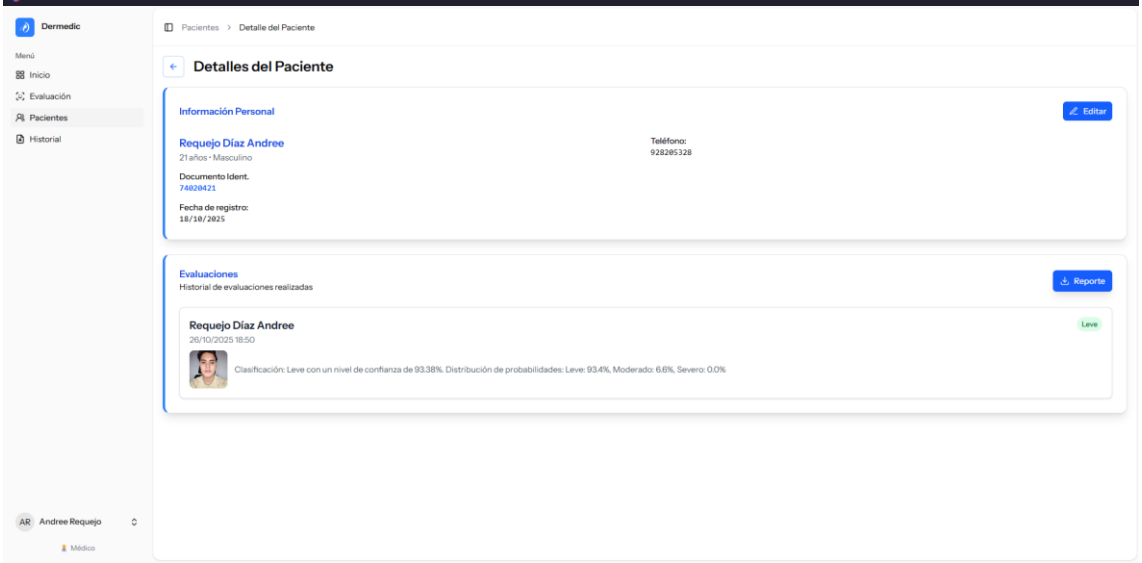
Módulo: Gestión de Pacientes
Descripción:
Validar que la consulta SQL que obtiene las evaluaciones asociadas a un paciente devuelva únicamente los registros pertenecientes al paciente indicado, garantizando la correcta relación entre las tablas *paciente* y *evaluacion*.
Datos de entrada:
ID de paciente: **12**
Procedimiento:

1. Acceder al módulo de “Gestión de Pacientes”.

2. Seleccionar el paciente con ID 12 desde la lista.
3. Consultar las evaluaciones registradas en su historial.
4. Confirmar que los resultados pertenezcan exclusivamente al paciente consultado.

Resultado esperado:
La consulta devuelve únicamente las evaluaciones que tienen el campo paciente_id = 12, sin incluir evaluaciones de otros pacientes.
Resultado obtenido:
Los resultados obtenidos en la interfaz y en la base de datos corresponden exclusivamente al paciente con ID 12.
Evidencia:

```php
public function show(Paciente $paciente)
{
    return Inertia::render('patients/detalle-patient', [
        'paciente' => $paciente->load(['evaluaciones.imagenPrincipal'])
    ]);
}
```



2.2.6. Prueba CB – 06: Validación de parámetros de búsqueda en el listado de pacientes

Módulo: Gestión de Pacientes
Descripción:
Comprobar que el método encargado de listar los pacientes aplique correctamente el filtro de búsqueda cuando se envía el parámetro search, retornando solo los registros que contengan coincidencias en los campos *nombres*, *apellidos* o *dni*.
Datos de entrada:
Parámetro de búsqueda: **“Andree”**
Procedimiento:
1. Acceder al módulo “Gestión de Pacientes”.
2. Ingresar el valor **“Andree”** en el campo de búsqueda.
3. Ejecutar la búsqueda para listar los resultados.
4. Confirmar que solo se muestran los registros con coincidencia parcial o total.

Resultado esperado:
El sistema muestra únicamente los pacientes cuyo nombre, apellido o DNI contenga la palabra “Andree”.
Resultado obtenido:
El listado devuelve correctamente solo los pacientes que cumplen con el criterio de búsqueda.
Evidencia:

```php
public function index(Request $request)
{
    $query = Paciente::query();

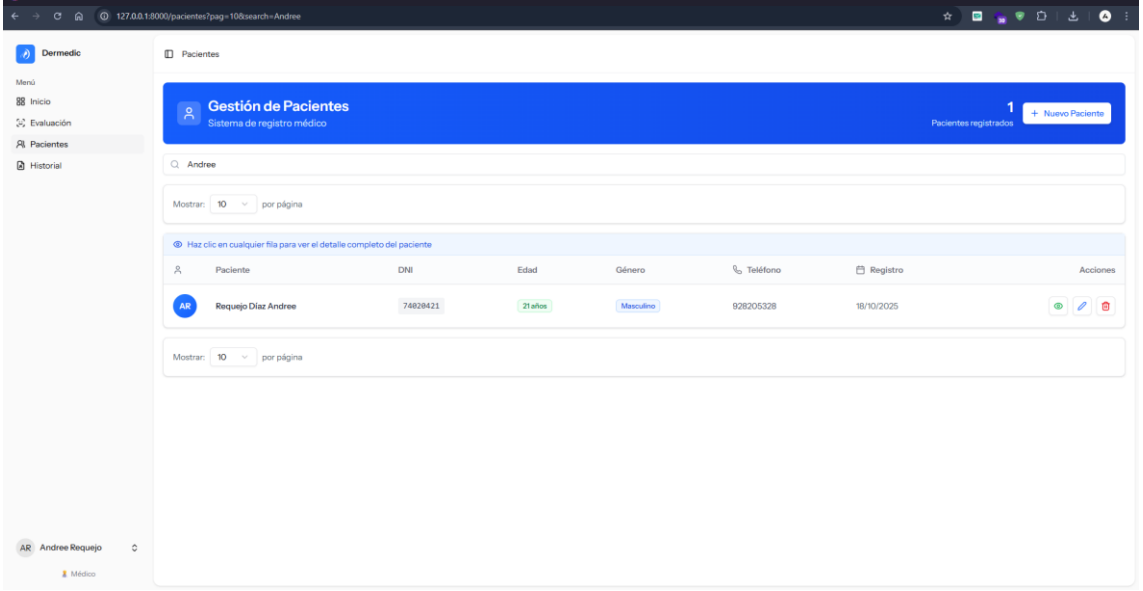
    // Búsqueda
    if ($request->has('search') && $request->search) {
        $search = $request->search;
        $query->where(function ($q) use ($search) {
            $q->where('nombres', 'like', "%{$search}%")
              ->orWhere('apellidos', 'like', "%{$search}%")
              ->orWhere('dni', 'like', "%{$search}%");
        });
    }

    // Paginación con parámetro personalizado 'pag'
    $perPage = $request->input('pag', 10); // Default 10 elementos por página

    // Validar que el valor esté dentro de los valores permitidos
    $allowedPerPage = [5, 10, 15, 25, 50];
    if (!in_array($perPage, $allowedPerPage)) {
        $perPage = 10;
    }

    $pacientes = $query->orderBy('created_at', 'desc')->paginate($perPage);

    return Inertia::render('patients/index', [
        'pacientes' => $pacientes,
        'filters' => $request->only(['search', 'pag'])
    ]);
}
```



2.2.7. Prueba CB – 07: Generación de reportes sin evaluaciones

Módulo: Reportes
Descripción:
Verificar que, al solicitar la generación de un reporte para un paciente que **no posee evaluaciones registradas**, el sistema redirija correctamente al usuario mostrando un mensaje de advertencia que indique que no existen evaluaciones disponibles para generar el reporte.
Datos de entrada:

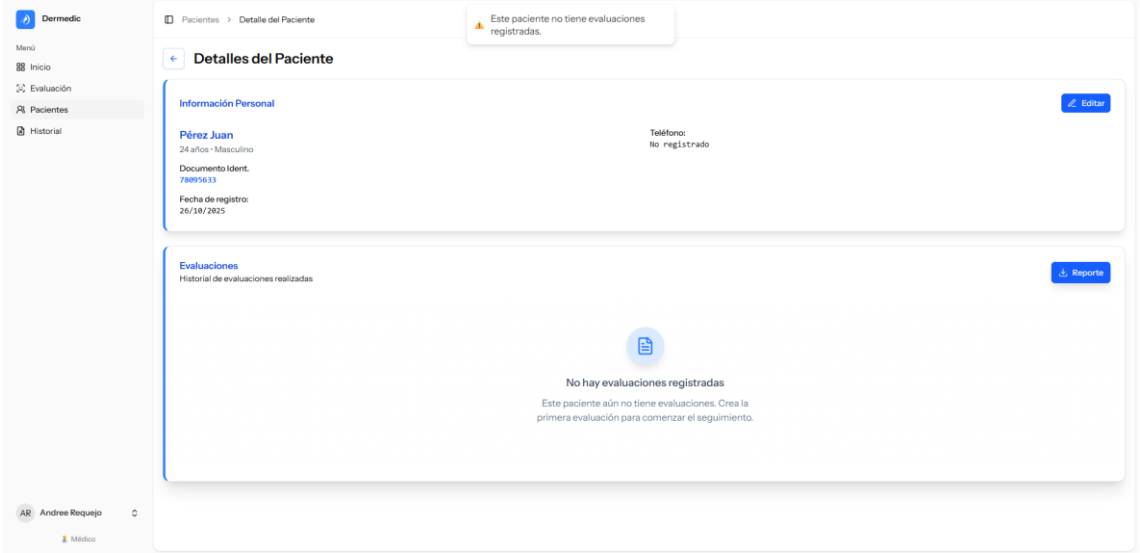
- ID del paciente: **15** (paciente sin evaluaciones en la base de datos).

Procedimiento:

1. Acceder al módulo **Paciente** desde la interfaz principal.
2. Seleccionar un paciente que no cuente con evaluaciones registradas.
3. Hacer clic en el botón **“Reporte”**.
4. Observar la respuesta del sistema y verificar si ocurre un mensaje en pantalla.

Resultado esperado:
El sistema detecta que el paciente no posee evaluaciones y muestra un mensaje como: “El paciente seleccionado no tiene evaluaciones registradas.”
Resultado obtenido:
El sistema redirige correctamente mostrando el mensaje de advertencia y evita la generación del reporte vacío.
Evidencia:

```php
// Si no tiene evaluaciones, redirigir con mensaje de error
if ($paciente->evaluaciones->isEmpty()) {
    return redirect()->back()->with('error', 'Este paciente no tiene evaluaciones registradas.');
}
```



2.2.8. Prueba CB – 08: Detección de tipo MIME en imágenes base64

Módulo: Reportes
Descripción:
Verificar que la función encargada de renderizar imágenes almacenadas en formato **Base64** detecte correctamente su tipo MIME y las muestre adecuadamente dentro de la interfaz o del reporte generado en PDF.
Datos de entrada:

- Imagen codificada en **Base64** obtenida desde la base de datos. Ejemplo: ...

Procedimiento:

1. Acceder a cualquier vista donde se visualizan las evaluaciones del paciente.
2. Seleccionar una evaluación que contenga imágenes almacenadas en formato Base64.
3. Inspeccionar el código fuente o la respuesta del servidor para confirmar que el campo contenido_base64 incluye el prefijo o .
4. Verificar que el método en el backend (o la función en el frontend) identifique correctamente el tipo MIME y lo use al mostrar la imagen.
5. Observar si la imagen se carga y visualiza correctamente en pantalla o dentro del PDF generado.

Resultado esperado:
El sistema interpreta correctamente el tipo de archivo según el prefijo MIME (image/jpeg, image/png, etc.) y renderiza la imagen sin errores.

Resultado obtenido:
La imagen se visualiza correctamente dentro de la interfaz y los reportes, confirmando que la detección del tipo MIME es funcional.

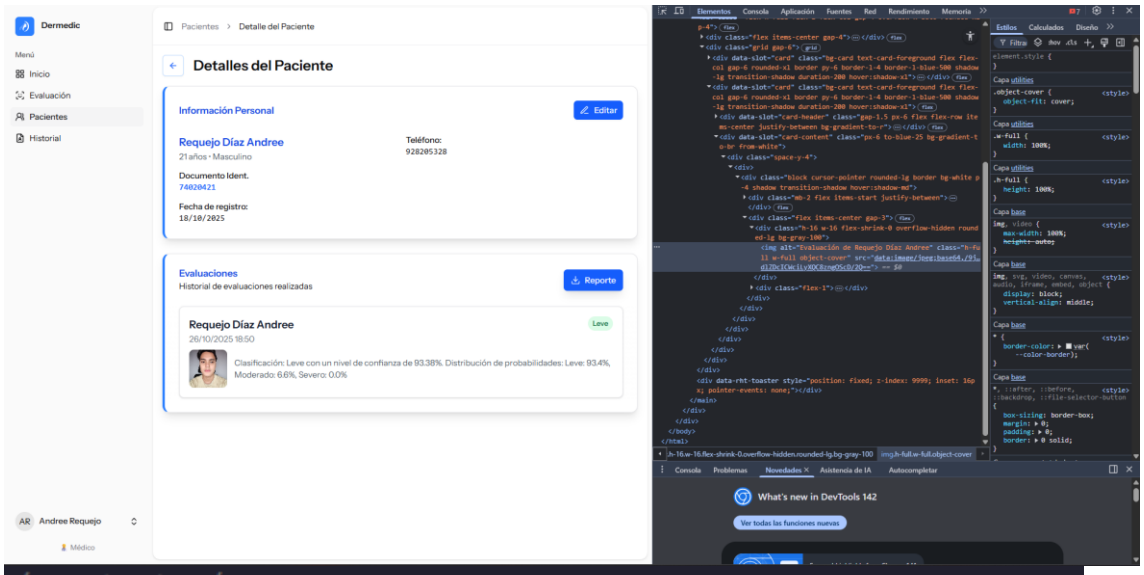
Evidencia:

```php
/**
 * Detecta el tipo MIME de una imagen basándose en su contenido base64
 */
private function detectMimeType($base64Content)
{
    // Decodificar el contenido base64 para obtener los primeros bytes
    $imageData = base64_decode($base64Content);

    // Obtener información de la imagen
    $finfo = new \finfo(FILEINFO_MIME_TYPE);
    $mimeType = $finfo->buffer($imageData);

    // Validar que sea una imagen válida
    $validMimeTypes = [
        'image/jpeg' => 'image/jpeg',
        'image/jpg' => 'image/jpeg',
        'image/png' => 'image/png',
        'image/gif' => 'image/gif',
        'image/webp' => 'image/webp',
        'image/bmp' => 'image/bmp'
    ];

    return isset($validMimeTypes[$mimeType]) ? $validMimeTypes[$mimeType] : 'image/png';
}
```




```php
/** @test */
public function detect_mime_type_reconoce_jpeg_y_png()
{
    $controller = new PacienteController();
    $reflection = new ReflectionClass($controller);
    $method = $reflection->getMethod('detectMimeType');
    $method->setAccessible(true);

    // Imagen JPEG
    $jpeg = imagecreatetruecolor(10, 10);
    ob_start();
    imagejpeg($jpeg);
    $jpegData = ob_get_clean();
    $base64Jpeg = base64_encode($jpegData);

    $mimeJpeg = $method->invoke($controller, $base64Jpeg);
    $this->assertEquals('image/jpeg', $mimeJpeg);

    // Imagen PNG
    $png = imagecreatetruecolor(10, 10);
    ob_start();
    imagepng($png);
    $pngData = ob_get_clean();
    $base64Png = base64_encode($pngData);

    $mimePng = $method->invoke($controller, $base64Png);
    $this->assertEquals('image/png', $mimePng);
}
```

```
PASS  Tests\Unit\Controllers\PacienteControllerTest
✓ detect mime type reconoce jpeg y png                    0.02s
```

2.2.9. Prueba CB – 09: Filtrado de evaluaciones

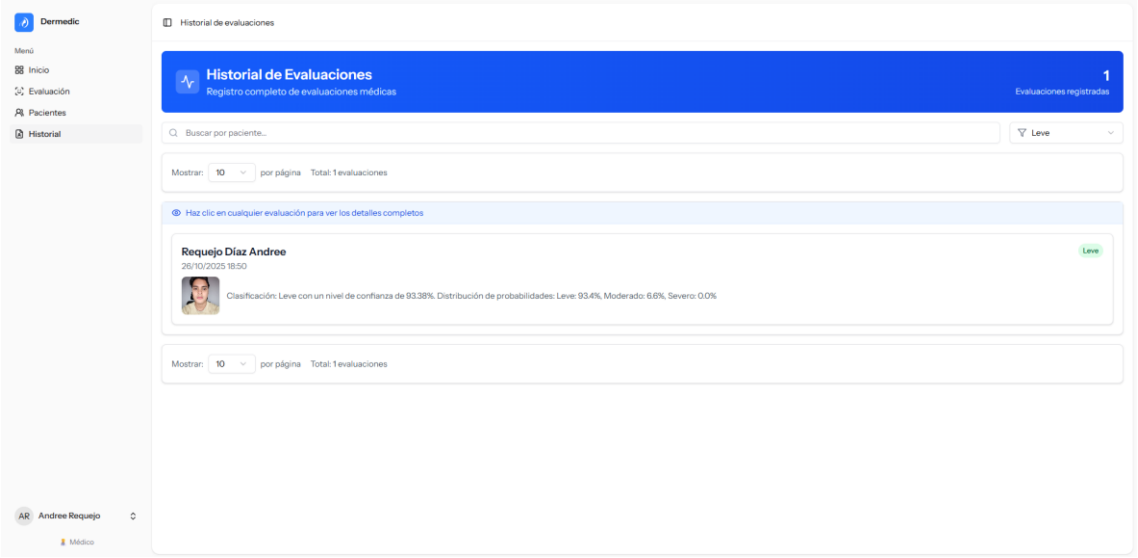
Módulo: Evaluaciones
Descripción:
Verificar que el sistema permita filtrar correctamente las evaluaciones de pacientes según el grado de severidad del acné (leve, moderado o severo), mostrando solo los registros correspondientes al filtro seleccionado.
Datos de entrada:

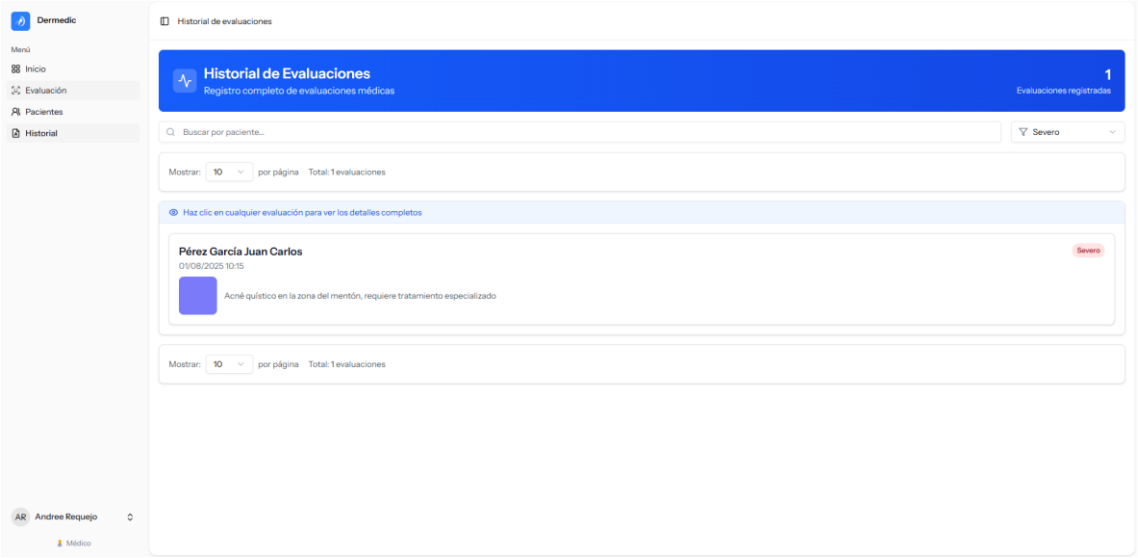
- Filtro aplicado: “Leve”, “Moderado”, “Severo”.

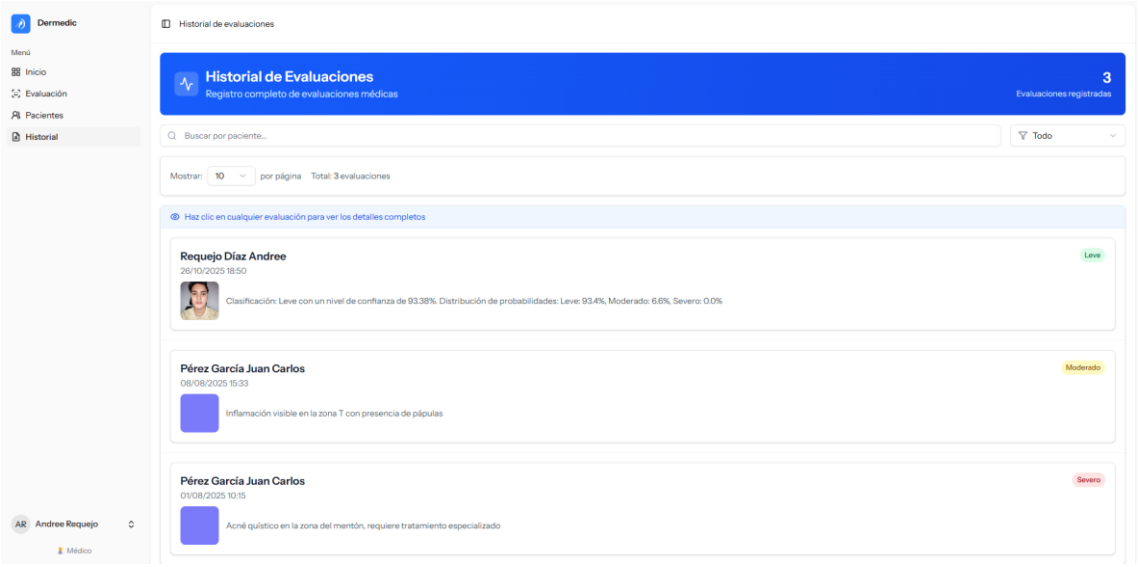
Procedimiento:

1. Acceder al módulo **Evaluaciones**.
2. Seleccionar una opción del filtro de **Grado de severidad**.
3. Observar la lista de evaluaciones listadas.
4. Verificar que únicamente se muestren las evaluaciones con el grado de severidad seleccionado.
5. Cambiar entre las diferentes opciones del filtro y comprobar que los resultados se actualicen dinámicamente.
6. Quitar el filtro (si existe opción “Todos”) y confirmar que se muestren nuevamente todas las evaluaciones registradas.

Resultado esperado:
El sistema muestra correctamente las evaluaciones según el grado de severidad seleccionado y restaura la lista completa al quitar el filtro.
Resultado obtenido:
El filtrado por grado de severidad funciona correctamente, mostrando los registros correspondientes al filtro aplicado y restaurando la lista completa al eliminar el filtro.
Evidencia:







2.2.10. Prueba CB – 10: Compresión de imagen

Módulo: Clasificación
Descripción:
Validar que el método de compresión de imagen funcione correctamente, reduciendo el tamaño del archivo sin pérdida significativa de calidad y retornando una cadena Base64 funcional que pueda ser utilizada en los procesos de clasificación.
Datos de entrada:

- Imagen de prueba en formato JPG.

Procedimiento:

1. Cargar la imagen de prueba en el método de compresión.
2. Ejecutar la función encargada de comprimir la imagen.
3. Verificar que la función retorne una cadena Base64 válida.
4. Convertir nuevamente la cadena Base64 a imagen para comprobar su integridad.
5. Comparar el tamaño del archivo original con el comprimido.

6. Evaluar que la imagen mantenga una calidad visual aceptable tras la compresión.

Resultado esperado:
El método genera correctamente una cadena Base64 funcional que representa la imagen comprimida, reduciendo su tamaño y conservando una calidad visual adecuada.
Resultado obtenido:
La imagen fue comprimida exitosamente. La cadena Base64 generada fue válida y permitió visualizar la imagen correctamente sin pérdida apreciable de calidad.
Evidencia:

```php
private function comprimirImagen($imagenBase64, $calidadJpeg = 85, $maxWidth = 800, $maxHeight = 600)
    {
        try {
            // Decodificar la imagen base64
            $imageData = base64_decode(preg_replace('/^data:image\/[^;]+; base64,/', '', $imagenBase64));

            // Crear imagen desde string
            $image = imagecreatefromstring($imageData);
            if (!$image) {
                throw new \Exception('No se pudo crear la imagen desde los datos base64');
            }

            // Obtener dimensiones originales
            $originalWidth = imagesx($image);
            $originalHeight = imagesy($image);

            // Calcular nuevas dimensiones manteniendo la proporción
            $ratio = min($maxWidth / $originalWidth, $maxHeight / $originalHeight);

            // Solo redimensionar si la imagen es más grande que los límites
            if ($ratio < 1) {
                $newWidth = (int)($originalWidth * $ratio);
                $newHeight = (int)($originalHeight * $ratio);

                // Crear nueva imagen redimensionada
                $resizedImage = imagecreatetruecolor($newWidth, $newHeight);

                // Mantener transparencia para PNG
                imagealphablending($resizedImage, false);
                imagesavealpha($resizedImage, true);

                // Redimensionar
                imagecopyresampled(
                    $resizedImage, $image,
                    0, 0, 0, 0,
                    $newWidth, $newHeight,
                    $originalWidth, $originalHeight
                );

                imagedestroy($image);
                $image = $resizedImage;
            }

            // Comprimir y convertir a JPEG
            ob_start();
            imagejpeg($image, null, $calidadJpeg);
            $compressedData = ob_get_contents();
            ob_end_clean();

            // Limpiar memoria
            imagedestroy($image);

            return base64_encode($compressedData);

        } catch (\Exception $e) {
            Log::error('Error al comprimir imagen: ' . $e->getMessage());
            // Si falla la compresión, retornar la imagen original
            return preg_replace('/^data:image\/[^;]+; base64,/', '', $imagenBase64);
        }
    }
```

```php
/** @test */
public function comprimir_imagen_devuelve_base64_jpeg()
{
    // Crear una imagen PNG de prueba
    Storage::fake('local');
    $file = UploadedFile::fake()->image('test.png', 1000, 800);
    $base64 = base64_encode(file_get_contents($file->getRealPath()));

    $controller = new EvaluacionController();
    $reflection = new ReflectionClass($controller);
    $method = $reflection->getMethod('comprimirImagen');
    $method->setAccessible(true);

    $base64Jpeg = $method->invoke($controller, $base64);

    // Debe devolver un string base64 diferente (comprimido)
    $this->assertIsString($base64Jpeg);
    $this->assertNotEquals($base64, $base64Jpeg);

    // Decodificar y verificar que es una imagen JPEG
    $imageData = base64_decode($base64Jpeg);
    $finfo = new \finfo(FILEINFO_MIME_TYPE);
    $mime = $finfo->buffer($imageData);
    $this->assertEquals('image/jpeg', $mime);
}
```

```
PASS Tests\Unit\Controllers\EvaluacionControllerTest
✓ comprimir imagen devuelve base64 jpeg                    0.23s
```